

```
QMRITools`$Verbose = False;
```

```
<< QMRITools`
```

```
In[15]:= QMRITools`$InstalledVersion
```

```
Grid[Transpose@{"Packages", "Functions", "Options"}, Length@Flatten[QMRIToolsFunctions[]][[All, #]] & /@ {1, 2, 3}], Alignment → Left]
```

```
Out[15]=
```

```
4.8.0
```

```
Out[16]=
```

```
Packages  28  
Functions 594  
Options   522
```

```
In[21]:= Column@QMRIToolsPackages[]
```

```
Out[21]=
```

```
CardiacTools  
CoilTools  
DenoiseTools  
DixonTools  
ElastixTools  
FasciculationTools  
GeneralTools  
GradientTools  
IVIMTools  
JcouplingTools  
Legacy  
LoggingTools  
MaskingTools  
MuscleBidsTools  
NeuralNetworkTools  
NiftiTools  
PlottingTools  
ProcessingTools  
ReconstructionTools  
RelaxometryTools  
ScientificColorData  
SegmentationTools  
ShapeTools  
SimulationTools  
SpectroTools  
TaggingTools  
TensorTools  
TractographyTools
```

```
In[25]:= QMRIToolsFunctions["All", 4]
```

QMRITools

ActivationLayer	ExportTCK	MakeChannelClassGrid	ReadVoxSize
ADCCalc	ExportTracts	MakeChannelClassImage	RegisterCardiacData
AddLossLayer	ExportVol	MakeChannelImage	RegisterData
AddNoise	ExpTensor	MakeCheckFile	RegisterDataSplit
AddScientificColors	ExtractColorData	MakeClassifyImage	RegisterDataTransform
AddToLog	ExtractDemoData	MakeClassifyNetwork	RegisterDataTransformSplit
AlignResPlog	ExtractNiiFiles	MakeClassImage	RegisterDiffusionData
AnalyzeActivations	FACalc	MakeCoilLayout	RegisterDiffusionDataSplit
AnalyzeNetworkFeatures	FConvert	MakeCoordinates	RegisterTensorData
AngleCalc	FConverti	MakeDistanceMap	RemoveIsoImages
AngleMap	FiberDensityMap	MakeECVBloodMask	RemoveMaskOverlaps
AnisoFilterData	FiberLengths	MakeFunctionGraph	ReplaceSegmentations
AnisoFilterTensor	FiberTractography	MakeGradientDerivatives	ResampleTracts
AnnalyzeTagging	FileSelect	MakeGradientMaps	RescaleData
ApodizeEcho	FilterTractLength	MakeHammingFilter	RescaleSegmentation
ApodizeFid	FilterTracts	MakeIntFunction	RescaleTracts
ApodizePadEcho	FinalGrads	MakeLineImage	ResetLog
ApodizePadFid	FindActivations	MakeMaskImage	ResidualCalc
ApodizePadSpectra	FindCoilPosition	MakeMuscleTemplate	ReverseCrop
ApodizeSpectra	FindCrop	MakeNiiOrientationQ	ReverseDimensions
ApplyCrop	FindInPhaseEchos	MakeNiiOrientationS	RMSNoZero
ApplySegmentationNetwork	FindMaxDimensions	MakeNode	ROIMask
ApplyShapeModel	FindMiddle	MakeNoisePlots	RotateData
AugmentImageData	FindOrder	MakeRegionMesh	RotateDimensionsLeft
AugmentTrainingData	FindOutliers	MakeSense	RotateDimensionsRight
AutoCropData	FindSpectraPpmShift	MakeShapeModel	RotateTensor
AverageKspace	FindTensorPermutation	MakeSliceImages	RotationMatrixToQuaternion
B1MapCalc	FitData	MakeSpectraResultPlot	RotationMatrixToQuaternionVector
B1Shimming	FitGradientMap	MakeSpinSystem	RPBMFunction
BayesianIVIMFit2	FitRPBMDictionary	MakeTemplatePlot	SagittalTranspose
BayesianIVIMFit3	FitRPBMFunction	MakeUnet	SaveImage
BidsDcmToNii	FitShapeModel	MakeWeightMask	ScaleToVolume
BlochSeries	FitSpectra	Mask	SeedDensityMap
Bmatrix	FitSpectraResultTable	MaskData	SegmentationCrossSection
BmatrixCalc	FitTracts	MaskFromDistanceMap	SegmentationVolume
BmatrixConv	FixDixonFlips	MaskHelix	SegmentData
BmatrixInv	FlipGradientOrientation	MaskSegmentation	SegmentDataGUI
BmatrixRot	FlipTensorOrientation	MaskToDistanceMap	SegmentLinesToMask
BmatrixToggle	FocalLossLayer	MaskToLines	SegmentMask
BSplineCurveFit	FourierKspace2D	MaskVolume	SegmentsPerSlice
BullseyePlot	FourierKspace3D	MBCount	SegmentTracts
BvalRead	FourierKspaceCSI	MeanBvalueSignal	SelectActivations
BVector	FourierRescaleData	MeanNoZero	SelectBidsFolders
CalculateDisplacementParameters	FourierShift	MeanRange	SelectBidsSessions
CalculateGfactor	FourierShifted	MeanSignal	SelectBidsSubjects
CalculateMoments	FracCorrect	MeanStd	SelectBvalueData
CalculateWallMap	FullGrad	MeanStdRange	SelectMaskComponents
CalibrateEPGT2Fit	GenerateAmps	MeanType	SelectSegmentations
CardiacCoordinateSystem	GenerateBidsFileName	MedCouple	SelectSubjects
CardiacSegment	GenerateBidsFolderName	MedFilter	SequencePulseAcquire
CardiacSegmentGUI	GenerateBidsName	MedianNoZero	SequenceSpaceEcho
CELossFunction	GenerateGradients	MemoryUsage	SequenceSpinEcho
CELossLaver	GenerateGradientsGUI	MergeConfig	SequenceSteam

Options

AcquisitionMethod	DixonRelaxivity	MeshOpacity	RescaleRecon
ActivationBackground	DixonTolerance	MeshOutput	Resolutions
ActivationIterations	DownsampleSchedule	MeshPointColor	ResolutionsA
ActivationOutput	DropoutRate	MeshPoints	ReverseData
ActivationSize	DropSamples	MeshPointSize	ReverseDirection
ActivationThreshold	DropSlices	Method	ReversePoints
ActivationType	EchoShiftData	MethodReg	ReverseSets
AffineDirections	EPGCalibrate	MethodRegA	RobustFit
AllowSelfDependencies	EPGFatShift	Monitor	RobustFitParameters
AnisoFilterSteps	EPGFitFat	MonitorCalc	RotateGradient
AnisoIterations	EPGFitPoints	MonitorInterval	RotateGradients
AnisoKappa	EPGMethod	MonitorTagging	RotationCorrect
AnisoKernel	EPGMethodCal	MonitorUnwrap	RoundLength
AnisoStepTime	EPGOutput	MotionCorrectSets	RowSize
AnisoWeightType	EPGRefocussingPhase	MultiChannel	RPBMDictionarySize
ApodizationFunction	EPGRelaxPars	NetworkArchitecture	RPBMRangeTau
AspectRatio	EPGSmoothB1	NetworkDepth	RPBMRangeZeta
Augment2D	FatFieldStrength	NiiDataType	Runs
AugmentData	FeatureSchedule	NiiLegacy	SampleStep
Axes	FiberAngle	NiiMethod	SaveLogFile
AxesLabel	FiberLengthRange	NiiOffset	ScaleCorrect
AxesMethod	FieldStrength	NiiScaling	Scaling
B1EqualPower	FileType	NiiSliceCode	SeedDensity
B1FilterData	Filling	NoisePrewiten	SegmentationDimension
B1Masking	FilterInternal	NoiseSize	SegmentationMethod
B1MaxPower	FilterMaps	NoiseType	SegmentationResolution
B1Output	FilterShape	NormalizationType	SegmentationsPerSlice
B1Scaling	FilterSize	NormalizeDensity	SenseSmoothing
B1ShimMethod	FilterType	NormalizeIVIM	SenseWeight
BackgroundValue	FindTransform	NormalizeMethod	SettingSchedule
Bandwidth	FineTuneFit	NormalizeOutputSpectra	ShowMetric
BasisSequence	FitConstraints	NormalizeOverlap	ShowOutliers
BatchSize	FitFunction	NormalizeSets	ShowPlot
BidsIncludeSession	FitKernel	NormalizeSignal	SimNucleus
BidsOutputImages	FitLineShape	NumberSamples	SliceRange
BidsTractographyMethod	FitOrder	NumberSamplesA	SliceRangeSamples
BinaryType	FitOutput	OrderSpan	SmartMaskOutput
BlockType	FittingOrder	OutlierCheckNormality	SmartMethod
BloodMaskRange	FitTractSegments	OutlierIncludeZero	SmoothHelix
BmatrixOut	FixPseudoDiff	OutlierIterations	SmoothIterations
Boxed	FixPseudoDiffSD	OutlierMethod	SmoothSNR
BsplineDirections	FlipAxes	OutlierOutput	SortVecs
BsplineSpacing	FlipBvec	OutlierRange	SparID
BullPlotMethod	FlipGrad	OutputCalibration	SparName
CatenateMethod	Frame	OutputCheckImage	SparOrientation
CenterFrequency	FrameStyle	OutputCoilSurface	SpectraBandwidth
CenterRange	FullOutput	OutputForm	SpectraFieldStrength
CenterVoxel	FullSphere	OutputImage	SpectraNucleus
ChainSteps	GetMaskOnly	OutputLabels	SpectraOutputPlots
Channels	GetMaskOutput	OutputMethod	SpectraPpmShift
Classes	GOutput	OutputPlot	SpectraSamples
CleanupSegmentations	GradientStepSize	OutputSamples	SpectraSpacing
ClusterAnalysis	GradientTime	OutputScales	SubtractScales

CardiacTools

Functions

BullseyePlot	ECVCalc	MakeECVBloodMask	RadialSample
CalculateWallMap	ExcludeSlices	MakeLineImage	SegmentLinesToMask
CardiacCoordinateSystem	GetMaskSegmentPoints	MakeMaskImage	SegmentsPerSlice
CardiacSegment	GetSegmentLines	MaskHelix	TransmuralPlot
CardiacSegmentGUI	GetSegmentSlices	MaskToLines	
CentralAxes	HelixAngleCalc	PlotSegmentMask	
CreateHeart	LinesToSegmentIndex	PlotSegments	

Options

AxesMethod	GridLineSpacing	PlotLabel	ShowOutliers
BackgroundValue	GroupPerSegment	PlotRange	ShowPlot
BloodMaskRange	ImageSize	PlotStyle	SmoothHelix
BullPlotMethod	LCMMethod	RadialSamples	StartPoints
ColorFunction	MakeSegmentPlots	ReverseDirection	StartSlices
CutOffMethod	MaskWallMap	ReversePoints	TextNumberForm
DistanceMeasure	Method	RowSize	TextOffset
DropSamples	OutputCheckImage	SegmentationMethod	TextSize

CoilTools

Functions

CoilSNRCalc	LoadCoilSetup	MakeCoilLayout	MakeWeightMask
FindCoilPosition	LoadCoilTarget	MakeNoisePlots	

Options

CoilArrayPlot	ColorFunction	OutputCoilSurface
CoilSurfaceVoxelSize	ImageSize	PlotRange

DenoiseTools

Functions

AnisoFilterData
AnisoFilterTensor
DeNoise

DenoiseCSIdata
DenoiseDynamicSpectraData
HarmonicDenoiseTensor

NNThreshold
P2SDenoise
PCADeNoise

WeightMapCalc
\$debugDenoise

Options

AnisoFilterSteps
AnisoIterations
AnisoKappa
AnisoKernel
AnisoStepTime
AnisoWeightType
DeNoiseIterations
DeNoiseKernel

DeNoiseMonitor
GradientStepSize
Iterations
MaxIterations
Method
Monitor
MonitorCalc
Parallelize

PCAclipping
PCAComplex
PCAKernel
PCANoiseSigma
PCAOutput
PCATolerance
PCAWeighting
RadialBasisKernel

RangeFA
RangeMD
TensorFlips
TensorPermutations
Threshold
Tolerance

DixonTools

Functions

DixonPhase
DixonReconstruct
DixonToPercent
FindInPhaseEchos

FixDixonFlips
GenerateAmps
OptimizeDixonEcho
SimulateDixonSignal

Unwrap
UnwrapDCT
UnwrapList
UnwrapSplit

Wrap

Options

DixonAmplitudes
DixonBipolar
DixonClipFraction
DixonConstrainPhase
DixonCorrectT1
DixonFieldStrength
DixonFilterInput

DixonFilterOutput
DixonFilterSize
DixonFilterType
DixonFitPhase
DixonFixT2
DixonFrequencies
DixonIterations

DixonMaskThreshold
DixonNucleus
DixonPhases
DixonPrecessions
DixonRelaxivity
DixonTolerance
MaxIterations

MonitorCalc
MonitorUnwrap
Parallelize
PhaseEchos
UnwrapDimension
UnwrapThresh

ElastixTools

Functions

ClearQMRIToolsTemp	RegisterDataSplit	RegisterDiffusionDataSplit	\$lastElastixTemp
ReadTransformParameters	RegisterDataTransform	RegisterTensorData	
RegisterCardiacData	RegisterDataTransformSplit	TransformData	
RegisterData	RegisterDiffusionData	\$debugElastix	

Options

AffineDirections	InterpolationOrderReg	NumberSamplesA	ShowMetric
BsplineDirections	InterpolationOrderRegA	OutputImage	SplitMethod
BsplineSpacing	Iterations	OutputTransformation	TempDirectory
DeleteTempDirectory	IterationsA	PCAComponents	TransformMethod
FindTransform	Method	PrintTempDirectory	UseGPU
HistogramBins	MethodReg	RegistrationTarget	
HistogramBinsA	MethodRegA	Resolutions	
ImportDeformation	NumberSamples	ResolutionsA	

FasciculationTools

Functions

AnalyzeActivations	FindActivations	SelectActivations
EvaluateActivation	NormalizeFasciculationData	

Options

ActivationBackground	ActivationOutput	ActivationThreshold	MaskDilation
ActivationIterations	ActivationSize	IgnoreSlices	ThresholdMethod

GeneralTools

Functions

ApplyCrop	FindCrop	MedianNoZero	RMSNoZero
AutoCropData	FindMaxDimensions	MemoryUsage	RotatedDimensionsLeft
BSplineCurveFit	FindMiddle	MonitorFunction	RotatedDimensionsRight
ClearTemporaryVariables	GetAssetLocation	NiiFileExistQ	RotationMatrixToQuaternion
CompilableFunctions	GridData	NNLeastSquares	RotationMatrixToQuaternionVector
ConvertExtension	GridData3D	OpenDemonstrationNotebook	SaveImage
CropData	GyromagneticRatio	OpenQMRIToolsLocation	SetDemoDirectory
CutData	LapFilter	PadToDimensions	SignNoZero
DataToVector	LightDarkV	ParseCommandLine	Squeeze
DateName	LLeastSquares	QMRIToolsFuncPrint	StandardDeviationNoZero
DecomposeAffineMatrix	LogNoZero	QMRIToolsFunctions	StdFilter
DecomposeScaleMatrix	MADNoZero	QMRIToolsPackages	StichData
DivideNoZero	MakeCoordinates	QuaternionToRotationMatrix	StringPadInteger
DynamicPartition	MakeFunctionGraph	QuaternionVectorToRotationMatrix	SumOfSquares
EmptyDirectoryQ	MakeIntFunction	RandomString	TensMat
ExpNoZero	MBCount	RescaleData	TensVec
ExtractDemoData	MeanNoZero	ReverseCrop	VectorToData
FileSelect	MedFilter	ReverseDimensions	

Options

AllowSelfDependencies	CropPadding	LabelPlacement	SplineDegree
CenterRange	FileType	Monitor	SplineKnotsNumber
CenterVoxel	FilterInternal	OutputWeights	SplineRegularization
CropAlways	ImageResolution	Padding	WindowTitle
CropInit	ImageSize	PadDirection	
CropOutput	InterpolationOrder	PadValue	

GradientTools

Functions

Bmatrix
BmatrixCalc
BmatrixConv
BmatrixInv
BmatrixRot
BmatrixToggle
BVector
CalculateMoments

ConditionNumberCalc
ConvertGrads
CorrectBmatrix
CorrectGradients
EnergyCalc
FinalGrads
FindOrder
FullGrad

GenerateGradients
GenerateGradientsGUI
GetGradientScanOrder
GetSliceNormal
GetSliceNormalDir
GradBmatrix
GradientCoilTensor
GradSeq

ImportGradObj
MakeGradientDerivatives
MakeGradientMaps
OverPlusCalc
SelectBvalueData
UniqueBvalPosition

Options

ConditionCalc
FlipAxes
FlipGrad
FullSphere
GradType

Method
MethodReg
OrderSpan
OutputPlot
OutputType

PhaseEncoding
Runs
Steps
StepSizeI
SwitchAxes

UnitMulti
UseGrad
VisualOpt

IVIMTools

Functions

BayesianIVIMFit2
BayesianIVIMFit3
CorrectParMap
FConvert

FConverti
FracCorrect
HistogramPar
IVIMCalc

IVIMCorrectData
IVIMFunction
IVIMResiduals
IVIMTensFit

MeanBvalueSignal
ThetaConv
ThetaConvi

Options

ChainSteps
CorrectPar
FilterMaps
FilterSize

FilterType
FitConstrains
FixPseudoDiff
FixPseudoDiffSD

IVIMComponents
IVIMConstrained
IVIMConstrains
IVIMFixed

Method
OutputSamples
Parallelize
UpdateStep

JcouplingTools

Functions

GetSpinSystem
MakeSpinSystem
SequencePulseAcquire
SequenceSpaceEcho

SequenceSpinEcho
SequenceSteam
SequenceTSE
SimAddPhase

SimEvolve
SimHamiltonian
SimReadout
SimRotate

SimSignal
SimSpoil
SysTable

Options

CenterFrequency
FieldStrength
Linewidth

LinewidthShape
ReadoutBandwidth
ReadoutMethod

ReadoutOutput
ReadoutPhase
ReadoutSamples

SimNucleus

Legacy

Functions

AlignRespLog
BvalRead
DatRead
DatWrite
DTItoolExp
DTItoolExpFile
DTItoolExpInd
DTItoolExpTens

ExportVol
FiberDensityMap
FiberLengths
GradRead
ImportDTI
ImportPhyslog
ImportRespirect
ImportVol

LoadFiberTracts
PlotPhyslog
PlotRespiract
ReadBrukerDiff
ReadBvalue
ReadDicom
ReadDicomDiff
ReadDicomDir

ReadDicomDirDiff
ReadGradients
ReadVoxSize
ROIMask
SetupDataStructure
ShiftPar

Options

BinaryType
BmatrixOut

ConvertDcm
OutputMethod

RotateGradient
SampleStep

ScaleCorrect
SeedDensity

LoggingTools

Functions

AddToLog
CheckFile
DirectoryTree

ExportDirectoryTree
ExportLog
ImportLog

MakeCheckFile
PrintDirectoryTree
ResetLog

SetLogFile
ShowLog

Options

SaveLogFile

MaskingTools

Functions

DilateMask	Mask	NormalizeMeanData	SegmentMask
FitGradientMap	MaskData	RemoveMaskOverlaps	SelectMaskComponents
GetCommonSegmentation	MaskSegmentation	ReplaceSegmentations	SelectSegmentations
GetSegmentationLabels	MaskVolume	RescaleSegmentation	SmoothMask
HomogenizeData	MergeSegmentations	SegmentationCrossSection	SmoothSegmentation
JoinSegmentations	NormalizeData	SegmentationVolume	SplitSegmentations

Options

FitOrder	MaskDilation	Method
MaskClosing	MaskFiltKernel	NormalizeMethod
MaskComponents	MaskSmoothing	SmoothIterations

MuscleBidsTools

Functions

BidsDcmToNii	GetJSONPosition	MuscleBidsMerge	SelectBidsSessions
CheckConfigLabels	GetProtocolNames	MuscleBidsProcess	SelectBidsSubjects
CheckDataDescription	ImportJSON	MuscleBidsSegment	SelectSubjects
GenerateBidsFileName	MergeConfig	MuscleBidsTractography	ViewConfig
GenerateBidsFolderName	MergeJSON	PartitionBidsFolderName	ViewProtocolNames
GenerateBidsName	MuscleBidsAnalysis	PartitionBidsName	\$debugBids
GetConfig	MuscleBidsConvert	SelectBidsFolders	

Options

BidsIncludeSession	BidsTractographyMethod	ProcessSubjects
BidsOutputImages	DeleteAfterConversion	VersionCheck

NeuralNetworkTools

Functions

ActivationLayer	ClassDecoder	MakeClassifyImage	OverlapLossFunction
AddLossLayer	ClassEncoder	MakeClassifyNetwork	TopKLossLayer
AnalyzeNetworkFeatures	DiceLossLayer	MakeNode	TverskyLossLayer
CeLossFunction	FocalLossLayer	MakeUnet	
CeLossLayer	GetNetNodes	NetDimensions	
ChangeNetDimensions	JaccardLossLayer	NetSummary	

Options

Channels	BlockType	FeatureSchedule	NetworkDepth
Classes	CatenateMethod	ImageSize	NormalizationType
Dimensions	DownsampleSchedule	MonitorCalc	RescaleMethod
ActivationType	DropoutRate	NetworkArchitecture	SettingSchedule

NiftiTools

Functions

CompressNiiFiles	ExportBvec	ImportBvalvec	ImportNiiT1
CorrectNiiOrientation	ExportNii	ImportBvec	ImportNiiT2
DcmToNii	ExtractNiiFiles	ImportExploreDTIens	MakeNiiOrientationQ
ExportBmat	GetNiiOrientation	ImportNii	MakeNiiOrientationS
ExportBval	ImportBmat	ImportNiiDiff	
ExportBvalvec	ImportBval	ImportNiiDix	

Options

CompressNii	Method	NiiMethod	PositiveZ
DeleteOutputFolder	MonitorCalc	NiiOffset	RotateGradients
FlipBvec	NiiDataType	NiiScaling	UseSubfolders
MergeEchos	NiiLegacy	NiiSliceCode	UseVersion

PlottingTools

Functions

ColorFAPlot
GenerateRotationFrames
GetSliceData
GetSlicePositions
GradientPlot
LegendImage

Link3DGraphic
ListSpherePlot
LoessPlot
MakeSliceImages
PlotContour
PlotCorrection

PlotData
PlotData3D
PlotDefGrid
PlotDuty
PlotIVIM
PlotMoments

PlotSegmentations
PlotSequence
ShowLink
\$plotOptions

Options

Axes
Bandwidth
ClippingStyle
ColorFunction
ConfidenceLevel
ContourColor
ContourColorRange
ContourOpacity
ContourResolution
ContourScaling

ContourSmoothRadius
ContourSpecularity
DropSlices
FitKernel
FitOrder
Frame
FrameStyle
ImageLegend
ImageOrientation
ImageSize

LabelStyle
MakeCheckPlot
MaskOpacity
MaxPlotPoints
Method
NormalizeIVIM
PeakNumber
PerformanceGoal
PlotColor
PlotHighlighting

PlotRange
PlotSpace
PlotStyle
PositiveZ
PredictionInterval
RandomizeColor
SphereColor
SphereSize

ProcessingTools

Functions

B1MapCalc	FindOutliers	JoinSets	ParameterFit
B1Shimming	FitData	MeanRange	ParameterFit2
CombineB1	GetMaskData	MeanSignal	RotateData
CorrectJoinSetMotion	GetMaskMeans	MeanStd	RotateTensor
DataTransformation	GetTractMeans	MeanStdRange	SmartMask
DatTot	Hist	MedCouple	SNRCalc
DatTotXLS	Hist2	MinMaxRange	SNRMapCalc
ErrorPlot	InvertDataset	NumberTableForm	SplitSets

Options

AxesLabel	GetMaskOutput	OutlierIncludeZero	Scaling
B1EqualPower	ImageSize	OutlierIterations	SmartMaskOutput
B1FilterData	InterpolationOrder	OutlierMethod	SmartMethod
B1Masking	JoinSetSplit	OutlierOutput	SmoothSNR
B1MaxPower	MaskCompartment	OutlierRange	Strictness
B1Output	MeanMethod	OutputSNR	TableAlignments
B1Scaling	Method	PadOutputDimensions	TableDepth
B1ShimMethod	MonitorCalc	PadOverlap	TableDirections
ColorValue	MotionCorrectSets	PlotLabel	TableHeadings
FitFunction	NormalizeOverlap	ReferenceB1	TableMethod
FitOutput	NormalizeSets	ReverseData	TableSpacing
GetMaskOnly	OutlierCheckNormality	ReverseSets	UseMask

ReconstructionTools

Functions

AverageKspace	FourierRescaleData	MakeSense	SagittalTranspose
CoilCombine	FourierShift	MeanType	ShiftedFourier
CoilWeightedRecon	FourierShifted	NoiseCorrelation	ShiftedInverseFourier
CoilWeightedReconCSI	HammingFilterCSI	NoiseCovariance	TotalType
DeconvolveCSIdata	HammingFilterData	NoisePrewhitening	ZeropadData
FourierKspace2D	InverseFourierShift	NormalizeSpectra	
FourierKspace3D	InverseFourierShifted	OrderKspace	
FourierKspaceCSI	MakeHammingFilter	ReadListData	

Options

AcquisitionMethod	EchoShiftData	NormalizeOutputSpectra	SenseSmoothing
CoilSamples	HammingFilter	OutputSense	SenseWeight
DeconvolutionMethod	Method	ReconFilter	WienerRegularization
DenoiseCSI	NoisePrewhitened	RescaleRecon	

RelaxometryTools

Functions

CalibrateEPGT2Fit	EPGSignal	ShiftPulseProfile	T2Fit
CreateT2Dictionary	EPGT2Fit	T1Fit	TriExponentialT2Fit
DictionaryMinSearch	NonLinearEPGFit	T1rhoFit	

Options

DictB1Range	EPGCalibrate	EPGMethodCal	Method
DictT2fRange	EPGFatShift	EPGOutput	MonitorCalc
DictT2fValue	EPGFitFat	EPGRefocussingPhase	OutputCalibration
DictT2IncludeWater	EPGFitPoints	EPGRelaxPars	WaterFatShift
DictT2Range	EPGMethod	EPGSmoothB1	WaterFatShiftDirection

ScientificColorData

Functions

AddScientificColors	ExtractColorData
---------------------	------------------

Options

SegmentationTools

Functions

ApplySegmentationNetwork	GetNeuralNet	MakeClassImage	SegmentDataGUI
AugmentImageData	GetTrainData	MakeDistanceMap	ShowTrainLog
AugmentTrainingData	ImportITKLabels	MuscleLabelToName	SplitDataForSegmentation
CheckSegmentation	JaccardSimilarity	MuscleNameToLabel	SurfaceDistance
ClassifyData	MakeChannelClassGrid	PatchesToData	TrainSegmentationNetwork
DataToPatches	MakeChannelClassImage	PrepareTrainingData	\$debugUnet
DiceSimilarity	MakeChannelImage	SegmentData	

Options

Augment2D	FeatureSchedule	MonitorInterval	RescaleMethod
ActivationType	InputLabels	MultiChannel	RoundLength
AugmentData	L2Regularization	NetworkArchitecture	SegmentationDimension
BatchSize	LabelTag	NetworkDepth	SegmentationResolution
BlockType	LearningRate	NormalizationType	SegmentationsPerSlice
CatenateMethod	LoadTrainingData	OutputLabels	SettingSchedule
CleanUpSegmentations	LossFunction	OutputTag	TargetDevice
DataPadding	MaxPatchSize	PadData	TestRun
DataTag	MaxTrainingRounds	PatchesPerSet	TrainVoxelSize
DistanceRange	Method	PatchNumber	
DownsampleSchedule	Monitor	PatchPadding	
DropoutRate	MonitorCalc	PatchSize	

ShapeTools

Functions

ApplyShapeModel	MakeRegionMesh	MaskToDistanceMap	ScaleToVolume
EvaluateModel	MakeShapeModel	MeshGridPlot	SplitRegionMesh
FitShapeModel	MakeTemplatePlot	PlotMesh	TemplateToVolume
MakeMuscleTemplate	MaskFromDistanceMap	PlotShapeVariation	

Options

MeshColor	MeshOutput	Monitor	TemplatePadding
MeshEdgeColor	MeshPointColor	SplineSpacing	VolumeRescale
MesherPerRow	MeshPoints	TemplateDilation	
MeshOpacity	MeshPointSize	TemplateOutput	

SimulationTools

Functions

AddNoise	GESignal	PlotSimulationAngleHist	SimAngleParameters
BlochSeries	GetPulseProfile	PlotSimulationHist	SimParameters
CalculateGfactor	GfactorSimulation	PlotSimulationVec	SimulateDualTR
CreateDiffData	PlotSimulation	Pulses	SimulateSliceEPG
ErnstAngle	PlotSimulationAngle	Signal	Tensor

Options

FatFieldStrength	NoiseSize	ReportFits	TensOutput
GOutput	NoiseType	SliceRange	
GRegularization	PlotRange	SliceRangeSamples	
MagnetizationVector	Reject	SortVecs	

SpectroTools

Functions

ApodizeEcho	CorrectTESpec	GetTimeRange	PlotFid
ApodizeFid	CSIIInterface	ImportSparSdat	PlotSpectra
ApodizePadEcho	ExportSparSdat	MakeSpectraResultPlot	ReadjMRUI
ApodizePadFid	FindSpectraPpmShift	PadEcho	ShiftSpectra
ApodizePadSpectra	FitSpectra	PadFid	SpectraFitResult
ApodizeSpectra	FitSpectraResultTable	PadSpectra	SpectraSimulator
ChangeDwellTimeFid	GetGyro	PhaseCorrectSpectra	TimeShiftEcho
CompareFidFitPlot	GetPpmRange	PhaseShiftFid	TimeShiftFid
CompareSpectraFitPlot	GetSpectraBasisFunctions	PhaseShiftSpectra	TimeShiftFidV
CorrectTEFid	GetTimePpmRange	PlotCSIData	

Options

ApodizationFunction	GridlineSpacing	PlotRange	SpectraOutputPlots
AspectRatio	ImageSize	ReadoutType	SpectraPpmShift
BasisSequence	InitializeFit	SparID	SpectraSamples
CenterFrequency	Method	SparName	SpectraSpacing
Filling	PaddingFactor	SparOrientation	SplineSpacingFactor
FineTuneFit	PlotColor	SpectraBandwidth	
FitLineShape	PlotLabel	SpectraFieldStrength	
GridLines	PlotLabels	SpectraNucleus	

TaggingTools

Functions

AnnalyzeTagging

CalculateDisplacementParameters

Options

HistoryWeighting

MonitorTagging

TensorTools

Functions

ADCCalc	ECalc	FlipGradientOrientation	SigmaCalc
AngleCalc	EigensysCalc	FlipTensorOrientation	SortDiffusionData
AngleMap	EigenvalCalc	GetRPBMValues	TensorCalc
ConcatenateDiffusionData	EigenvecCalc	LogTensor	TensorCorrect
Correct	ExpTensor	ParameterCalc	TransformTensor
CreateRPBMDictionary	FACalc	RemoveIsoImages	WestinMeasures
Deriv	FitRPBMDictionary	ResidualCalc	
DriftCorrect	FitRPBMFunction	RPBMFunction	

Options

Distribution	MonitorCalc	RejectMap	RPBMRangeTau
FilterShape	NormalizeSignal	RobustFit	RPBMRangeZeta
FullOutput	Parallelize	RobustFitParameters	UseMask
MeanRes	PerformanceGoal	RotationCorrect	
Method	Reject	RPBMDictionarySize	

TractographyTools

Functions

EndpointDensityMap	FitTracts	ResampleTracts	TractCurvatureMap
ExportTCK	GetTractValues	RescaleTracts	TractDensityMap
ExportTracts	ImportTCK	SeedDensityMap	TractLength
FiberTractography	ImportTracts	SegmentTracts	TractLengthMap
FilterTractLength	MoveTracts	TractAngle	
FilterTracts	PlotSegmentedTracts	TractAngleMap	
FindTensorPermutation	PlotTracts	TractCurvature	

Options

Boxed	ImageSize	OutputForm	TractColoring
ColorFunction	InterpolationOrder	Parallelization	TractMonitor
ContourOpacity	MaxSeedPoints	PerformanceGoal	TractReduction
FiberAngle	MaxTracts	StepSize	TractScaling
FiberLengthRange	Method	StopThreshold	TractSize
FittingOrder	Monitor	TensorFlips	
FitTractSegments	NormalizeDensity	TensorPermutations	

```
In[24]:= QMRIToolsFuncPrint[]
```

CardiacTools

Functions

Symbol



BullseyePlot[data, segmask] generates a AHA–17 segment bullseye plot.

BullseyePlot[list] generates a AHA–17 segment bullseye plot of the lists (which needs to have 17 values) provide.

data is a 3D volume used for the plot.

segmask is the AHA–17 segmentation resulting form the CardiacSegment function when AHA17 is selected.

Output is a bullseye plot or a plotWindow, depending on the Method which can be "Dynamic" else it will be static.

BullseyePlot[] is based on DOI: 10.1161/hc0402.102975.

Documentation [Local »](#)

Default Definitions SyntaxInformation[BullseyePlot] = {ArgumentsPattern → {_, _}, OptionsPattern[]}}

Options ▶ TextOffset → 0.5 ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`BullseyePlot



Symbol



CalculateWallMap[mask,vox] calculates the wall distance map and the wall derivative.

Output is {wallmap, wallDerivative}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CalculateWallMap] = {ArgumentsPattern → {_, _}, OptionsPattern[]}}

Options {ShowPlot → True, MaskWallMap → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`CalculateWallMap



Symbol i

CardiacCoordinateSystem[mask, vox] creates the cardiac coordinate system within the mask and is used in HelixAngleCalc. Output is a set of vectors {radvecn, norvecc, cirvec}, being the radial, normal and circular axes of each voxel respectively. If the option showPlot is true the output is {{radvecn, norvecc, cirvec}, plots}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CardiacCoordinateSystem] = {ArgumentsPattern -> {_, _}, OptionsPattern[]}`

Options {ShowPlot -> False, LCMMMethod -> WallMap, AxesMethod -> Quadratic}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`CardiacCoordinateSystem`

^

Symbol i

CardiacSegment[mask, vox, pts] segments the mask in the AHA17 segmentation using pts to indicate the attachments.

CardiacSegment[mask, back, vox, pts] the same where back is used for image generation.

CardiacSegment[mask, vox, pts, seg] does the same but seg can be an alternate segmentation to the AHA17.

CardiacSegment[mask, back, vox, pts, seg] does the same but seg can be an alternate segmentation to the AHA17 where back is used for image generation.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CardiacSegment] = {ArgumentsPattern -> {_, _, _, _}, OptionsPattern[]}`

Options {ReversePoints -> True, ReverseDirection -> False, MakeSegmentPlots -> True}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`CardiacSegment`

^

Symbol i

CardiacSegmentGUI[data, mask, vox] allows to segment the heart in 1, 4, 6 or AHA–17 segments for each slice 360 radial samples are generated.

data is a background image on which all overlays are projected.

mask is the mask of the left ventricle (same as used for CentralAxes) and defines the area in which the data is sampled.

off is the centerpoints generated by CentralAxes.

Output is {points, slices , {rev, flip}}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CardiacSegmentGUI] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ➤ StartPoints → Default ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`CardiacSegmentGUI

^

Symbol i

CentralAxes[mask, vox] calculates the center of the lumen from a mask of the left ventricle. vox is the voxels size, {slice, x, y}.

CentralAxes[mask, maskp, vox] allows for fancy visualization of the other structures using maskp.

Output is {centerpoints, normalvecs, inout} or {centerpoints, normalvecs, inout, fit}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CentralAxes] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {ShowPlot → False, RowSize → Automatic, AxesMethod → Cubic}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`CentralAxes

^

Symbol i

CreateHeart[] creates a simulated left ventricle shape.

CreateHeart[pars] creates a simulated left ventricle shape with predefined parameters pars.

Output is the heart shape, the voxel size and the parameters needed to generate the heart, {mask, vox, pars}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CreateHeart] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`CreateHeart`

^

Symbol i

ECVCalc[T1pre, T1post, hema] calculates the ECVmap using MakeECVBloodMask.

ECVCalc[T1pre, T1post, bloodMask, hema] calculates the ECVmap using bloodMask.

The T1pre and T1post maps are assumed to be in ms.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`ECVCalc`

^

Symbol i

ExcludeSlices[data] excludes slices that do not look like the others based on various distance measures.

Output is an array with 1 or 0 with the dimensions {slices, diff dirs}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExcludeSlices] = {ArgumentsPattern → {..., OptionsPattern[]}}`

Options {CutOffMethod → Auto, DistanceMeasure → 5, ShowOutliers → False}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`ExcludeSlices`

^

Symbol i

GetMaskSegmentPoints[mask] get the attachment points from a cardiac segmentation where the heart has label 1, and the attachment points have label 2 and 3.

Output is {maks, pts} where now in mask the points are removed.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetMaskSegmentPoints] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`GetMaskSegmentPoints`

^

Symbol i

GetSegmentLines[lines, lineIndex, segments] groups the transmural lines per segment.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetSegmentLines] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`GetSegmentLines`

^

Symbol i

GetSegmentSlices[mask] based on the mask it gives back the slice numbers of the apex, apical, mid-vertical, and basal slices.

GetSegmentSlices[points] does the same but then based on the points obtained form `GetMaskSegmentPoints`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetSegmentSlices] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`GetSegmentSlices`

^

HelixAngleCalc[eigenvectors, mask, vox] calculates the helix angle matrix of cardiac data using only a left ventricle mask.

HelixAngleCalc[eigenvectors, mask, maskp, vox] calculates the helix angle matrix of cardiac data using only a left ventricle mask, and a maskp for visualization.

HelixAngleCalc[eigenvectors, mask, centerpoint, vec, inout, vox] calculates the helix angle matrix of cardiac data using only a left ventricle mask.

HelixAngleCalc[eigenvectors, mask, maskp, centerpoint, vec, inout, vox] calculates the helix angle matrix of cardiac data using a left ventricle mask and a maskp for visualization.

eigenvectors are the tensor eigenvectors calculated with EigenvecCalc.

mask is a mask of the left ventricle.

maskp is a mask used for visualization.

vox is the voxels size, {slice, x, y}.

The following values are calculated automatically Using CentralAxes but can also be provided as an input.

centerpoint is the center of each slice calculated with CentralAxes.

inout is the inner and outer radius calculated with CentralAxes.

vec is the vector describin the central axes of the heart, calculated with CentralAxes.

Output is het fiber angle matrix FAM = {9, slice, x, y} or {FAM, plot}.

The angles are in degrees.

HelixAngleCalc[] is based on DOI: 10.1186/1532-429X-17-S1-P15.

Documentation [Local »](#)

Default Definitions SyntaxInformation[HelixAngleCalc] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options {ShowPlot → True, LCMMMethod → WallMap, AxesMethod → Quadratic}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`HelixAngleCalc



Symbol



LinesToSegmentIndex[lines, points, segments] finds the lines indices corresponding to the points and the segments borders. Additionally it finds all the lines indices for all lines within each segment. The lines are computed by MaskToLines, the points are computed by GetMaskSegmentPoints, and the segments is the output of SegmentsPerSlices.

Output {pointIndex, segmentIndex, lineIndex}.

Documentation [Local](#) »

Default Definitions SyntaxInformation[LinesToSegmentIndex] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options {ReversePoints → True, ReverseDirection → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`LinesToSegmentIndex



Symbol



MakeECVBloodMask[T1pre, T1post] makes a bloodpool mask based on the T1pre and T1post images. It assumes that the hart is cropped with the blood in the center.

The T1pre and T1post maps are assumed to be in ms.

Documentation [Local](#) »

Default Definitions SyntaxInformation[MakeECVBloodMask] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {BloodMaskRange → {1400, {0, 700}}, OutputCheckImage → True}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`MakeECVBloodMask



Symbol



MakeLineImage[back, segLines, pts] makes an image of the cardiac segmentation lines.

Documentation [Local](#) »

Default Definitions SyntaxInformation[MakeLineImage] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`MakeLineImage



Symbol i

MakeMaskImage[back, mask] masks an image of the cardiac segmentation mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeMaskImage] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`MakeMaskImage`

^

Symbol i

MaskHelix[helix, mask] masks helix angle data, sets the background to -100 and allows for Median filter of the helix mask.

helix can be a singel map or the FAM.

Output is the masked helix angle data.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MaskHelix] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {BackgroundValue → -100, SmoothHelix → False}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`MaskHelix`

^

Symbol i

MaskToLines[mask, vox] calculates lines perpendicular to the heart wall per slice within the mask. Internally it uses `CalculateWallMap` and `CentralAxes` to obtain the cardiac geometry from mask.

MaskToLines[mask, wall, cent] where mask is the first output of `CalculateWallMap` and cent is the first output of `CentralAxes`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MaskToLines] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`MaskToLines`

^

Symbol i

PlotSegmentMask[mask, segmask, vox] plots the mask segments created by CardiacSegment.

mask is a mask the left ventricle that was used in the CardiacSegment.

segmask is the output of CardiacSegment.

vox is the voxels size, {slice, x, y}.

Output is a plot window.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSegmentMask] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`PlotSegmentMask`

^

Symbol i

PlotSegments[mask, data, segang] shows how the heart will be sampled by RadialSample.

mask is a mask the left ventricle that was used in the CardiacSegment.

function and the segang is the output of the cardiac SegmentFunction.

Output is a plot window.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSegments] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `RadialSamples → 10`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`PlotSegments`

^

Symbol i

RadialSample[mask, data, segang] radially samples the provided parameter map data.

The mask should be a mask of the left ventricle that was used in the CardiacSegment.
segang is the output of the cardiac SegmentFunction.

Output is {points, vals} which are ordered as indicated by the user.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RadialSample] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options {RadialSamples → 10, DropSamples → 0}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`RadialSample

^

Symbol i

SegmentLinesToMask[mask, segLines] cuts the mask based one the tranmural lines per segments which can be obtained by GetSegmentLines.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SegmentLinesToMask] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`SegmentLinesToMask

^

Symbol i

SegmentsPerSlice[points] gives the number of segments per slice where the slice distribution is determined by GetSegmentSlices.
SegmentsPerSlice[slices, points] does the same but the slices are given manually.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SegmentsPerSlice] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options {GroupPerSegment → True, SegmentationMethod → AHA}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`SegmentsPerSlice

^

Symbol i

TransmuralPlot[data] plots transmural profiles of the data which are created by RadialSample.

data can be a single profile or a list of profiles. In the second case the mean and standard deviations are plotted.

Output is a plot of the transmural profile.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TransmuralPlot] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options » GridLineSpacing → 10 ... (6 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`TransmuralPlot`

^

Options

Symbol i

AxesMethod is an option for HelixAngleCalc and CentralAxes and CardiacCoordinateSystem. Can be "Linear", "Quadratic", "Cubic".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`AxesMethod`

^

Symbol i

BackgroundValue is an option for MaskHelix. Sets the background value (default is -100).

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`BackgroundValue`

^

Symbol i

BloodMaskRange is an option for MakeECVBloodMask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`CardiacTools`BloodMaskRange`

^

Symbol 

BullPlotMethod is an option for BullseyePlot. Can be "Dynamic" or "Normal".

"Dynamic" allows to change plotting parameters in Manipulation window.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`BullPlotMethod

^

Symbol 

ColorFunction is an option for graphics functions that specifies a function to apply to determine colors of elements.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ColorFunction

^

Symbol 

CutOffMethod is an option for ExcludeSlices. Default value is "Auto" or it can be a fixed percentage (value between 0 and .5).

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`CutOffMethod

^

Symbol 

DistanceMeasure is an option for ExcludeSlices. Default value is 5. (1 ManhattanDistance, 2 SquaredEuclideanDistance, 3 EuclideanDistance, 4 Correlation, 5 SpearmanRho).

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`DistanceMeasure

^

Symbol 

DropSamples is an option for RadialSample and PlotSegments. Defines how many samples are dropped from star and end. Can be a number or set (start, end) of numbers.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`DropSamples

^

Symbol i

GridLineSpacing is an option of TransmuralPlot. It defines the spacing of the grid lines.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`GridLineSpacing

^

Symbol i

GroupPerSegment is an option for SegmentsPerSlice. If set False segments are grouped per slice and not per segment.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`GroupPerSegment

^

Symbol i

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol i

LCMMMethod is an option for HelixAngleCalc and LMCSysytemCalc. Can be "CentralAxes" or "WallMap".

"CentralAxes" uses wall distance calculation using projection of the central axes and circular approximation of the ventricle. This method is fairly fast and uses CentralAxes internally.

"WallMap" uses wall distance interpolation and subsequential gradient calculation. Can take long for high res datasets but is most accurate. Uses CalculateWallMap internally.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`LCMMMethod

^

Symbol i

MakeSegmentPlots is an option for CardiacSegment. If True plots of the segmentation are made.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`MakeSegmentPlots

^

Symbol 

MaskWallMap is an option for CalculateWallMap. if True or False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`MaskWallMap

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

OutputCheckImage is an option for MakeECVBloodMask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`OutputCheckImage

^

Symbol 

PlotLabel is an option for graphics functions that specifies an overall label for a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PlotLabel

^

Symbol 

PlotRange is an option for graphics functions that specifies what range of coordinates to include in a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotRange

^

Symbol i

PlotStyle is an option for plotting and related functions that specifies styles in which objects are to be drawn.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PlotStyle

^

Symbol i

RadialSamples is an option for RadialSample and PlotSegments. Defines how many transmural samples are taken.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`RadialSamples

^

Symbol i

ReverseDirection is an option for LinesToSegmentIndex, CardiacSegment. Defines the direction of rotation, clockwise or anti-clockwise, can be True or False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`ReverseDirection

^

Symbol i

ReversePoints is an option for LinesToSegmentIndex, CardiacSegment. Defines at which point to start, can be True or False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`ReversePoints

^

Symbol i

RowSize is an option for CentralAxes. defines the number of images per showing the segmentation.

Can be "Automatic" or an integer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`RowSize

^

Symbol 

SegmentationMethod is an option for SegmentsPerSlice. Values can be "AHA", "AHA+", 1, 2, 3, 6 or 8.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`SegmentationMethod

^

Symbol 

ShowOutliers is an option for ExcludeSlices.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`ShowOutliers

^

Symbol 

ShowPlot is an option for CentralAxes, HelixAngleCalc and CardiacCoordinateSystem. True shows the fit of the central axes and output the plot as extra output.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`ShowPlot

^

Symbol 

SmoothHelix is an option for MaskHelix, sets the kernel size for the MedianFilter.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`SmoothHelix

^

Symbol 

StartPoints is an option for CardiacSegmentGUI. Value is "Default" or the point list given by CardiacSegment.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`StartPoints

^

Symbol 

StartSlices is an option for CardiacSegmentGUI. Value is "Default" or the list given by CardiacSegment.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`StartSlices

^

Symbol 

TextNumberForm is an option for BullseyePlot. Specifies how many number and decimals to use like in NumberForm.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`TextNumberForm

^

Symbol 

TextOffset is an option for BullseyePlot. Determines where the text is placed, can be 0 to 1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`TextOffset

^

Symbol 

TextSize is an option for BullseyePlot. Determines the text size.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`TextSize

^

CoilTools

Functions

Symbol i

CoilSNRCalc[coils, noise] calculates the sensitivity weighted snr of multiple coil elements using magnitude signal and noise.

Output is {data, noise, sos, snr, sigmap, weights}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CoilSNRCalc] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`CoilSNRCalc

^

Symbol i

FindCoilPosition[weights] finds the coil position by locating the highest intensity location in the coil weight map, which can be obtained by LoadCoilSetup or SumOfSquares.

Internally it uses MakeWeightMask to remove the noise of the weightmasks.

FindCoilPosition[weights, mask] limits the search region to the provided mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindCoilPosition] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {OutputCoilSurface → False, CoilSurfaceVoxelSize → {1, 1, 1}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`FindCoilPosition

^

Symbol i

LoadCoilSetup[file] load a very specific type of coil experiment, a dynamic scan with a setup of which the second dynamic is a noise measurement.

The input file is the Nii file that contains the individually reconstructed coil images and the noise data.

Internally it uses CoilSNRCalc and SumOfSquares.

Output is the coil data with coil noise data and snrmap based on the SumOfSquares addition, the SOS reconstruction and the SOS weights.

{dataC, noiseC, sosC, snrC, sigmapC, weights, vox}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[LoadCoilSetup] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`LoadCoilSetup

^

Symbol i

LoadCoilTarget[file] loads a very specific typ of experiment, a dynamic scan with with the second dynmaic is a noise measurement. The input file is the Nii file that conatins the scanner reconstruction and the noise data.

Internally it uses SNRMapCalc,

Output is the reconstructed data with noise data and snrMap {dataC, noiseC, sosC, snrC, sigmapC, weights, vox}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[LoadCoilTarget] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`LoadCoilTarget

^

Symbol i

MakeCoilLayout[{name, size, number}] makes a coil grid with label name, partioned in size rows and with label number.

MakeCoilLayout[{name, size, number}, val] makes a coil grid with label name, partioned in size rows and with label the val at location number.

MakeCoilLayout[{coils..}] same but for multile coils grids. Each coil grid is defined as {name, size, number}.

MakeCoilLayout[{coils..}, val] savem but for multiple coil grids.

Documentation [Local »](#)

Options » PlotRange → Automatic ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`MakeCoilLayout

^

Symbol i

MakeNoisePlots[noise] returns a grid of plots of the noise per channel

MakeNoisePlots[noise, {met, prt}] met can be "Grid" with prt a number or Automatic. Else all plots will be returend as a list of plots.

MakeNoisePlots[noise, {met, prt}, sub] sub defines how much the noise is subsampled, default is 40 (every 40th sample is used in plot).

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeNoisePlots] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`MakeNoisePlots

^

Symbol i

MakeWeightMask[weights] creates a mask of homogeneous regions of weightmaps removing the noise.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeWeightMask] = {ArgumentsPattern → {`_`}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`MakeWeightMask

^

Options

Symbol i

CoilArrayPlot is an option for MakeCoilLayout. If True and values are provided it makes an arrayplot of the coil layouts.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`CoilArrayPlot

^

Symbol i

CoilSurfaceVoxelSize is an option for FindCoilPosition. Specifies the voxel size used for OutputCoilSurface.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`CoilSurfaceVoxelSize

^

Symbol i

ColorFunction is an option for graphics functions that specifies a function to apply to determine colors of elements.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ColorFunction

^

Symbol i

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol i

OutputCoilSurface is an option for FindCoilPosition. If set true it will also output a SurfacePlot of the coil location volume.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CoilTools`OutputCoilSurface

^

Symbol i

PlotRange is an option for graphics functions that specifies what range of coordinates to include in a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotRange

^

DenoiseTools

Functions

Symbol i

AnisoFilterData[data] Filter the diffusion tensor data using an anisotropic filter based on the strucure tensor of the data.

Output is the smoothed data.

AnisoFilterData[] is based on DOI: 10.1016/j.jbiomech.2021.110540 and 10.1016/j.mri.2009.10.001 and 10.1371/journal.pone.0126953.

Documentation [Local »](#)

Default Definitions SyntaxInformation[AnisoFilterData] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options {AnisoStepTime → 1, AnisoIterations → 1, AnisoKernel → {0.25, 0.5}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoFilterData

^

Symbol i

AnisoFilterTensor[tens, diffdata] Filter the tensor tens using an anisotropic diffusion filter (Perona–Malik). It uses the diffusion weighted data diffdata to find edges that are not visible in the tensor. Edge weights based on the diffusion data are averaged over all normalized diffusion direction.

AnisoFilterTensor[tens] Same but does not use the data for edge identification.

Output is the smoothed tensor.

AnisoFilterTensor[] is based on DOI: 10.1109/ISBI.2006.1624856.

Documentation [Local »](#)

Default Definitions SyntaxInformation[AnisoFilterTensor] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options ▸ AnisoWeightType → 2 ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoFilterTensor

^

Symbol i

DeNoise[data,sigma,filtersize] removes Rician noise with standard deviation "sigma" from the given dataset using a kernel with size "filtersize" a gaussian kernel.

Output is data denoised.

DeNoise[] is based on DOI: 10.1109/TMI.2008.920609.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DeNoise] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {DeNoiseKernel → Gaussian, DeNoiseMonitor → False, DeNoiseIterations → 1}

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`DeNoise

^

Symbol i

DenoiseCSldata[spectra] performs PCA denoising of the complex values spectra, data has to be 3D and the spectral dimensions is last, {x,y,z,spectra}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DenoiseCSldata] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options ▶ `PCAKernel` → 5 ... (4 total)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`DenoiseCSldata`

^

Symbol i

DenoiseDynamicSpectraData[spectra] performs PCA denoising of the complex values spectra, The data is given as a list of dynamicly acquired spectra {dynamic ,spectra}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DenoiseDynamicSpectraData] = {ArgumentsPattern → {_,}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`DenoiseDynamicSpectraData`

^

Symbol i

HarmonicDenoiseTensor[tens, mask, vox] uses the harmonic denoising method to denoise the tensor within the mask.

Values for which the tensor is not defined but are within the mask will be filled in.

HarmonicDenoiseTensor[tens, seg, vox] will do the same for each segmentation label in seg.

HarmonicDenoiseTensor[tens, seg, vox, labs] will do the same for each segmentation label in seg that is specified in labs.

HarmonicDenoiseTensor[] is based on 10.1016/j.media.2011.01.005.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[HarmonicDenoiseTensor] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options ▶ `RadialBasisKernel` → 12 ... (10 total)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`HarmonicDenoiseTensor`

^

Symbol i

NNThreshold is an options for P2SDenoise and specifies the automated back ground masking value.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`NNThreshold

^

Symbol i

P2SDenoise[data] removes rician noise from the data using self supersized neural net.

P2SDenoise[data, mask] removes rician noise from the data with PCA using self supersized neural net withing the mask.

PCADeNoise[] is based on DOI:10.48550/arXiv.2011.01355.

Documentation [Local »](#)

Default Definitions SyntaxInformation[P2SDenoise] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}

Options {Threshold → 2, Iterations → 10}

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`P2SDenoise

^

Symbol i

PCADeNoise[data] removes rician noise from the data with PCA.

PCADeNoise[data, mask] removes rician noise from the data with PCA only withing the mask.

PCADeNoise[data, mask, sig] removes rician noise from the data with PCA only withing the mask using sig as prior knowledge or fixed value.

Output is de {data denoise, sigma map} by default if PCAOutput is Full then fitted {data
 dnoise , {sigma fit, average sigma}, {number components, number of fitted voxels, number of max fits}, total fit –time per 500 ittt}.

PCADeNoise[] is based on DOI: 10.1016/j.neuroimage.2016.08.016 and 10.1002/mrm.26059.

Documentation [Local »](#)

Default Definitions SyntaxInformation[PCADeNoise] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}

Options > PCAKernel → 5 ... (9 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCADeNoise

^

Symbol i

WeightMapCalc[diffdata] calculates a weight map which is used in AnisoFilterTensor.

Output is a weight map of the diffdata which is high in isotropic regions and low at edges.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[WeightMapCalc] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{AnisoWeightType → 2, AnisoKappa → 10.}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`WeightMapCalc`

^

Symbol i

QMRITools`DenoiseTools`\$debugDenoise

Documentation [Local »](#)

Attributes `{ReadProtected}`

Full Name `QMRITools`DenoiseTools`$debugDenoise`

^

Options

Symbol i

AnisoFilterSteps is an option for AnisoFilterTensor and defines the amountn of diffusin steps taken. Higher is more smoothing.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`AnisoFilterSteps`

^

Symbol i

AnisoIterations is an options for AnisoFilterData. It specifies the amount of denoising iterations.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DenoiseTools`AnisoIterations`

^

Symbol 

AnisoKappa is an option for AnisoFilterTensor and WeightMapCalc and defines the weighting strenght, all data is normalize to 100 before filetering.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoKappa

^

Symbol 

AnisoKernel is an options for AnisoFilterData. It defines the kernel size.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoKernel

^

Symbol 

AnisoStepTime is an option for AnisoFilterTensor and defines the diffusion time, when small more step are needed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoStepTime

^

Symbol 

AnisoWeightType is an option for AnisoFilterTensor and WeightMapCalc and defines the weighting, eigher 1, the exponent of $(-g/kappa)$ or 2, $1/(1+g/kappa)$.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`AnisoWeightType

^

Symbol 

DeNoiseIterations is and option for DeNoise. Specifies the number of the denoising iterations.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`DeNoiseIterations

^

Symbol i

DeNoiseKernel is and option for DeNoise. Values can be "Disk", "Box" or "Gaussian".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`DeNoiseKernel

^

Symbol i

DeNoiseMonitor is and option for DeNoise. Monitor the denoising progres.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`DeNoiseMonitor

^

Symbol i

GradientStepSize is an option for HarmonicDenoiseTensor. It defines the step size of the gradient descent for the harmonic and radial parts.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`GradientStepSize

^

Symbol i

Iterations is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform. It specifies the number of iterations used by the registration functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`Iterations

^

Symbol i

MaxIterations is an option that specifies the maximum number of iterations that should be tried in various built-in functions and algorithms.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`MaxIterations

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol i

Monitor[*expr*, *mon*] generates a temporary monitor cell in which the continually updated current value of *mon* is displayed during the course of evaluation of *expr*.

Documentation [Local »](#) | [Web »](#)

Attributes {HoldAll, Protected, ReadProtected}

Full Name System`Monitor

^

Symbol i

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol

`Parallelize[expr]` evaluates *expr* using automatic parallelization.

Definitions

```
Parallelize[Parallel`Evaluate`Private`expr_, Parallel`Evaluate`Private`opts : OptionsPattern[]] :=
Module[{Parallel`Evaluate`Private`res, Parallel`Evaluate`Private`ahead, Parallel`Evaluate`Private`fopts}, Parallel`Protected`tryRelaunch[];
If[Parallel`Protected`$seQ, Message[Parallelize::nopar];
Return[Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`fopts = Sequence @@ {Method → OptionValue[Method],
DistributedContexts → Parallel`Protected`DistOptCheck[Parallelize, OptionValue[DistributedContexts]], ProgressReporting → OptionValue[ProgressReporting]};
Parallel`Evaluate`Private`ahead = Replace[Head[Unevaluated[Parallel`Evaluate`Private`expr], Hold],
{Hold[Parallel`Evaluate`Private`s_Symbol] => Parallel`Evaluate`Private`s, _ => $Failed}];
With[{Parallel`Evaluate`Private`handler = Lookup[Parallel`Evaluate`$ParallelExtensions, Parallel`Evaluate`Private`ahead, $Failed]},
If[Parallel`Evaluate`Private`handler != $Failed, Return[Parallel`Evaluate`Private`handler[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts]]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`tryCombine[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`tryCombine, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`wrapAround[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`wrapAround, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`silentFail[Parallel`Evaluate`Private`expr];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`silentFail, Return[Parallel`Evaluate`Private`res]];
If[Parallel`Evaluate`Private`$seqWarning, Message[Parallelize::nopar1, Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`expr]
```

Documentation [Local »](#) | [Web »](#)

Options {DistributedContexts => \$Context, Method → Automatic, ProgressReporting => \$ProgressReporting}

Attributes {HoldFirst, Protected}

Full Name System`Parallelize



Symbol



`PCAClipping` is an option of `PCADeNoise` and can be `True` or `False`. If `True` the output is clipped between 0 and the max absolute value of the input data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCAClipping



Symbol 

PCAComplex is an option of PCADeNoise and can be True or False. If set true the input data is expected to be {real, imag}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCAComplex

^

Symbol 

PCAKernel is an option of PCADeNoise. It sets the kernel size.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCAKernel

^

Symbol 

PCANoiseSigma is an option of DenoiseCSldata and can be "Corners" or "Automatic".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCANoiseSigma

^

Symbol 

PCAOOutput is an option of PCADeNoise. If output is full the output is {datao, {output[[1]], sigmat}, {output[[2]], output[[3]], j}, timetot}.

Else the output is {datao, sigmat}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCAOOutput

^

Symbol 

PCATolerance is an option of PCADeNoise and should be an integer > 0. Default value is 0. When increased the denoise method removes less noise.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCATolerance

^

Symbol



PCAWeighting is an option of PCADeNoise and can be True of False. Default value is False. When True the weights of the per voxel result are calculated based on the number of non noise components.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`PCAWeighting



Symbol



RadialBasisKernel is an option for HarmonicDenoiseTensor. It defines the kernel size of the radial basis functions in mm.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`RadialBasisKernel



Symbol



RangeFA is an option for HarmonicDenoiseTensor. It defines the range of the FA values of voxels to include in the minization.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`RangeFA



Symbol



RangeMD is an option for HarmonicDenoiseTensor. It defines the range of the MD values of voxels to include in the minization.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DenoiseTools`RangeMD



Symbol



TensorFlips is an option for FiberTractography and specifies if the tensor orientation is flipped, see FlipTensorOrientation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TensorFlips



Symbol i

TensorPermutations is an option for FiberTractography and specifies if the tensor orientation is permuted, see FlipTensorOrientation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TensorPermutations

^

Symbol i

Threshold[*data*] thresholds *data* by replacing values close to zero by zero.

Threshold[*data*, *tspec*] thresholds *data* using threshold specification *tspec*.

Threshold[*image*, ...] replaces small values of *image* by zero.

Threshold[*sound*, ...] replaces small values of *sound* by zero.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Threshold

^

Symbol i

Tolerance is an option for various numerical options which specifies the tolerance that should be allowed in computing results.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Tolerance

^

DixonTools

Functions

Symbol i

DixonPhase[real, imag, echos] calculates the b0 and ph0 maps.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DixonPhase] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options [»](#) DixonPrecessions → 1 ... (9 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonPhase

^

Symbol i

DixonReconstruct[{real, imag}, echo] reconstructs Dixon data with initial guess $b_0 = 0$ and $T_{2\text{star}} = 0$.

DixonReconstruct[{real, imag}, echo, {b0}] reconstructs Dixon data with initial guess $T_{2\text{star}} = 0$.

DixonReconstruct[{real, imag}, echo, {b0, t2}] reconstructs Dixon data with $t_{2\text{star}}$ and B_0 .

DixonReconstruct[{real, imag}, echo, {b0, t2, ph0}] reconstructs Dixon data with initial phase.

DixonReconstruct[{real, imag}, echo, {b0, t2, ph0, phb}] reconstructs Dixon data with bipolar phase .

Output is $\{\{\text{watF}, \text{fatF}\}, \{\text{watSig}, \text{fatSig}\}, \{\text{inphase}, \text{outphase}\}, \{\{b_0, \text{ph0}, \text{phb}\}, \{t_2, t_2\}\}, \text{iterations}\}$.

The fractions are between 0 and 1, the B_0 field map is in Hz and the $T_{2\text{star}}$ map is in ms.

DixonReconstruct[] is based on DOI: 10.1002/mrm.20624 and 10.1002/mrm.21737 (10.1002/nbm.3766).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DixonReconstruct] = {ArgumentsPattern → {_, _, _, _, _}, OptionsPattern[]}`

Options » DixonPrecessions → 1 ... (21 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonReconstruct

^

Symbol i

DixonToPercent[water, fat] converts the dixon water and fat data to percent maps.

Output is {waterFraction, fatFraction}.

The values of water and fat are arbitrary units and the output fractions are between 0 and 1.

DixonToPercent[] is based on DOI: 10.1002/mrm.21301.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DixonToPercent] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonToPercent

^

Symbol i

FindInPhaseEchos[echos, iop] finds the two nearest echos to inphase which are best used for unwrapping using the iop time.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindInPhaseEchos] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `DixonBipolar → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DixonTools`FindInPhaseEchos`

^

Symbol i

FixDixonFlips[{mag, phase, real, imag}] checks if any volumes are 180 degrees out of phase and corrects them.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FixDixonFlips] = {ArgumentsPattern → {{_, _}, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DixonTools`FixDixonFlips`

^

Symbol i

GenerateAmps[amp] generates the fat based chemical species amplitudes needed for DixonReconstruct.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DixonTools`GenerateAmps`

^

Symbol i

OptimizeDixonEcho[] shows a manipulate panel which allows to optimize the dixon echos.

OptimizeDixonEcho[echos] shows a manipulate panel which allows to optimize the predefined dixon echos.

Documentation [Local »](#)

Options `{DixonNucleus → 1H, DixonFrequencies → {{0.}, {-3.81, -3.4, -3.12, -2.67, -2.45, -1.94, -0.63, -0.4, 0.52, 0.62}}, DixonAmplitudes → {{1}, {0.089, 0.577, 0.059, 0.093, 0.059, 0.013, 0.02, 0.02, 0.01, 0.059}}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`DixonTools`OptimizeDixonEcho`

^

Symbol 

SimulateDixonSignal[echo, fr, B0, T2] simulates an Dixon gradient echo sequence with echotimes.

Echotimes echo in ms, fat fraction fr between 0 and 1, field of resonance B0 in Hz and relaxation T2 in ms.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimulateDixonSignal] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}}

Options  DixonNucleus → 1H... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`SimulateDixonSignal

^

Symbol 

Unwrap[data] unwraps the given dataset. The data should be between $-\pi$ and π .

Unwrap[] is based on DOI: 10.1364/AO.46.006623 and 10.1364/AO.41.007437.

Documentation [Local »](#)

Default Definitions SyntaxInformation[Unwrap] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options {MonitorUnwrap → False, UnwrapDimension → 2D, UnwrapThresh → 0.5}

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`Unwrap

^

Symbol 

UnwrapDCT[data] unwraps the given dataset using DCT transform . The data should be between $-\pi$ and π .

UnwrapDCT[] is based on DOI: 10.1364/JOSAA.11.000107.

Documentation [Local »](#)

Default Definitions SyntaxInformation[UnwrapDCT] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`UnwrapDCT

^

Symbol i

UnwrapList[list] unwraps a 1D list of values between $-\pi$ and π .

Documentation [Local »](#)

Default Definitions `SyntaxInformation[UnwrapList] = {ArgumentsPattern → { $\_$ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`DixonTools`UnwrapList`

^

Symbol i

UnwrapSplit[phase, data] unwraps the give phase dataset but splits the data into left and right using `SplitData` based in the data and performs the unwrapping separately. The data should be between $-\pi$ and π . **UnwrapSplit[]** is based on DOI: 10.1364/AO.46.006623 and 10.1364/AO.41.007437.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[UnwrapSplit] = {ArgumentsPattern → { $\_$ ,  $\_$ , OptionsPattern[]}}`

Options {MonitorUnwrap → False, UnwrapDimension → 2D, UnwrapThresh → 0.5}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`DixonTools`UnwrapSplit`

^

Symbol i

Wrap[data] wraps phase values between $-\pi$ and π .

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Wrap] = {ArgumentsPattern → { $\_$ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`DixonTools`Wrap`

^

Options

Symbol i

DixonAmplitudes is an option for `DixonReconstruct`. Defines the relative amplitudes of the fat peaks being used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`DixonTools`DixonAmplitudes`

^

Symbol i

DixonBipolar is an option for FindInPhaseEchos. If set True the found echos will always be of the same polarity.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonBipolar

^

Symbol i

DixonClipFraction is an option for DixonReconstruct. If set True the fat fraction is clipped between 0 and 1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonClipFraction

^

Symbol i

DixonConstrainPhase is an option for DixonReconstruct. If set True everything is fitted as real values.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonConstrainPhase

^

Symbol i

DixonCorrectT1 is an option for DixonReconstruct. To perform T1 correction provide the TR and FA as a list, {TR, FA}. TR is in ms and FA in degrees.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonCorrectT1

^

Symbol i

DixonFieldStrength is an option for DixonReconstruct. Defines the fieldstrengths in Tesla on which the data was acquired.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFieldStrength

^

Symbol i

DixonFilterInput is an option for DixonReconstruct. If True the input b0 and T2star values are smoothed using a gaussian kernel.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFilterInput

^

Symbol i

DixonFilterOutput is an option for DixonReconstruct. If True the out b0 and T2star values are smoothed Median filter and lowpassfiltering after which the water and fat maps are recomputed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFilterOutput

^

Symbol i

DixonFilterSize is an option for DixonReconstruct. Defines the number of voxel with which the input b0 and T2star values are smoothed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFilterSize

^

Symbol i

DixonFilterType is an option for DixonReconstruct. FilterType can me "Median" or "Laplacian".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFilterType

^

Symbol i

DixonFitPhase is an option for DixonReconstruct. If set to True the phase maps are fitted using polynomial functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFitPhase

^

Symbol i

DixonFixT2 is an option for DixonReconstruct. If set to true the R2' is fitted rather than the R2*. This is done by fixing T2–water to 30ms and T2–fat to 100ms.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFixT2

^

Symbol i

DixonFrequencies is an option for DixonReconstruct. Defines the frequencies in ppm of the fat peaks being used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonFrequencies

^

Symbol i

DixonIterations is an option for DixonReconstruct. Defines the maximum iterations the fit can use.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonIterations

^

Symbol i

DixonMaskThreshold is an option for DixonReconstruct. Defines at which Threshold the dixon reconstruction considers a voxel to be background noise. default values is 0.05.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonMaskThreshold

^

Symbol i

DixonNucleus is an option for DixonReconstruct. Defines the nucleus for which the reconstruction is performed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonNucleus

^

Symbol 

DixonPhases is an option for DixonReconstruct. It defines which phases to fit within the model.

The order is {T2*, B0, bipolar, initial, bipolar}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonPhases

^

Symbol 

DixonPrecessions is an option for DixonReconstruct. Defines the rotation of the signal {-1,1} default is -1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonPrecessions

^

Symbol 

DixonRelaxivity is an option for DixonReconstruct. Defines the T2 relaxation times of the fat peaks being used. Is only used if DixonFixT2 is set to True.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonRelaxivity

^

Symbol 

DixonTolerance is an option for DixonReconstruct. Defines at which change per iteration of b0 and R2star the iterative methods stops. Default value is 0.1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`DixonTolerance

^

Symbol 

MaxIterations is an option that specifies the maximum number of iterations that should be tried in various built-in functions and algorithms.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`MaxIterations

^

Symbol 

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol 

MonitorUnwrap is an option for Unwrap. Monitor the unwrapping progress.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`MonitorUnwrap

^

Symbol 

Parallelize[*expr*] evaluates *expr* using automatic parallelization.

Definitions

```
Parallelize[Parallel`Evaluate`Private`expr_, Parallel`Evaluate`Private`opts : OptionsPattern[]] :=
Module[{Parallel`Evaluate`Private`res, Parallel`Evaluate`Private`ahead, Parallel`Evaluate`Private`fopts}, Parallel`Protected`tryRelaunch[];
If[Parallel`Protected`$seQ, Message[Parallelize::nopar];
Return[Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`fopts = Sequence @@ {Method → OptionValue[Method],
DistributedContexts → Parallel`Protected`DistOptCheck[Parallelize, OptionValue[DistributedContexts]], ProgressReporting → OptionValue[ProgressReporting]};
Parallel`Evaluate`Private`ahead = Replace[Head[Unevaluated[Parallel`Evaluate`Private`expr], Hold],
{Hold[Parallel`Evaluate`Private`s_Symbol] => Parallel`Evaluate`Private`s, _ => $Failed}];
With[{Parallel`Evaluate`Private`handler = Lookup[Parallel`Evaluate`$ParallelExtensions, Parallel`Evaluate`Private`ahead, $Failed]},
If[Parallel`Evaluate`Private`handler != $Failed, Return[Parallel`Evaluate`Private`handler[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts]]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`tryCombine[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`tryCombine, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`wrapAround[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`wrapAround, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`silentFail[Parallel`Evaluate`Private`expr];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`silentFail, Return[Parallel`Evaluate`Private`res]];
If[Parallel`Evaluate`Private`$seqWarning, Message[Parallelize::nopar1, Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`expr]
```

Documentation [Local »](#) | [Web »](#)

Options {DistributedContexts => \$Context, Method → Automatic, ProgressReporting => \$ProgressReporting}

Attributes {HoldFirst, Protected}

Full Name System`Parallelize

^

Symbol 

PhaseEchos is an option for DixonPhase. Can be Automatic or a list of two integers.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`PhaseEchos

^

Symbol i

UnwrapDimension is an option for Unwrap. Can be "2D" or "3D". 2D is for unwrapping 2D images or unwrapping the individual images from a 3D dataset (does not unwrap in the slice direction). 3D unwraps a 3D dataset in all dimensions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`UnwrapDimension

^

Symbol i

UnwrapThresh is an option for Unwrap. Is a value between 0.6 and 0.9, and defines when to unwrap, the higher the value the less unwrapping will be done.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`DixonTools`UnwrapThresh

^

ElastixTools

Functions

Symbol i

ClearQMRIToolsTemp[] clears the temporary directory used by QMRITools for registration.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ClearQMRIToolsTemp] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`ClearQMRIToolsTemp

^

Symbol i

ReadTransformParameters[directory] reads the transformation parameters generated by RegisterData. The directory should be the TempDirectory were the registration is stored. DeleteTempDirectory should be False.

Output is the affine transformation vector per volume.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ReadTransformParameters] = {ArgumentsPattern → {_,_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`ReadTransformParameters

^

Symbol i

RegisterCardiacData[data] registers the data using a 2D algorithm. data can be 3D or 4D.

RegisterCardiacData[{data,vox}] registers the data series using the given voxel size.

RegisterCardiacData[{data,mask}] registers the data series only using data within the mask.

RegisterCardiacData[{data,mask,vox}] registers the data series using the given voxel size only using data within the mask.

Output is the registered data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterCardiacData] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options ▶ RegistrationTarget → Mean ... (18 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterCardiacData

^

Symbol i

RegisterData[data] registers the data series. If data is 3D it performs multiple 2D registration, if data is 4D it performs multiple 3D registration.

The input data can be in the forms: data, {data, vox}, {data, mask} or {data, mask, vox}.

RegisterData[target, moving] registers the moving data to the target data. target can be 2D or 3D. moving can be the same of one dimension higher than the target.

The inputs target and moving can be in the forms: data, {data, vox}, {data, mask} or {data, mask, vox}.

Output is the registered data with the dimensions of the moving data.

If OutputTransformation is True it also outputs the translation, rotation scale and skew of all images or volumes.

RegisterData[] is based on DOI: 10.1109/TMI.2009.2035616 and 10.3389/fninf.2013.00050.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterData] = {ArgumentsPattern → {_, ___, OptionsPattern[]}}

Options ▶ Iterations → 250 ... (17 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterData

^

Symbol



RegisterDataSplit[target, moving] is identical to RegisterData data however left and right side of the data are registered separately.

Splitting the data is done using the function CutData and merged with Stich data.

Output is the registered data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterDataSplit] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options ▶ Iterations → 250 ... (18 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterDataSplit



Symbol



RegisterDataTransform[target, moving, {moving2nd, vox}] performs the registration exactly as RegisterData. target and moving are the inputs for Registerdata, which can be {data,mask,vox}.

After the registration is done the moving2nd data is deformed according to the output of the registration of moving.

moving2nd can have the same dimensions of moving or one dimension higher (e.g. 3D and 3D or 3D and 4D).

Output is {registered moving, deformed moving2nd}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterDataTransform] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options ▶ Iterations → 250 ... (19 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterDataTransform



Symbol



RegisterDataTransformSplit[target, moving, {moving2nd, vox}] is identical to RegisterDataTransform with the same functionality as RegisterDataSplit.

This means the data is split in two using the function CutData and merged with Stich data.

Output is {registered moving, deformed moving2nd}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterDataTransformSplit] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ▶ Iterations → 250 ... (19 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterDataTransformSplit



Symbol



RegisterDiffusionData[{dtidata, vox}] registers a diffusion dataset. dtidata should be 4D {slice, diff, x, y}. vox is the voxelsize of the data.

RegisterDiffusionData[{dtidata, dtimask, vox}] registers the data series using the given voxel size only using data within the mask.

RegisterDiffusionData[{dtidata, vox}, {anatdata, vox}] registers a diffusion dataset. The diffusion data is also registered to the anatdata.

RegisterDiffusionData[{dtidata, dtimask, vox}, {anatdata, vox}] registers the data series using the given voxel size only using data within the mask.

RegisterDiffusionData[{dtidata, vox}, {anatdata, anatmask, vox}] registers the data series using the given voxel size only using data within the mask.

RegisterDiffusionData[{dtidata, dtimask, vox}, {anatdata, anatmask, vox}] registers the data series using the given voxel size only using data within the mask.

Output is the registered dtidata and, if anatdata is given, the registered dtidata in

anatomical space. If OutputTransformation is True it also outputs the translation, rotation scale and skew of all images or volumes.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterDiffusionData] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ▶ Iterations → 250 ... (24 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterDiffusionData



Symbol i

RegisterDiffusionDataSplit[dtidata, vox] is identical to Register diffusion data however left and right side of the data are registered separately.

RegisterDiffusionDataSplit[{dtidata, vox}, {anatdata, vox}] is identical to Register diffusion data however left and right side of the data are registered separately.

RegisterDiffusionDataSplit[{dtidata, dtimask, vox}, {anatdata, anatmask, vox}] is identical to Register diffusion data however left and right side of the data are registered separately.

Splitting the data is done using the function CutData and merged with Stich data.

Output is the registered data.

Documentation [Local »](#)

Default Definitions Options[RegisterDiffusionDataSplit] := Options[RegisterDiffusionData]

SyntaxInformation[RegisterDiffusionDataSplit] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterDiffusionDataSplit

^

Symbol i

RegisterTensorData[target, moving] registers the moving tensor to the target tensor. After registration the tensor is reorientated appropriately.

The inputs target and moving can be in the forms: tensor, {tensor, vox}, {tensor, mask} or {tensor, mask, vox}.

RegisterTensorData[target, moving, tensor] registers the moving data to the target data and applies the deformation to the tensor.

The inputs target and moving can be in the form: data, {data, vox}, {data, mask} or {data, mask, vox}.

The input tensor can be in the form: tensor or {tensor, vox}.

only "rigid", "affine" and "bspline" can be used as MetReg.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RegisterTensorData] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ▶ Iterations → 250 ... (17 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`RegisterTensorData

^

Symbol i

TransformData[{data,vox}] deforms the data according to the last output of register data.

The directory should be the TempDirectory were the registration is stored. DeleteTempDirectory should be False.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TransformData] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options ▶ TempDirectory → Default ... (6 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`TransformData

^

Symbol i

QMRITools`ElastixTools`\$debugElastix

Documentation [Local »](#)

Attributes {ReadProtected}

Full Name QMRITools`ElastixTools`\$debugElastix

^

Symbol i

QMRITools`ElastixTools`\$lastElastixTemp

Documentation [Local »](#)

Attributes {ReadProtected}

Full Name QMRITools`ElastixTools`\$lastElastixTemp

^

Options

Symbol i

AffineDirections is an option for RegisterData ad RegisterDiffusionData.

It gives the directions in which data can be moved when registering diffusion data to anatomical space.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`AffineDirections

^

Symbol i

BsplineDirections is an option for RegisterData and RegisterDiffusionData.

It gives the direction in which the bsplines are allowed to move when registering diffusion data to anatomical space.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`BsplineDirections

^

Symbol i

BsplineSpacing is an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.

It specifies the spacing of the bsplines if the method is "bspline".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`BsplineSpacing

^

Symbol i

DeleteTempDirectory an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.

It specifies if the temp directory should be deleted after the registration is finished.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`DeleteTempDirectory

^

Symbol i

FindTransform is an option for TransformData and RegisterTransformData. It specifies where to find the transformfile.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`FindTransform

^

Symbol 

HistogramBins is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform. It specifies the number of bins of the joined histogram used by the registration functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`HistogramBins



Symbol 

HistogramBinsA is an option for RegisterDiffusionData. It specifies the number of bins of the joined histogram used when registering diffusion data to anatomical space.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`HistogramBinsA



Symbol 

ImportDeformation is an option for TransformData and RegisterTransformData. It specifies if the transformation field which is generated by Transformix should be imported along side the transformation result.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`ImportDeformation



Symbol 

InterpolationOrderReg is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform. It specifies the interpolation order used in the registration functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`InterpolationOrderReg



Symbol 

InterpolationOrderRegA is an option for RegisterDiffusionData.

It specifies the interpolation order used in the registration functions when registering diffusion data to anatomical space.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`InterpolationOrderRegA

^

Symbol 

Iterations is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform.

It specifies the number of iterations used by the registration functions.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`Iterations

^

Symbol 

IterationsA is an option for RegisterDiffusionData.

It specifies the number of iterations used when registering diffusion data to anatomical space.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`IterationsA

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`Method

^

Symbol i

MethodReg is an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.
It specifies which registration method to use.
Methods can be be "translation", "rigid", "affine", "bspline", "rigidDTI", "affineDTI", "PCAtranslation", "PCArigid", "PCAaffine", or "PCAbspline".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`MethodReg

^

Symbol i

MethodRegA is an option for RegisterDiffusionData.
It specifies which registration method to use when registering diffusion data to anatomical space. Methods can be be "rigid", "affine" or "bspline".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`MethodRegA

^

Symbol i

NumberSamples is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform.
It specifies the number of random samples that are taken each iteration used by the registration functions.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`NumberSamples

^

Symbol i

NumberSamplesA is an option for RegisterDiffusionData.
It specifies the number of random samples that are taken each iteration when registering diffusion data to anatomical space.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`NumberSamplesA

^

Symbol i

OutputImage is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform.

It specifies if the result image should be written in the TempDirectory as nii file.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`OutputImage

^

Symbol i

OutputTransformation is an option for RegisterData ad RegisterDiffusionData.

It specifies if the transformation parameters (translation, rotation, scale and skew) should be given as output in the registration functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`OutputTransformation

^

Symbol i

PCAComponents is an option for RegisterData. It specifies how many PCA components are used if method is set to "PCA".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`PCAComponents

^

Symbol i

PrintTempDirectory is an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.

It specifies if the location of the temp directory should be displayed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`PrintTempDirectory

^

Symbol i

RegistrationTarget is an option for RegisterDiffusionData and RegisterCardiacData. Specifies which target to uses for registration if using "rigid", "affine" or "bspline" as MethodReg.
If the MethodReg is "PCA" based it does not need a target and this options does nothing.
Values can be "First", "Mean" or "Median".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`RegistrationTarget

^

Symbol i

Resolutions is an options for RegisterData, RegisterDiffusionData, and RegisterDataTransform.
It specifies the number of scale space resolutions used by the registration functions.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`Resolutions

^

Symbol i

ResolutionsA is an option for RegisterDiffusionData.
It specifies the number of scale space resolutions used when registering diffusion data to anatomical space.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`ResolutionsA

^

Symbol i

ShowMetric is an option for RegisterData. If set to True it will print a plot of the Metric.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ElastixTools`ShowMetric

^

Symbol i

SplitMethod is an option for RegisterDataSplit and RegisterDataTransformSplit. values can be "mean", "moving", "target".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`SplitMethod

^

Symbol i

TempDirectory is an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.

It specifies the temporary directory used to perform and output the registration.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`TempDirectory

^

Symbol i

TransformMethod is an option for RegisterDataSplit and RegisterDataTransformSplit. values can be "Data", "Mask", "Segmentation".

If set to "Mask" a binary mask is expected as the second moving input.

If set to "Segmentation" a multi label segmentation is expected as the second moving input.

Uses SplitSegmentations internally then.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`TransformMethod

^

Symbol i

UseGPU is an option for RegisterData. The value is {bool, gpu} where bool is True or False, and gpu is the gpu ID which is an integer or Automatic.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`UseGPU

^

FasciculationTools

Functions

Symbol i

AnalyzeActivations[actMap, mask] Analysis of the activation map generated from the mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`AnalyzeActivations

^

Symbol i

EvaluateActivation[out] allows to evaluate the activation detection using FindActivations, where out is the output of that function with the option ActivationOutput set to True.

EvaluateActivation[out, actS] The same with the extra analysis of the SelectActivations function output given as actS.

Documentation [Local »](#)

Default Definitions SyntaxInformation[EvaluateActivation] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`EvaluateActivation

^

Symbol i

FindActivations[data] Finds the activation in MUMRI or DTI data after data normalization.

FindActivations[data, mask] Finds the activation in MUMRI or DTI data after data normalization within the mask.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FindActivations] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ▶ ActivationThreshold → {3., 0.75}... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`FindActivations

^

Symbol i

NormalizeFasciculationData[data_, mask, {tens, grad, val}] will normalize DWI data based on the tensor fit.

For each diffusion direction and b-value the estimated unweighted image is calculated.

Documentation [Local »](#)

Default Definitions SyntaxInformation[NormalizeFasciculationData] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`NormalizeFasciculationData

^

Symbol i

SelectActivations[act] selects the activations above the given ActivationSize.

SelectActivations[act, vox] selects the activations above the given ActivationSize where the activation size is in mm^3 .

SelectActivations[act, mask] selects the activations above the given ActivationSize within the given mask or masks. The mask can be 3D or 4D.

SelectActivations[act, {mask, back}] selects the activations above the given ActivationSize within the given mask or masks. All voxels outside the back are ignored.

Output is {actSelected, actTotal} is mask is 3D.

Output is {{actSelected, Total[actSelected]}, {actTotal, Total[actTotal]}} is mask is 4D where actSelected and actTotal are per mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SelectActivations] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options `{ActivationSize → 4, IgnoreSlices → {0, 0}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`FasciculationTools`SelectActivations`

^

Options

Symbol i

ActivationBackground is an option for FindActivations. If all normalized signals, which range between 0–150, are below this value the algorithm does nothing.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`FasciculationTools`ActivationBackground`

^

Symbol i

ActivationIterations is an option for FindActivations. The maximum number of iteration that can be used for activation detection.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`FasciculationTools`ActivationIterations`

^

Symbol 

ActivationOutput is an option for ActivationOutput. If set to All also the mn and threshold values are returned.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`ActivationOutput

^

Symbol 

ActivationSize is an option for SelectActivations. Its the size of the activations selected defined in number of voxels if no voxel size is given. If a voxel size is given its the volume.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`ActivationSize

^

Symbol 

ActivationThreshold is an option for FindActivations. Fist value is the number of standard deviations second is the percentage threshold.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`ActivationThreshold

^

Symbol 

IgnoreSlices is an option for FindActivations and SelectActivations. determines how many slices of the start and end of the dataset are ignored.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`IgnoreSlices

^

Symbol 

MaskDilation is an option for Mask, SmoothMask and SmoothSegmentation. If the value is greater than 0 it will dilate the mask, if the value is smaller than 0 it will erode the mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskDilation

^

Symbol i

ThresholdMethod is an option for FindActivations. Values can be "StandardDeviation", "Fraction" or "Both".

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`FasciculationTools`ThresholdMethod

^

GeneralTools

Functions

Symbol i

ApplyCrop[data, crop] applies the cropped region obtained form CropData to the data.

ApplyCrop[data, crop, {voxorig, voxnew}] applies the cropped region obtained form CropData to the data.

Documentation [Local](#) »

Default Definitions SyntaxInformation[ApplyCrop] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options CropAlways → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`ApplyCrop

^

Symbol i

AutoCropData[data] crops the data by removing all background zeros.

Documentation [Local](#) »

Default Definitions SyntaxInformation[AutoCropData] = {ArgumentsPattern → {_, OptionsPattern[]}

Options CropPadding → 5

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`AutoCropData

^

Symbol i

BSplineCurveFit[points] fits a b-spline to the points. Output is a list of same size as points.

Documentation [Local](#) »

Default Definitions SyntaxInformation[BSplineCurveFit] = {ArgumentsPattern → {_, OptionsPattern[]}

Options {SplineDegree → 2, SplineKnotsNumber → 50, SplineRegularization → 0}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`BSplineCurveFit

^

Symbol i

ClearTemporaryVariables[] Clear temporary variables.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ClearTemporaryVariables] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ClearTemporaryVariables`

^

Symbol i

CompilableFunctions[] generates a formatted table of all compilable functions generated by `Compile`CompilerFunctions`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CompilableFunctions] = {ArgumentsPattern → {}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`CompilableFunctions`

^

Symbol i

ConvertExtension[filename, extension] checks if file has correct extension. Removes .gz or changes the extension or adds extension if not present.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ConvertExtension] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ConvertExtension`

^

Symbol i

CropData[data] creates a dialog window to crop the data (assumes voxel size (1,1,1)).

CropData[data, vox] creates a dialog window to crop the data.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CropData] = {ArgumentsPattern → {...}, OptionsPattern[]}`

Options {CropOutput → All, CropInit → Automatic}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`CropData`

^

Symbol 

CutData[data] splits the data in two sets left and right base on a automated algorithm assuming leg data.

CutData[data, cut] splits the data in two sets left and right at location cut.

CutData[data, {cut, over}] splits the data in two sets left and right at location cut but with overlap defined by over.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CutData] = {ArgumentsPattern → {_, _}}`

Options `Monitor → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`CutData`

^

Symbol 

DataToVector[data] converts the non zero data to vector.

DataToVector[data, mask] converts the data within the mask to vector.

The data can be reconstructed using VectorToData.

Output is the vectorized data and a list containing the original data dimensions and a list with the data coordinates. {vec, {dim,pos}}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DataToVector] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`DataToVector`

^

Symbol 

DateName[] gives the current date and time in the format "YYYYMMDD-HH".

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`DateName`

^

Symbol i

DecomposeAffineMatrix[S] decomposes the scale matrix in S1, S2 and S3.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DecomposeAffineMatrix] = {ArgumentsPattern → {…}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`DecomposeAffineMatrix`

^

Symbol i

DecomposeScaleMatrix[mat] decomposes the affine matrix in T, R, S and Q.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DecomposeScaleMatrix] = {ArgumentsPattern → {…}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`DecomposeScaleMatrix`

^

Symbol i

DivideNoZero[a, b] divides a/b but when b=0 the result is 0. a can be a number or vector.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DivideNoZero] = {ArgumentsPattern → {…}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`DivideNoZero`

^

Symbol i

DynamicPartition[data, {part}] partitions the data into parts which is a list of integers. The remainders is los.

DynamicPartition[data,part,last] partitions the data into parts which is a list of integers. The remainders is partitioned into equal parts defined by last.

If last is All, the remainders is just one partition.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DynamicPartition] = {ArgumentsPattern → {…}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`DynamicPartition`

^

Symbol i

EmptyDirectoryQ[dir] checks if directory dir is empty.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EmptyDirectoryQ] = {ArgumentsPattern → {}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`EmptyDirectoryQ`

^

Symbol i

ExpNoZero[val] return the Exp of the val which can be any dimension array. if val=0 the output is 0.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExpNoZero] = {ArgumentsPattern → {}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ExpNoZero`

^

Symbol i

ExtractDemoData[] Extracts the demo data archive.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ExtractDemoData`

^

Symbol i

FileSelect[action] creates a system dialog which returns file/foldername action can be "FileOpen", "FileSave" or "Directory".

FileSelect[action, {type}] same but allows the definition of filetypes for "FileOpen" and "FileSave" e.g. "jpg" or "pdf".

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FileSelect] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}`

Options WindowTitle → Automatic

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`FileSelect`

^

Symbol 

FindCrop[data] finds the crop values of the data by removing all zeros surrounding the data.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindCrop] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `CropPadding → 5`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`FindCrop`

^

Symbol 

FindMaxDimensions[[data1, data2, ...]] finds the maximal dimensions of all datasets. Each dataset is 3D.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindMaxDimensions] = {ArgumentsPattern → {_, ...}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`FindMaxDimensions`

^

Symbol 

FindMiddle[data] find the center of a two leg data.

FindMiddle[data, what] find the center of the data for the anatomy defined by what.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindMiddle] = {ArgumentsPattern → {_, ...}}`

Options `Monitor → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`FindMiddle`

^

Symbol 

GetAssetLocation[name] Gets the location of the executable assets of the package for the highest installed version.

Current assess are "Elastix", "Transformix" and "DcmToNii".

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`GetAssetLocation`

^

Symbol i

GridData[[data1,data2,...], part] makes a grid of multiple datasets with part sets on each row.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GridData] = {ArgumentsPattern → {_, ...}, OptionsPattern[]}`

Options `Padding → None`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`GridData`

^

Symbol i

GridData3D[[data1,data2,...], part] same as grid data, but only works on 4D data where the data is gridded in axial, coronal and sagittal.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GridData3D] = {ArgumentsPattern → {_, ...}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`GridData3D`

^

Symbol i

GyromagneticRatio[] gives the gyromagnetic ratio for "1H" in MHz/T.

GyromagneticRatio[nuclei] gives the gyromagnetic ratio for the nuclei, e.g. "31P" of "1H".

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`GyromagneticRatio`

^

Symbol i

LapFilter[data] Laplacian filter of data with kernel size 0.8.

LapFilter[data, ker] Laplacian filter of data with kernel ker.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`LapFilter`

^

Symbol 

LightDarkV[] gives Black for light mode and White for dark mode in version 14.3+ else it gives White.

LightDarkV[light, dark] does the same as LighhDarkSwitched but only for version 14.3 or higher.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`LightDarkV

^

Symbol 

LLeastSquares[A, y] = performs a Linear Linear Least Squares fit.

It uses a compiled version of the Pseudo inverse of A.

Documentation [Local »](#)

Default Definitions SyntaxInformation[LLeastSquares] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`LLeastSquares

^

Symbol 

LogNoZero[val] return the log of the val which can be any dimension array. if val=0 the output is 0.

Documentation [Local »](#)

Default Definitions SyntaxInformation[LogNoZero] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`LogNoZero

^

Symbol 

MADNoZero[vec] return the MAD error of the vec which can be any dimension array. if vec={0...} the output is 0. Zeros are ignored.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MADNoZero] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`MADNoZero

^

Symbol i

MakeCoordinates[data, vox] gives the coordinates of every voxel.

MakeCoordinates[dim, vox] gives the coordinates of every voxel for a dataset with dimensions dim.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeCoordinates] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`MakeCoordinates`

^

Symbol i

MakeFunctionGraph[function] makes a function dependency graph of the function.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeFunctionGraph] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{LabelPlacement → Tooltip, AllowSelfDependencies → False, FilterInternal → False}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`MakeFunctionGraph`

^

Symbol i

MakeIntFunction[data] makes an interpolation function of the data using voxel size {1, 1, 1} and interpolation order 1

MakeIntFunction[data, int] makes an interpolation function of the data using voxel size {1, 1, 1} and interpolation order int.

MakeIntFunction[data, vox, int] makes an interpolation function of the data using voxel size vox and interpolation order int.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeIntFunction] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options `{CenterVoxel → True, CenterRange → False}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`MakeIntFunction`

^

Symbol i

MBCount[expr] gives the memory usage of the expression in MB.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`MBCount`

^

Symbol 

MeanNoZero[data] calculates the mean of the data ignoring the zeros.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MeanNoZero] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`MeanNoZero`

^

Symbol 

MedFilter[data] Median filter of data with kernel size 1.

MedFilter[data, ker] Median filter of data with kernel ker.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`MedFilter`

^

Symbol 

MedianNoZero[data] calculates the Median of the data ignoring the zeros.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MedianNoZero] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`MedianNoZero`

^

Symbol 

MemoryUsage[] gives a table of which definitions use up memory.

MemoryUsage[n] gives a table of which definitions use up memory, where n is the amount of definitions to show.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MemoryUsage] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`MemoryUsage`

^

Symbol i

MonitorFunction[] either prints or echos its input.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`MonitorFunction

^

Symbol i

NiiFileExistQ[file] checks if the *.nii or *.nii.gz file exists.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`NiiFileExistQ

^

Symbol i

NNLeastSquares[A, y] performs a Non Negative Linear Least Squares fit.

finds an x that solves the linear least-squares problem for the matrix equation $Ax=y$.

output is the solution x.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[NNLeastSquares] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`NNLeastSquares

^

Symbol i

OpenDemonstrationNotebook[] Opens the demonstration notebook.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`OpenDemonstrationNotebook

^

Symbol 

OpenQMRIToolsLocation[] Opens the QMRITools location in the file explorer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`OpenQMRIToolsLocation

^

Symbol 

PadToDimensions[data] pads the data to the max dimensions of data, using FindMaxDimensions.

PadToDimensions[data, dim] pads the data to dimensions dim.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PadToDimensions] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {PadValue → 0., PadDirection → Center}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`PadToDimensions

^

Symbol 

ParseCommandLine[arguments] parses command line arguments and in qmrtools is used for using arguments given with command line scripts for processing.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`ParseCommandLine

^

Symbol 

QMRIToolsFuncPrint[] gives a list of all the QMRITools functions with their usage information.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[QMRIToolsFuncPrint] = {ArgumentsPattern → {_.}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`QMRIToolsFuncPrint

^

Symbol i

QMRIToolsFunctions[] give list of all the QMRITools packages, functions and options.

QMRIToolsFunctions[p] print a table with length p of all the QMRITools functions and options.

QMRIToolsFunctions["toolbox"] gives a list of all the functions and options in toolbox.

QMRIToolsFunctions["toolbox", p] gives a table of length p of all the functions and options in toolbox. If toolbox is "All" it will list all toolboxes.

Documentation [Local »](#)

Default Definitions SyntaxInformation[QMRIToolsFunctions] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`QMRIToolsFunctions

^

Symbol i

QMRIToolsPackages[] give list of all the QMRITools packages.

Documentation [Local »](#)

Default Definitions SyntaxInformation[QMRIToolsPackages] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`QMRIToolsPackages

^

Symbol i

QuaternionToRotationMatrix[{a, b,c,d}] converts quaternion to rotation matrix R.

Documentation [Local »](#)

Default Definitions SyntaxInformation[QuaternionToRotationMatrix] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`QuaternionToRotationMatrix

^

Symbol i

QuaternionVectorToRotationMatrix[{b,c,d}] converts quaternion to rotation matrix R.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`QuaternionVectorToRotationMatrix

^

Symbol 

RandomString[n] generates a random string of length n with letters and numbers.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RandomString] = {ArgumentsPattern → {  }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RandomString`

^

Symbol 

RescaleData[data, dim] rescales the data to given dimensions.

RescaleData[data, {vox1, vox2}] rescales the data from size vox1 to size vox2.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RescaleData] = {ArgumentsPattern → {  ,   , OptionsPattern[]}}`

Options `InterpolationOrder → 3`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RescaleData`

^

Symbol 

ReverseCrop[data, dim, crop] reverses the crop on the cropped data with crop values crop to the original size dim.

ReverseCrop[data, dim, crop, {voxorig, voxnew}] reverses the crop on the cropped data with crop values crop to the original size dim.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReverseCrop] = {ArgumentsPattern → {  ,   ,   ,   }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ReverseCrop`

^

Symbol 

ReverseDimensions[data] reverses the dimensions of the data.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReverseDimensions] = {ArgumentsPattern → {  ,   }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`ReverseDimensions`

^

Symbol i

RMSNoZero[vec] return the RMS error of the vec which can be any dimension array. if vec={0...} the output is 0. Zeros are ignored.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RMSNoZero] = {ArgumentsPattern → { _ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RMSNoZero`

^

Symbol i

RotateDimensionsLeft[data] rotates the dimensions of the data one to the left.

RotateDimensionsLeft[data, i] rotates the dimensions of the data i to the left.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotateDimensionsLeft] = {ArgumentsPattern → { _ , _ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RotateDimensionsLeft`

^

Symbol i

RotateDimensionsRight[data] rotates the dimensions of the data one to the right.

RotateDimensionsRight[data, i] rotates the dimensions of the data i to the right.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotateDimensionsRight] = {ArgumentsPattern → { _ , _ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RotateDimensionsRight`

^

Symbol i

RotationMatrixToQuaternion[R] converts rotation matrix to quaternions {a, b,c,d}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotationMatrixToQuaternion] = {ArgumentsPattern → { _ }}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RotationMatrixToQuaternion`

^

Symbol i

RotationMatrixToQuaternionVector[R] converts rotation matrix to quaternions {b,c,d}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotationMatrixToQuaternionVector] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`RotationMatrixToQuaternionVector`

^

Symbol i

SaveImage[image] exports graph to image, ImageSize, FileType and ImageResolution can be given as options.

SaveImage[image, "filename"] exports graph to image with "filename", ImageSize, FileType and ImageResolution can be given as options.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SaveImage] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {ImageSize → 6000, FileType → .jpg, ImageResolution → 300}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`SaveImage`

^

Symbol i

SetDemoDirectory[] Sets the directory to the demo data directory.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`SetDemoDirectory`

^

Symbol i

SignNoZero[val] gives the sign of the val, where the sign of val > 0 is 1 and val < 0 is -1.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SignNoZero] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GeneralTools`SignNoZero`

^

Symbol i

Squeeze[data] Removes the singleton dimensions from data.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Squeeze] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`Squeeze

^

Symbol i

StandardDeviationNoZero[data] calculates the mean of the data ignoring the zeros.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[StandardDeviationNoZero] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`StandardDeviationNoZero

^

Symbol i

StdFilter[data] StandardDeviation filter of data using gaussian kernel 2.

StdFilter[data, ker] StandardDeviation filter of data using kernel with size ker.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`StdFilter

^

Symbol i

StichData[data,datar] joins left and right part of the data generated by CutData.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[StichData] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`StichData

^

Symbol i

StringPadInteger[num] converts the integer num to a string and pads it with zeros to length 3.

StringPadInteger[{num, len}] converts the integer num to a string and pads it with zeros to length len.

StringPadInteger[pre, num] the same but with prefix pre.

StringPadInteger[pre, {num, len}] the same but with prefix pre.

StringPadInteger[num, post] the same but with postfix post.

StringPadInteger[{num, len}, post] the same but with postfix post.

StringPadInteger[pre, num, post] the same but with pre and postfix pre and post.

StringPadInteger[post, {num, len}, post] the same but with pre and postfix pre and post.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[StringPadInteger] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`StringPadInteger

^

Symbol i

SumOfSquares[{data1, data2, ..., datan}] calculates the sum of squares of the datasets.

Output is the SoS and the weights, or just the SoS.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SumOfSquares] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options OutputWeights → True

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`SumOfSquares

^

Symbol i

TensMat[tensor] transforms tensor form vector format {xx,yy,zz,xy,xz,yz} to matrix format {{xx,xy,xz},{xy,yy,yz},{xz,yz,zz}}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TensMat] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`TensMat

^

Symbol i

TensVec[tensor] transforms tensor form matrix format $\{\{xx,xy,xz\},\{xy,yy,yz\},\{xz,yz,zz\}\}$ to vector format $\{xx,yy,zz,xy,xz,yz\}$.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TensVec] = {ArgumentsPattern → {...}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`TensVec`

^

Symbol i

VectorToData[vec, {dim, pos}] converts the vectorized data from DataToVector back to its original Dimensions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[VectorToData] = {ArgumentsPattern → {..., {..., ...}}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`VectorToData`

^

Options

Symbol i

AllowSelfDependencies is an option for MakeFunctionGraph. Can be True or False. If True a function that calls itself is also shown.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`AllowSelfDependencies`

^

Symbol i

CenterRange is an option for MakeIntFunction. If set True the centers of the dataset is the origin else its the corner.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`CenterRange`

^

Symbol i

CenterVoxel is an option for MakeIntFunction. If set True the centers of the voxels are interpolated else its the corners.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GeneralTools`CenterVoxel`

^

Symbol i

CropAlways is an option for ApplyCrop. If set True is will always crop even if outside the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`CropAlways

^

Symbol i

CropInit is an option for CropData. By default the crop is not initialized bu can be with {{xmin,xmax},{ymin,ymax},{zmin,zmax}}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`CropInit

^

Symbol i

CropOutput is an option for CropData, can be "All", "Data" or "Crop".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`CropOutput

^

Symbol i

CropPadding is an option for AutoCropData or FindCrop. It specifies how much padding to use around the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`CropPadding

^

Symbol i

FileType["file"] gives the type of a file, typically File, Directory or None.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`FileType

^

Symbol

FilterInternal is an option for `MakeFunctionGraph`. If set to `True` it will filter out all internal functions.

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`FilterInternal



Symbol



ImageResolution is an option for `Export`, `Rasterize`, and related functions that specifies at what resolution bitmap images should be rendered.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageResolution



Symbol



ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize



Symbol



InterpolationOrder is an option for `Interpolation`, as well as `ListLinePlot`, `ListPlot3D`, `ListContourPlot`, and related functions, that specifies what order of interpolation to use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`InterpolationOrder



Symbol



LabelPlacement is an option for `MakeFunctionGraph`. Defines where to place the label of the function graph. Accepts values that can be used in `Placed`.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`LabelPlacement



Symbol i

Monitor[*expr*, *mon*] generates a temporary monitor cell in which the continually updated current value of *mon* is displayed during the course of evaluation of *expr*.

Documentation [Local »](#) | [Web »](#)
Attributes {HoldAll, Protected, ReadProtected}
Full Name System`Monitor
^

Symbol i

OutputWeights is an option for SumOfSquares. If True it also output the SoS weights.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`GeneralTools`OutputWeights
^

Symbol i

Padding is an option to various array and image operations that specifies what padding to use when extending beyond the original data specified.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected, ReadProtected}
Full Name System`Padding
^

Symbol i

PadDirection is an option for PadToDimensions. It specifies the direction of padding, "Center", "Left" or "Right".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`GeneralTools`PadDirection
^

Symbol i

PadValue is an option for PadToDimensions. It specifies the value of the padding.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`GeneralTools`PadValue
^

Symbol i

SplineDegree is an option for spline functions and graphics primitives that specifies the degree of polynomial basis to use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`SplineDegree

^

Symbol i

SplineKnotsNumber is an option for BSplineCurveFit and defines how many knots the b-spline has.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`SplineKnotsNumber

^

Symbol i

SplineRegularization is an option for BSplineCurveFit and defines the amount of regularization for the linear fit.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GeneralTools`SplineRegularization

^

Symbol i

WindowTitle is an option that specifies the title to give for a window.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`WindowTitle

^

GradientTools

Functions

Symbol i

Bmatrix[bvec, grad] creates bmatrix form grad and bvec in form $\{-b_{xx}, -b_{yy}, -b_{zz}, -b_{xy}, -b_{xz}, -b_{yz}, 1\}$.

Bmatrix[{bvec, grad}] creates bmatrix form grad and bvec in form $\{b_{xx}, b_{yy}, b_{zz}, b_{xy}, b_{xz}, b_{yz}\}$.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Bmatrix] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `Method → DTI`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`Bmatrix`

^

Symbol i

BmatrixCalc["folder", grads] calculates the true bmatrix from the exported sequence parameters from the philips scanner that are stored in "folder" for each of the gradient directions grads.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BmatrixCalc] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options **▶** `UseGrad → {1, 1, {1, 1}, 1, 1} ... (8 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`BmatrixCalc`

^

Symbol i

BmatrixConv[bm] converts the bmatrix form 7 to 6 or from 6 to 7.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BmatrixConv] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`BmatrixConv`

^

Symbol i

BmatrixInv[bm] generates a bvecotr and gradient directions form a given bmatrix.

BmatrixInv[bm, bvi] generates a bvecotr and gradient directions form a given bmatrix using the given bvalues bvi.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BmatrixInv] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`BmatrixInv`

^

Symbol i

BmatrixRot[bmat, rotmat] Rotates the B-matrix.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BmatrixRot] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`BmatrixRot

^

Symbol i

BmatrixToggle[bmat, axes, flip], axes can be any order of {"x","y","z"}. flip should be {1,1,1},{1,1,-1},{1,-1,1} or {-1,1,1}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BmatrixToggle] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`BmatrixToggle

^

Symbol i

BVector[bvec, grad, gradField] Calculates the actual b-value based on the gradField. This field is calculated by the GradientCoilTensor functions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`BVector

^

Symbol i

CalculateMoments[{gt, hw, te}, t] calculates the 0th to 3th order moments of the sequence created by GradSeq. Output is {{gt, M0, M1, M2, M3}, vals}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CalculateMoments] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`CalculateMoments

^

Symbol 

ConditionNumberCalc[grads] calculates the condition number of the gradient set.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ConditionNumberCalc] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`ConditionNumberCalc

^

Symbol 

ConvertGrads[grad, bv] converts the gradients to txt format, which is needed for FinalGrads.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ConvertGrads] = {ArgumentsPattern → {_ , _ .}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`ConvertGrads

^

Symbol 

CorrectBmatrix[bmat, transformation] corrects the bmatrix bmat with the transformation parameters from RegisterData or RegisterDiffusionData.

Output is the corrected bmatrix.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CorrectBmatrix] = {ArgumentsPattern → {_ , OptionsPattern[]}}

Options MethodReg → Full

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`CorrectBmatrix

^

Symbol i

CorrectGradients[grad, transformation] corrects the gradient directions grad with the transformation parameters from RegisterData or RegisterDiffusionData.

Output is the corrected gradient vector.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CorrectGradients] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `MethodReg → Rotation`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`CorrectGradients`

^

Symbol i

EnergyCalc[grads] calculates the total Energy of the gradient set.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EnergyCalc] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`EnergyCalc`

^

Symbol i

FinalGrads[grtxt,{int,intn},{rand,order}] finalizes the gradient txt file.

grtxt is the output from the function ConvertGrads, which convert the grad to txt format.

int is True or False, if set to True it interleaves b=0 gradients every intn directions.

rand indicates if the gradients need to be randomized, for this it uses the order which is the output of FindOrder.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FinalGrads] = {ArgumentsPattern → {_, {_, _}, {_, _}}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`FinalGrads`

^

Symbol i

FindOrder[grad,bv] finds the optimal order of the gradient directions which minimizes the duty cycle.

The output is needed for FinalGrads.

grad is a list of gradient sets and bv is a list of b-values with the same number as the list of gradient sets.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FindOrder] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `OrderSpan → Auto`

Attributes `{Protected, ReadProtected}`

Full Name `QMRTTools`GradientTools`FindOrder`

^

Symbol i

FullGrad is an option for Grad. Default is True. When true the gradient directions will be loaded with the first gradient {0,0,0}.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRTTools`GradientTools`FullGrad`

^

Symbol i

GenerateGradients[numb] optimizes a set with numb gradients, numb must be an integer.

GenerateGradients[{numb, fixed}] optimizes a set with numb gradients, numb must be an integer and fixed a list of 3D coordinates e.g. {{0,0,1},{0,1,0}}. The fixed gradients will not be moved.

GenerateGradients[{numb1, numb2 ...}, alpha] optimizes a multi shell gradient set with numb gradients per shell. If alpha is set to 0.5 equal importance is given to the optimal distribution of each shell in the entire set. If alpha is 0 only the sub shells will be optimized, if alpha is set to 1 only the global set will be optimized.

GenerateGradients[] is based on DOI: 10.1002/mrm.26259 and 10.1002/(SICI)1522-2594(199909)42:3<515::AID-MRM14>3.0.CO;2-Q.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GenerateGradients] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options ▸ `Steps → 1000 ... (6 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRTTools`GradientTools`GenerateGradients`

^

Symbol i

GenerateGradientsGUI[] runs the GenerateGradients function in GUI with output for the philips system.

GenerateGradientsGUI[] is based on DOI: 10.1002/mrm.26259 and 10.1002/(SICI)1522-2594(199909)42:3<515::AID-MRM14>3.0.CO;2-Q.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`GenerateGradientsGUI

^

Symbol i

GetGradientScanOrder[grad, bval] determines the scanorder based on the txt file provided to the scanner as input.

GetGradientScanOrder[file, grad, bval] determines the scanorder based on the txt file provided to the scanner as input.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetGradientScanOrder] = {ArgumentsPattern → {_, _ ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`GetGradientScanOrder

^

Symbol i

GetSliceNormal[file] imports the slice normal from a dicom image.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetSliceNormal] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`GetSliceNormal

^

Symbol i

GetSliceNormalDir[file] imports the slice normal from a enhanced dicom image.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetSliceNormalDir] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`GetSliceNormalDir

^

Symbol i

GradBmatrix[gt, hw, te, t] Calculates the true bmatrix from the sequence created by GradSeq.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GradBmatrix] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}`

Options `{OutputPlot → False, Method → Analytical, StepSize → 0.025}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`GradBmatrix`

^

Symbol i

GradientCoilTensor[mask, vox, off, dint] makes the gradient coil tensor from the mask, with the resolution defined by vox. The offset off is the left most vox coordinate in the scanner coordinate system and the gradient tensor dint comes from the function `MakeGradientDerivatives`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GradientCoilTensor] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}`

Options `Method → Vector`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`GradientCoilTensor`

^

Symbol i

GradSeq[pars, t, grad] Creates a sequence from the gradient pars imported by `ImportGradObj`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GradSeq] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `UseGrad → {0, 1, {1, 0}, 1} ... (6 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`GradSeq`

^

Symbol i

ImportGradObj[folder] Imports the gradient par files exported from the philips scanner.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportGradObj] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`GradientTools`ImportGradObj`

^

Symbol i

MakeGradientDerivatives[vox, type] generates two interpolation functions using the resolution defined by vox. One of the gradient field and one of the dx dy and dz of each of the gradient componets.

The type is the gradient of the system which now can be "WA1" or "WA2".

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeGradientDerivatives] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GradientTools`MakeGradientDerivatives`

^

Symbol i

MakeGradientMaps[{int, dint}, fov, vox, start] will produce the gradient maps from the interpolation functions int and dint.

The field of view is defined by fov, the resolution by vox and the start is the left most coordinate in the scanner coordinate system.

The interpolation functions int and dint are generated by MakeGradientDerivatives.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeGradientMaps] = {ArgumentsPattern → {_, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GradientTools`MakeGradientMaps`

^

Symbol i

OverPlusCalc[grads] determines the minimal overplus factor of of the gradient set.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[OverPlusCalc] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`GradientTools`OverPlusCalc`

^

Symbol i

SelectBvalueData[{data, vals}, sel] selects the volumes and bvalues based on sel. The value of sel can be a number, for which all higher bvalues are selected, or a list {min, max}.

SelectBvalueData[{data, vals, grads}, sel] the same but also including the gradients.

Output is {data, vals} or {data, vals, grads}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`SelectBvalueData

^

Symbol i

UniqueBvalPosition[bval] generates a list of all the unique bvalues and their positions.

UniqueBvalPosition[bval, num] generates a list of all the unique bvalues and their positions that are present in the dataset equal or more than num times.

Documentation [Local »](#)

Default Definitions SyntaxInformation[UniqueBvalPosition] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`UniqueBvalPosition

^

Options

Symbol i

ConditionCalc is an option for GenerateGradients if set to true GenerateGradients will also give the condition number evolution of the system.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`ConditionCalc

^

Symbol i

FlipAxes is an option for GradSeq. Default value is {{1,1,1},{1,1,1}}. First three values are for diffusion gradients last three are for the acquisition gradients.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`FlipAxes

^

Symbol i

FlipGrad is an option for GradSeq. When FlipGrad is true the gr180 is flipped.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`FlipGrad

^

Symbol i

FullSphere is an option for GenerateGradients. If set True the gradients will be optimized on a full sphere rather than half a sphere.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`FullSphere

^

Symbol i

GradType is an option GenerateGradients. It specifies what type of gradient set will be produced, "Normal" or "OverPlus".

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`GradType

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local](#) » | [Web](#) »

Attributes {Protected}

Full Name System`Method

^

Symbol i

MethodReg is an options for RegisterData, RegisterDiffusionData, RegisterCardiacData and RegisterDataTransform.

It specifies which registration method to use.

Methods can be be "translation", "rigid", "affine", "bspline", "rigidDTI", "affineDTI", "PCATranslation", "PCARigid", "PCAaffine", or "PCAbspline".

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ElastixTools`MethodReg

^

Symbol 

OrderSpan is an options for FindOrder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`OrderSpan

^

Symbol 

OutputPlot is an option for GradBmatrix. It specifies if the plots of the gradients should also be exported.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`OutputPlot

^

Symbol 

OutputType is an option for BmatrixCalc. Values can be "Matrix" of "Gradients".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`OutputType

^

Symbol 

PhaseEncoding is an options of GradSeq. Values can be "A", "P", "R" and "L".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`PhaseEncoding

^

Symbol 

Runs is an option for GenerateGradients. Set how often the minimization function is run. The best solution of all runs is the output. Default value is 1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`Runs

^

Symbol 

Steps is an option GenerateGrads and is the number of step that is used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`Steps

^

Symbol 

StepSize is an option for GradBmatrix. Specifies the integration stepsize is Method -> "Numerical" is used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`StepSize

^

Symbol 

SwitchAxes is an option for GradSeq. Defaul value is {{1,2,3},{1,2,3}}. First three values are for diffusion gradients last three are for the acquisition gradients.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`SwitchAxes

^

Symbol 

UnitMulti is an option for GradSeq. Defaul value is 10^{-3} . Defines the scaling of the gradient strength.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`UnitMulti

^

Symbol 

UseGrad is an option for GradSeq. The default value is {0, 1, {1, 0}, 1} where {grex, gr180, {grepi1, grepi2}, grdiff, grflow}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`UseGrad

^

Symbol i

VisualOpt is an option for GenerateGradients. Show the minimization proces of eacht calculation step. Default is False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`GradientTools`VisualOpt

^

IVIMTools

Functions

Symbol i

BayesianIVIMFit2[data, bval, init, mask] performs bayesian IVIM fit of data.

data is the data which should be {slice, Ndiff, x, y}.

bval is the bvector would be length Ndiff.

init is the initialization of the bayesian fit which comes from IVIMCalc, (without s0 using 2 compartments).

mask is the region in which the bayesian fit is performed.

output is {f1, dc, pdc1}. The fraction is defined between 0 and 1, the dc, pdc1 is in mm²/s.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BayesianIVIMFit2] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options ▶ ChainSteps → {20000, 1000, 10} ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`BayesianIVIMFit2

^

Symbol



BayesianIVIMFit3[data, bval, init, mask] performs bayesian IVIM fit of data.

data is the data which should be {slice, Ndiff, x, y}.

bval is the bvector would be length Ndiff.

init is the initialization of the bayesian fit which comes from IVIMCalC, (without s0 using 3 compartments).

mask is the region in which the bayesian fit is performed.

output is {f1, f2, dc, pdc1, pdc2}. The fractions f1 and f2 are defined between 0 and 1, the dc, pdc1 and pdc1 is in mm²/s.

Documentation [Local »](#)

Default Definitions SyntaxInformation[BayesianIVIMFit3] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options ▶ ChainSteps → {20000, 1000, 10} ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`BayesianIVIMFit3



Symbol



CorrectParMap[par, constraints, mask] removes the IVIM parameters outside the constraints within the mask.

par is {f1, dc, pdc1} or {f1, f2, dc, pdc1, pdc2}.

constraints are the lower and upper constraints for each parameters {{min, max},...}.

mask has the same dimensions as the parameter maps.

output are the corrected parameter maps.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CorrectParMap] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`CorrectParMap



Symbol i

FConvert[f] converts the fraction f from log space.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FConvert] = {ArgumentsPattern → { $\_$ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FConvert

^

Symbol i

FConverti[f] converts the fraction f to log space.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FConverti] = {ArgumentsPattern → { $\_$ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FConverti

^

Symbol i

FracCorrect[fraction, time] corrects the signal fraction calculated with the IVIM model for tissue relaxation and acquisition parameters. After correction the signal fraction can be regarded as volume fraction.

FracCorrect[{fraction1, fraction2}, time] corrects the signal fraction1 and fraction2 from a 3 compartment IVIM model.

time is $\{\{te, tr\}, \{t2t, t21\}, \{t1t, t11\}\}$ or $\{\{te, tr\}, \{t2t, t21, t22\}, \{t1t, t11, t12\}\}$.

where t2t and t1t are "tissue" relaxation times and t11 t12, t21 and t22 the "fluid" relaxation times.

The te and tr as well as the relaxation times T2 and T1 can be defines in any time unit as long as they are consistant for all, e.g. all in ms.

output is the corrected fraction maps.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FracCorrect] = {ArgumentsPattern → { $\_$ ,  $\_$ , ...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FracCorrect

^

Symbol



HistogramPar[data, {constraints, Nbins}, style, color, range] plots histograms of IVIM solution.

HistogramPar[data, {constraints, Nbins, mu, conv}, components, color, range] plots histograms of IVIM solution.

data is {f1, dc, pdc1} or {f1, f2, dc, pdc1, pdc2}.

constraints are the ranges of the x-axes for the plots.

Nbins are the number of histogram bins.

style is the plot type, can be 1, 2, or 3.

color is the color of the histogram.

range are the ranges of the y-axes.

output is a row of histograms.

Documentation [Local »](#)

Default Definitions SyntaxInformation[HistogramPar] = {ArgumentsPattern → {_, _, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`HistogramPar



Symbol



IVIMCalc[data, binp, init] calculates the IVIM fit.

data should be 1D, 2D, 3D or 4D.

binp should be full bmatrix which can be calculated from the bvecs and bvals using Bmatrix with the bvalues in s/mm^2 .

init should be the initialization parameters for 2 components this is {s0, f, D, Dp} for 3 components this is {s0, f1, f2, D, Dp1, Dp2}.

The fraction is defined between 0 and 1, the D, Dp, Dp1 and Dp2 is in mm^2/s .

output is {s0, f1, D, pD1} or {s0, f1, f2, D, pD1, pD2}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[IVIMCalc] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}

Options ▶ Method → Automatic ... (6 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMCalc



Symbol



IVIMCorrectData[data, {s0, f, pdc}, bval] removes the ivim signal from the data.

data is the original data.

{s0, f, pdc} are the solution to a 2 compartment IVIM fit using IVIMCalc or BayesianIVIMFit2.

bval are the bvalues.

The fraction is defined between 0 and 1, the pdc is in mm^2/s .

output is the corrected data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[IVIMCorrectData] = {ArgumentsPattern → {_, {_, _, _}, _}, OptionsPattern[]}

Options {FilterMaps → True, FilterType → Median, FilterSize → 1}

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMCorrectData



Symbol i

IVIMFunction[] gives the IVIM function with 2 comps.
 IVIMFunction[components] gives the IVIM function.
 IVIMFunction[components, type] gives the IVIM function.

type can be "Normal" or "Exp".
 components can be 2 or 3.

output is the function with b, s0, f1, f2, D, pD1, pD2 as parameters. The fraction is defined between 0 and 1, the D, Dp, Dp1 and Dp2 is in mm²/s.

Documentation [Local »](#)

Default Definitions SyntaxInformation[IVIMFunction] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMFunction

^

Symbol i

IVIMResiduals[data, binp, pars] calculates the root mean square residuals of an IVIM fit using IVIMCalc, BayesianIVIMFit2 or BayesianIVIMFit3.

Documentation [Local »](#)

Default Definitions SyntaxInformation[IVIMResiduals] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMResiduals

^

Symbol i

IVIMTensFit is an option for IVIMCalc. When set True the tissue diffusion component will be calculated as a tensor.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMTensFit

^

Symbol i

MeanBvalueSignal[data, bval] calculates the geometric mean of the data for each unique bval.
output is the mean data and the unique bvalues.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MeanBvalueSignal] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`MeanBvalueSignal

^

Symbol i

ThetaConv[{f1, Fc, pDc}] converts the parameters from Log space to normal space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.
ThetaConv[{f1, f2, dc, pDc1}] converts the parameters from Log space to normal space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.
ThetaConv[{f1, f2, dc, pDc1, pDc2}] converts the parameters from Log space to normal space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`ThetaConv

^

Symbol i

ThetaConvi[{f, dc, pdc}] converts the parameters from Normal space to Log space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.
ThetaConvi[{f1, f2, dc, pdc1}] converts the parameters from Normal space to Log space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.
ThetaConvi[{f1, f2, dc, pdc1, pdc2}] converts the parameters from Normal space to Log space. Is used in BayesianIVIMFit2 and BayesianIVIMFit3.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ThetaConvi] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`ThetaConvi

^

Options

Symbol i

ChainSteps is an option for BayesianIVIMFit2 and BayesianIVIMFit3. It determines how long the algorithm runs.
three values must be given {iterations, burn steps, sample density}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`ChainSteps

^

Symbol i

CorrectPar is an option for BayesianIVIMFit2 and BayesianIVIMFit3. If True it removes the values outside the constraints using CorrectParMap.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`CorrectPar

^

Symbol i

FilterMaps is an option for IVIMCorrectData. If True the IVIM parameter maps are filtered before signal correction.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FilterMaps

^

Symbol i

FilterSize is an option for IVIMCorrectData. If FilterMaps is True it gives the kernel size.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FilterSize

^

Symbol i

FilterType is an option for IVIMCorrectData. If FilterMaps is True it tells which filter to use. can be "Median" of "Gaussian".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FilterType

^

Symbol i

FitConstraints is an option for BayesianIVIMFit2 and BayesianIVIMFit3. Gives the constraints of the parameters.

The values are used for displaying the histograms and for the initialization if CorrectPar is True.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FitConstraints

^

Symbol i

FixPseudoDiff is an option for BayesianIVIMFit2 and BayesianIVIMFit3. If the pDc1 and pD2 were fixed in IVIMCalc this value should be True.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FixPseudoDiff

^

Symbol i

FixPseudoDiffSD is an option for BayesianIVIMFit2 and BayesianIVIMFit3. Gives the standard deviation of pDc1 and pD2 if FixPseudoDiff is True.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`FixPseudoDiffSD

^

Symbol i

IVIMComponents is an option for IVIMCalc. Default value is 2, the tissue and the blood component. can also be set to 3.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMComponents

^

Symbol i

IVIMConstrained is an option for IVIMCalc. When set True the fit will be constrained to the values given in IVIMConstrains.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMConstrained

^

Symbol i

IVIMConstrains is an option for IVIMCalc.

Default values are: {{0.8, 1.2}, {0, 1}, {0.0005, 0.0035}, {0.005, 0.5}, {0.002, 0.015}}.

Where {{s0 in percentage},{fractions},{tissue diffusion},{blood compartment Dp},{third compartment}}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMConstrains

^

Symbol 

IVIMFixed is an option for IVIMCalc and the default value is False.

When set True the pseudo diffusion wil be fixed to the parameter given as init.

When set to "One" only the fast component of a 3 compartment fit is fixed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`IVIMFixed



Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method



Symbol 

OutputSamples is an option for BayesianIVIMFit2 and BayesianIVIMFit3. If set True the full marcov chain is given as an additionaln output.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`OutputSamples



Symbol

Parallelize[*expr*] evaluates *expr* using automatic parallelization.

Definitions

```
Parallelize[Parallel`Evaluate`Private`expr_, Parallel`Evaluate`Private`opts : OptionsPattern[]] :=
Module[{Parallel`Evaluate`Private`res, Parallel`Evaluate`Private`ahead, Parallel`Evaluate`Private`fopts}, Parallel`Protected`tryRelaunch[];
If[Parallel`Protected`$seQ, Message[Parallelize::nopar];
Return[Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`fopts = Sequence @@ {Method → OptionValue[Method],
DistributedContexts → Parallel`Protected`DistOptCheck[Parallelize, OptionValue[DistributedContexts]], ProgressReporting → OptionValue[ProgressReporting]};
Parallel`Evaluate`Private`ahead = Replace[Head[Unevaluated[Parallel`Evaluate`Private`expr], Hold],
{Hold[Parallel`Evaluate`Private`s_Symbol] => Parallel`Evaluate`Private`s, _ => $Failed}];
With[{Parallel`Evaluate`Private`handler = Lookup[Parallel`Evaluate`$ParallelExtensions, Parallel`Evaluate`Private`ahead, $Failed]},
If[Parallel`Evaluate`Private`handler != $Failed, Return[Parallel`Evaluate`Private`handler[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts]]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`tryCombine[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`tryCombine, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`wrapAround[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`wrapAround, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`silentFail[Parallel`Evaluate`Private`expr];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`silentFail, Return[Parallel`Evaluate`Private`res]];
If[Parallel`Evaluate`Private`$seqWarning, Message[Parallelize::nopar1, Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`expr]
```

Documentation [Local »](#) | [Web »](#)

Options {DistributedContexts => \$Context, Method → Automatic, ProgressReporting => \$ProgressReporting}

Attributes {HoldFirst, Protected}

Full Name System`Parallelize

Symbol

UpdateStep is an option for BayesianIVIMFit2 and BayesianIVIMFit3. It determines how often the parameters are updated. Is optimized during the first 500 burn steps.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`IVIMTools`UpdateStep

JcouplingTools

Functions

Symbol



GetSpinSystem[name] get a spin system that can be used in SimHamiltonian. Current implements systems are "glu", "lac", "gaba", "fatGly", "fatAll", "fatEnd", "fatDouble", "fatStart", and "fatMet".

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetSpinSystem] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options CenterFrequency → 0

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`GetSpinSystem



Symbol



MakeSpinSystem[name, freqs, jcoup] makes a spin system for jcoupling simulations. The with name is defined by the freqs of the nuclei and the jcoup values {{n1, nx}, j} between nuclei.

MakeSpinSystem[{name,labs}, freqs, jcoup] same but each nuclei has a specific name, e.g.{"ATP", {" γ ", " α ", " β "}}.

MakeSpinSystem[name, freqs, jcoup, scales] same but each nuclei has a scale, default scales are 1.

MakeSpinSystem[{name,labs}, freqs, jcoup, scales] same as alle before.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeSpinSystem] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options CenterFrequency → 0

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`MakeSpinSystem



Symbol



SequencePulseAcquire[din, ham] performs a pulsaquire experiment of the spin system din given the hamiltonian ham with a 90 Degree pulse.

SequencePulseAcquire[din, ham, b1] performs a pulsaquire experiment of the spin system din given the hamiltonian ham with a 90 Degree pulse and b1.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SequencePulseAcquire] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SequencePulseAcquire



Symbol i

SequenceSpaceEcho[din, ham, t1, t2, necho, b1] performs a multi echo spin echo experiment with a 90 degree spin echo, with t1 the time between the 90 degree RF pulse and the first 180 degree RF pulse, t2 the time between a 180 degree RF pulse and the following readout (and 2xt1 the time between two consecutive 180 degree RF pulses. Further defines necho the number of 180 degree RF pulses, din the spin system given the hamiltonian ham using b1. The t1 and t2 are defined in ms, and b1 of 100% is defines as 1. The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SequenceSpaceEcho] = {ArgumentPattern → {_, _, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SequenceSpaceEcho

^

Symbol i

SequenceSpinEcho[din, ham, te] performs a spin echo experiment with echo time te of the spin system din given the hamiltonian ham with a 90 and 180 Degree pulse. SequenceSpinEcho[din, ham, te, b1] performs a spin echo experiment with echo time te of the spin system din given the hamiltonian ham with a 90 and 180 Degree pulse and b1. The te is defined in ms and the b1 of 100% is defined as 1. The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SequenceSpinEcho] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SequenceSpinEcho

^

Symbol i

SequenceSteam[din, ham, {te, tm}] performs a stimulated echo experiment with echo time te and mixing time tm of the spin system din given the hamiltonian ham with 3 90 Degree pulses. The te and tm are defined in ms. The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SequenceSteam] = {ArgumentsPattern → {_, _ {_, _}}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SequenceSteam

^

Symbol



SequenceTSE[din ,ham, {te, necho}, {ex, ref}] performs a multi echo spin echo experiment with echo time te with necho echos of the spin system din given the hamiltonian ham using ex Degree excitation and ref Degree refocus pulses.

SequenceTSE[din ,ham, {te, necho}, {ex, ref}, b1] performs a multi echo spin echo experiment with echo time te with necho echos of the spin system din given the hamiltonian ham using ex Degree excitation and ref Degree refocus pulses and b1.

The te is defined in ms, the ex and ref are defined in degree and b1 of 100% is defined as 1.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SequenceTSE] = {ArgumentsPattern → {_, _ {_, _ _}, {_, _}, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SequenceTSE



Symbol



SimAddPhase[din ,ham ,phase] adds phase to the spin system din given the hamiltonian ham.

din and ham are generated by SimHamiltonian.

The phase is defined in degree.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimAddPhase] = {ArgumentsPattern → {_, _ _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimAddPhase



Symbol



SimEvolve[din,ham,t] evolves the spin system din given the hamiltonian ham over a time t. din and ham are generated by SimHamiltonian.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimEvolve] = {ArgumentsPattern → {_, _ _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimEvolve



Symbol i

SimHamiltonian[sysi] simulates the hamiltonian for a given spin system. The spin system is generated by GetSpinSystem.

The output is the spin system and hamiltonian structure.

SimHamiltonian[] is based on DOI: 10.1016/j.jmr.2010.12.008 and 10.1002/mrm.24340.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimHamiltonian] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options {FieldStrength → 3, SimNucleus → 1H, CenterFrequency → 0}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimHamiltonian

^

Symbol i

SimReadout[din, ham] performs a readout of a spin system din with hamiltonian ham.

Output is {time,fids,ppm,spec,dout}, which are the free induction decay fids with its time, the spectrum spec with its ppm and the evolved spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimReadout] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options > ReadoutOutput → all ... (8 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimReadout

^

Symbol i

SimRotate[din, ham ,angle] rotates the spin system din given the hamiltonian ham over angele with phase 90 degrees.

SimRotate[din, ham ,angle, phase] rotates the spin system din given the hamiltonian ham over angele with phase.

din and ham are generated by SimHamiltonian.

The angle and phase are defined in degree.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SimRotate] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimRotate

^

Symbol 

SimSignal[din, ham] performs a readout of a spin system din with hamiltonian ham.

Output is the complex signal.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimSignal] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `ReadoutOutput → all`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`JcouplingTools`SimSignal`

^

Symbol 

SimSpoil[din] spoils all the non zeroth order states of a spin system.

The output is a new spin system dout.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimSpoil] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`JcouplingTools`SimSpoil`

^

Symbol 

SysTable[sys] shows the spin system as a table. The spin system is obtained form GetSpinSystem.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SysTable] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`JcouplingTools`SysTable`

^

Options

Symbol 

CenterFrequency is an option for GetSpinSystem and defines the center frequency in ppm.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`JcouplingTools`CenterFrequency`

^

Symbol 

FieldStrength is an option for SimHamiltonian. It defines the field strength for which the hamiltonian is calculated defined in Tesla.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`FieldStrength

^

Symbol 

Linewidth is an option for SimReadout and defines the spectral linewidth in Hz.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`Linewidth

^

Symbol 

LinewidthShape is an option for SimReadout and defines the linewidth shape, values can be "Lorentzian", "Gaussian" or "Voigt".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`LinewidthShape

^

Symbol 

ReadoutBandwidth is an option for SimReadout defines the spectral bandwidth in Hz.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`ReadoutBandwidth

^

Symbol 

ReadoutMethod is an option of SimReadout and can be "Fid" or "Echo". With "Fid" it is also possible to define a delay time in ms {"Fid", delay}.
With "Echo" it is also possible to define a delay time in ms {"Echo", delay} and it then assumes te is half the readout, or a custom te can be defined {"Echo", delay, te}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`ReadoutMethod

^

Symbol



ReadoutOutput is an option for SimReadout and SimSignal and values can be "all" and "each". When set to "all" the total signal and signal is given, when set to "each" the signal or spectrum for each peak is given seperately.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`ReadoutOutput



Symbol



ReadoutPhase is an option for SimReadout and defines the readout phase in degrees.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`ReadoutPhase



Symbol



ReadoutSamples is an option for SimReadout and defines the number of readout samples for the spectrum.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`ReadoutSamples



Symbol



SimNucleus is an option for SimHamiltonian. It defines the nucleus for which to simulate the spectra.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`SimNucleus



Legacy

Functions

Symbol i

AlignRespLog[physLog, resirect, scanTime] aligns respirect and physlog data. physLog is output from ImportPhyslog. resirect is the first output from ImportRespirect.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[AlignRespLog] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {OutputMethod → val, SampleStep → 0.005}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`AlignRespLog

^

Symbol i

BvalRead[file] imports the bvalue from a .dcm file. file must be a string.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[BvalRead] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`BvalRead

^

Symbol i

DatRead[file] imports data from file (dtitool *.dat format) as binary data using Real32 format.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DatRead] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DatRead

^

Symbol i

DatWrite[file, data] exports data to *.dat file as binary data using Real32 format.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DatWrite] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DatWrite

^

Symbol



DTItoolExp[tensor, voxsize] exports tensor to {XX.dat, YY.dat, ZZ.dat, XY.dat, XZ.dat, YZ.dat} and uses XX.dat as background and generates corresponding *.dti files.

DTItoolExp[tensor, voxsize, folder] exports tensor to {XX.dat, YY.dat, ZZ.dat, XY.dat, XZ.dat, YZ.dat} to the given folder and uses XX.dat as background and generates corresponding *.dti files.

DTItoolExp[tensor, voxsize, folder, add] exports tensor to {XX.dat, YY.dat, ZZ.dat, XY.dat,

XZ.dat, YZ.dat} to the given folder and uses XX.dat as background and generates corresponding *.dti files adds – add to the filenames.

DTItoolExp[back, tensor, voxsize] exports background to back.dat and tensor to {XX.dat, YY.dat, ZZ.dat, XY.dat, XZ.dat, YZ.dat} and generates corresponding *.dti files.

DTItoolExp[back, tensor, voxsize, folder] exports background to back.dat and tensor to {XX.dat, YY.dat, ZZ.dat, XY.dat, XZ.dat, YZ.dat} to the given folder and generates corresponding *.dti files.

DTItoolExp[back, tensor, voxsize, folder, add] exports background to back.dat and tensor to

{XX.dat, YY.dat, ZZ.dat, XY.dat, XZ.dat, YZ.dat} to the given folder and generates corresponding *.dti files and adds – add to the filenames.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DTItoolExp] = {ArgumentsPattern → {_, _, _, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DTItoolExp



Symbol



DTItoolExpFile[file, background, add, voxsize] exports a *.dti text file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DTItoolExpFile] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DTItoolExpFile



Symbol



DTItoolExpInd[data, file] exports a 3D array data to the file filename DTItool format (*.dat) using DatWrite.

DTItoolExpInd[data, file ,folder] exports data to given file and folder.

DTItoolExpInd[data, file ,folder, add] exports data to given file and folder and adds –add to the filename.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DTItoolExpInd] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DTItoolExpInd



Symbol i

DTItoolExpTens[tensor] exports a diffusion tensor array to the DTItool format (*.dat).

DTItoolExpTens[tensor, add] exports tensor and adds – add to the filenames.

DTItoolExpTens[tensor, add, folder] exports tensor to the given folder and adds – add to the filenames.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DTItoolExpTens] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`DTItoolExpTens

^

Symbol i

ExportVol[filename, data, voxsize] exports a .vol and .raw file which can be loaded in DTItool 3.0.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportVol] = {ArgumentsPattern → {_, _, OptionsPattern[]}}`

Options BinaryType → Integer16

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`ExportVol

^

Symbol i

FiberDensityMap[fiberPoins, dim, vox] generates a fiber density map for the fiberPoins which are imported by LoadFiberTracts.

The dimensions dim should be the dimensions of the tracked datasets van vox its volxel size.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FiberDensityMap] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options SeedDensity → Automatic

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`FiberDensityMap

^

Symbol i

FiberLengths[fpoints, flines] calculates the fiber lenght using the output from LoadFiberTacts.

FiberLengths[{fpoints, flines}] calculates the fiber lenght using the output from LoadFiberTacts.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FiberLengths] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`FiberLengths

^

Symbol i

GradRead[filename] imports the diffusion gradient direction from a .dcm file.

filename must be a string.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GradRead] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `ConvertDcm → True`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`GradRead

^

Symbol i

ImportDTI[folder] imports xx.dat, yy.dat, zz.dat, xy.dat, xz.dat and yz.dat from the given folder.

ImportDTI[folder, add] imports xx-add.dat, yy-add.dat, zz-add.dat, xy-add.dat, xz-add.dat and yz-add.dat from the given folder.

ImportDTI[{file1, file2, ...}] imports the given *.dat files.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportDTI] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`ImportDTI

^

Symbol 

ImportPhyslog[] imports all physlog files from the folder selcted.

ImportPhyslog["forder"] imports all physlog files from "folder" selcted.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportPhyslog] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ImportPhyslog`



Symbol 

ImportRespirect[] impors all the respirect log files from the folder selcted.

ImportRespirect["folder"] impors all the respirect log files from the "folder" selcted.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportRespirect] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ImportRespirect`



Symbol 

ImportVol[] prompts for a vol file to open.

ImportVol["file"] inpormts the file.

the function returns data and voxsize.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportVol] = {ArgumentsPattern → {..., OptionsPattern[]}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ImportVol`



Symbol i

LoadFiberTracts[] prompts for a .fbs to open.

LoadFiberTracts["file"] imports the file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[LoadFiberTracts] = {ArgumentsPattern → {...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`LoadFiberTracts

^

Symbol i

PlotPhyslog[{time, resp}, {start, stop}] plots the physlog from ImportPhyslog.

PlotPhyslog[{time, resp}, {start, stop}, scanTime] plots the physlog from ImportPhyslog.

Documentation [Local »](#)

Default Definitions SyntaxInformation[PlotPhyslog] = {ArgumentsPattern → {...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`PlotPhyslog

^

Symbol i

PlotRespiract[data, dataP, scantimes] plots the resirect data to correct peaks. data and dataP are the first outputs of ImportResirect. scantimes is the output from AlignRespLog.

PlotRespiract[data, dataP, scantimes, steps].

Documentation [Local »](#)

Default Definitions SyntaxInformation[PlotRespiract] = {ArgumentsPattern → {...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`PlotRespiract

^

Symbol i

ReadBrukerDiff[""] imports the bruker diffusion data selected by the input dialog.

ReadBrukerDiff["file"] imports the bruker diffusion data from "file", file must be location of 2dseq.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadBrukerDiff] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `BmatrixOut → True`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadBrukerDiff`

^

Symbol i

ReadBvalue[folder,nr] imports the gradient directions from the dicom header of the first nr of files in de given folder.

folder must be a string, nr must be a int. Uses BvalRead.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadBvalue] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadBvalue`

^

Symbol i

ReadDicom[folder] imports all dicom files from the given folder.

ReadDicom[{file1, file2,...}] imports all the given filenames.

ReadDicom[folder, {file1, file2,...}] imports all the given filenames from the given folder.

ReadDicom[folder, partsize] imports all dicom files from the given folder and partions them in given partsize.

ReadDicom[{file1, file2, ...}, partsize] imports all the given filenames and partions them in given partsize.

ReadDicom[folder, {file1, file2, ...}, partsize] imports all the given filenames from the given folder and partions them in given partsize.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadDicom] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}`

Options `ScaleCorrect → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadDicom`

^

Symbol



ReadDicomDiff[folder, part] imports all dicom files from the given folder and the corresponding diffusion parameters.

part is the number of diffusion images per slice including the unweighted images.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadDicomDiff] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `ScaleCorrect → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadDicomDiff`



Symbol



ReadDicomDir[file] reads the image data from a dicom directory.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadDicomDir] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadDicomDir`



Symbol



ReadDicomDirDiff[file] reads the image data and relevant diffusion parameters from a dicom directory.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadDicomDirDiff] = {ArgumentsPattern → {_, OptionsPattern[]}`

Options `RotateGradient → True`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`Legacy`ReadDicomDirDiff`



Symbol i

ReadGradients[folder, nr] imports the diffusion gradient directions from the dicom header of the first nr of files in de given folder.

folder must be a string, nr must be a int. Uses GradRead.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadGradients] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ReadGradients`

^

Symbol i

ReadVoxSize[filename] imports the voxelsize from a .dcm file. filename must be a string.

Imports the pixel and slice spacing from the dicom header. Output is a list containing the voxels size {slice thickness, x, y}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadVoxSize] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ReadVoxSize`

^

Symbol i

ROIMask[maskdim, {name→{{{x,y},slice}..}}] crates mask from coordinates x and y at slice.

maskdim is the dimensions of the output {zout,xout,yout}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ROIMask] = {ArgumentsPattern → {_, _ .. _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`ROIMask`

^

Symbol i

SetupDataStructure[dcmFolder] makes nii folders and generates nii files for a directory of dmc data where the data is structured per subject.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`Legacy`SetupDataStructure`

^

Symbol i

ShiftPar[B0file.dcm,DTIfile.dcm] imports the parameters from the dicom headend and calculates the needed values to preform B0 field map correction.

Needs a B0 dicom file and a diffusion dicom file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ShiftPar] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`ShiftPar

^

Options

Symbol i

BinaryType is an option for ExportVol and must be "Integer16" for an integer array and "Real32" for a Double array.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`BinaryType

^

Symbol i

BmatrixOut is a option for ImportBrukerData if True the bmatrix is given, if false the gradients and bvec are given.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`BmatrixOut

^

Symbol i

ConvertDcm is an option for GradRead.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`ConvertDcm

^

Symbol i

OutputMethod can be "val" or "plot".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`Legacy`OutputMethod

^

Symbol

RotateGradient is an option for ReadDicomDirDiff. If False it will also output the gradient direction as stored in the dicom header.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`Legacy`RotateGradient

^

Symbol

SampleStep is an option for AlignRespiract.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`Legacy`SampleStep

^

Symbol

ScaleCorrect is an option for ReadDicom, ReadDicomDiff, ReadDicomDir and ReadDicomDirDiff. The dicom image values are corrected for rescale slope, scale slope and rescale intercept.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`Legacy`ScaleCorrect

^

Symbol

SeedDensity is an option for FiberDensityMap. The seedpoint spacing in mm.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`Legacy`SeedDensity

^

LoggingTools

Functions

Symbol 

AddToLog[list] add the list to the log at level 1. All elements of the list are converted to strings and joined with spaces.

AddToLog[list, level] add the list to the log at level.

AddToLog[list, True] add the list to the log at level 1 with a timestamp.

AddToLog[list, True, level] specifies both the level and the timestamp.

AddToLog[list, level, True] specifies both the level and the timestamp.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[AddToLog] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `SaveLogFile → True`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`AddToLog`

^

Symbol 

CheckFile[{fol, name, tag}, stat, ver] checks a checkfile for status stat and software version.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CheckFile] = {ArgumentsPattern → {{_, _, _}, _, _}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`CheckFile`

^

Symbol 

DirectoryTree[dir] makes a directory tree from the dir.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DirectoryTree] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`DirectoryTree`

^

Symbol 

ExportDirectoryTree[fol] make a DirectoryTree for the folder and exports it there as FileTree.txt.

ExportDirectoryTree[fol,name] make a DirectoryTree for the folder and exports it there as name .txt.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportDirectoryTree] = {ArgumentsPattern -> { _ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`LoggingTools`ExportDirectoryTree

^

Symbol 

ExportLog[file] exports the log as a plain text to file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportLog] = {ArgumentsPattern -> { _ , _ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`LoggingTools`ExportLog

^

Symbol 

ImportLog[file] imports the log as a list of string from a plain text file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportLog] = {ArgumentsPattern -> { _ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`LoggingTools`ImportLog

^

Symbol 

MakeCheckFile[{fol, name, tag}, stat] makes a check json file in the fol with filename containing name and tag. Where stat is a list of rules that contain any information that needs to be put in the check file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeCheckFile] = {ArgumentsPattern -> {{ _ , _ , _ }, _ }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`LoggingTools`MakeCheckFile

^

Symbol i

PrintDirectoryTree[tree] prints the directory tree generated by `DirectoryTree`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PrintDirectoryTree] = {ArgumentsPattern → {…}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`PrintDirectoryTree`

^

Symbol i

ResetLog[] resets the log to {}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ResetLog] = {ArgumentsPattern → {}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`ResetLog`

^

Symbol i

SetLogFile[file] sets the log file name to file.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`SetLogFile`

^

Symbol i

ShowLog[] shows the log in a popup window.

ShowLog[False] shows the log in the notebook.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ShowLog] = {ArgumentsPattern → {…}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`LoggingTools`ShowLog`

^

Options

Symbol i

SaveLogFile is an option for AddToLog. If True the log is saved to the log file each time something is added.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`LoggingTools`SaveLogFile

^

MaskingTools

Functions

Symbol i

DilateMask[mask,size] if size > 0 the mask is dilated and if size < 0 the mask is eroded.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DilateMask] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`DilateMask

^

Symbol i

FitGradientMap[data, ord] fit of gradient trough all non zero values withing the data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FitGradientMap] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`FitGradientMap

^

Symbol i

GetCommonSegmentation[dat, seg, vox] For a list of multiple datasets dat the common segmentations from the list seg are determined. Output is a list of segmentations where for each region only the part present in all datasets is selected.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetCommonSegmentation] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`GetCommonSegmentation

^

Symbol i

GetSegmentationLabels[segmentation] gives a list of all labels in the segmentation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`GetSegmentationLabels

^

Symbol i

HomogenizeData[data, mask] tries to homogenize the data within the mask by removing intensity gradients.

Documentation [Local »](#)

Default Definitions SyntaxInformation[HomogenizeData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options FitOrder → 5

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`HomogenizeData

^

Symbol i

JoinSegmentations[seg, joinRules] joins the segmentations in seg according to the rules in joinRules.

JoinRules is a list of rules {{join, new}..} where join is a list of labels to be joined and new is the new label number.

For example {{1, 2}, 3} joins the labels 1 and 2 to label 3.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`JoinSegmentations

^

Symbol i

Mask[data] creates a mask by automatically finding a threshold.

Mask[data, min] creates a mask which selects only data above the min value.

Mask[data, {min, max}] creates a mask which selects data between the min and max value.

Documentation [Local »](#)

Default Definitions SyntaxInformation[Mask] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options ▸ MaskSmoothing → False... (6 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`Mask

^

Symbol i

MaskData[data, mask] applies a mask to data. mask can be 2D or 3D, data can be 2D, 3D or 4D.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MaskData] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`MaskData`

^

Symbol i

MaskSegmentation[seg, mask] applies a mask to a splitted segmentation seg from SplitSegmentations. The mask is 3D, seg is 4D.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MaskSegmentation] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`MaskSegmentation`

^

Symbol i

MaskVolume[mask] calculates the volume of the mask.

MaskVolume[{mask, ..}] calculates the Median volume of the List of masks.

MaskVolume[mask, vox] calculates the volume of the mask in cm^3 where vox is in mm.

MaskVolume[{mask, ..}, vox] calculates the Median volume of the List of masks in cm^3 where vox is in mm.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MaskVolume] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`MaskVolume`

^

Symbol



MergeSegmentations[masks, labels] generates an ITKsnap or slices3D compatible segmentation from individual masks and label numbers.

Output is a labeled segmentation.

MergeSegmentations[masks] does the same but automatically numbers the segmentations.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MergeSegmentations] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`MergeSegmentations`



Symbol



NormalizeData[data] normalizes the data to the mean signal of the data. For 4D data it normalizes to the first volume of the 4th dimension.

NormalizeData[data, mask] normalizes the data based on the mean signal only within the mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[NormalizeData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `NormalizeMethod → Set`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`NormalizeData`



Symbol



NormalizeMeanData[data] calculates the mean normalized data from a 4D dataset.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[NormalizeMeanData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `NormalizeMethod → Set`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`NormalizeMeanData`



Symbol



RemoveMaskOverlaps[mask] removes the overlaps between multiple masks. Mask is a 4D dataset with {z, masks, x, y}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RemoveMaskOverlaps] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`RemoveMaskOverlaps`



Symbol i

ReplaceSegmentations[seg, labs, new] replaces the labels labs from the segmentation seg for labels new. Both labs and new should be lists of integers of the same size. If seg contains more labels than given in labs these will be replaced by 0.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReplaceSegmentations] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`ReplaceSegmentations

^

Symbol i

RescaleSegmentation[data, dim] rescales segmentations to given dimensions.

RescaleSegmentation[data, {vox1, vox2}] rescales segmentations from voxelsize vox1 to voxelsize vox2.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RescaleSegmentation] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`RescaleSegmentation

^

Symbol i

SegmentationCrossSection[seg] calculates the maximum cross section of each label in the segmentation in voxels.

SegmentationCrossSection[seg, vox] calculates the maximum cross section of each label in the segmentation in cm^3 where vox is in mm.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SegmentationCrossSection] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options Method → Max

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`SegmentationCrossSection

^

Symbol i

SegmentationVolume[seg] calculates the volume of each label in the segmentation in voxels.

SegmentationVolume[seg, vox] calculates the volume of each label in the segmentation in cm^3 where vox is in mm.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SegmentationVolume] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`SegmentationVolume`

^

Symbol i

SegmentMask[mask, n] divides a mask in n segments along the slice direction, n must be an integer. The mask is divided in n equal parts where each parts has the same number of slices.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SegmentMask] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`SegmentMask`

^

Symbol i

SelectMaskComponents[mask] selects the largest connected component in the mask.

SelectMaskComponents[mask,n] selects the n largest connected components in the mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SelectMaskComponents] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`SelectMaskComponents`

^

Symbol i

SelectSegmentations[seg, labs] selects only the segmentations from seg with label number labs.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SelectSegmentations] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`MaskingTools`SelectSegmentations`

^

Symbol i

SmoothMask[mask] generates one clean masked volume from a noisy mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SmoothMask] = {ArgumentsPattern -> {_, OptionsPattern[]}}`

Options ▸ MaskComponents -> 1 ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`SmoothMask

^

Symbol i

SmoothSegmentation[segmentation] smooths segmentations and removes the overlaps between multiple segmentations.

SmoothSegmentation[segmentation, labs] only smooths the selected label number labs.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SmoothSegmentation] = {ArgumentsPattern -> {_, _, OptionsPattern[]}}`

Options ▸ MaskComponents -> 1 ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`SmoothSegmentation

^

Symbol i

SplitSegmentations[segmentation] splits a label mask from ITKsnap or slicer3D in separate masks and label numbers.

Output is masks and label numbers, {mask, labs}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SplitSegmentations] = {ArgumentsPattern -> {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`SplitSegmentations

^

Options

Symbol i

FitOrder is an option for HomogenizeData. It specifies the order of harmonics to be used for the homogenization.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`FitOrder

^

Symbol 

MaskClosing is an option for Mask, SmoothMask and SmoothSegmentation. The size of the holes in the mask that will be closed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskClosing

^

Symbol 

MaskComponents is an option for Mask, SmoothMask and SmoothSegmentation. Determines the amount of largest clusters used as mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskComponents

^

Symbol 

MaskDilation is an option for Mask, SmoothMask and SmoothSegmentation. If the value is greater than 0 it will dilate the mask, if the value is smaller than 0 it will erode the mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskDilation

^

Symbol 

MaskFiltKernel is an option for Mask, SmoothMask and SmoothSegmentation. How much the contours are smoothed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskFiltKernel

^

Symbol 

MaskSmoothing is an options for Mask, SmoothMask and SmoothSegmentation, if set to True it smooths the mask, by closing holse and smoothing the contours.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`MaskSmoothing

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol i

NormalizeMethod is an option for NormalizeData. Can be "Set" or "Volumes" wich normalizes to the first volume or normalizes each volume individually, respectively. If "Uniform" normalizes the histogram of the data to have a uniform distribution between 0 and 1 where 0 is treated as background of the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`NormalizeMethod

^

Symbol i

SmoothIterations is an option for Mask, SmoothMask and SmoothSegmentation and defines how often the smoothing is repeated.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MaskingTools`SmoothIterations

^

MuscleBidsTools

Functions

Symbol i

BidsDcmToNii[dir] converts the bids sourceFolder with dicom files to raw nii files based on the config file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[BidsDcmToNii] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {BidsIncludeSession → True, ProcessSubjects → All}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`BidsDcmToNii

^

Symbol i

CheckConfigLabels[dir, subs] checks for each sub in subs if the protocol names that are defined in the config file in dir are present.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`CheckConfigLabels

^

Symbol i

CheckDataDescription[description] checks the data description config file used in BidsDcmToNii, MuscleBidsConvert, MuscleBidsProcess, and MuscleBidsMerge.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CheckDataDescription] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`CheckDataDescription

^

Symbol i

GenerateBidsFileName[parts] generates a Bids file name from the Bids labels association which can be generated by PartitionBidsName.

GenerateBidsFileName[fol, parts] the same but with a custom root folder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GenerateBidsFileName] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GenerateBidsFileName

^

Symbol i

GenerateBidsFolderName[parts] generates a Bids folder name from the Bids labels association which can be generated by PartitionBidsFolderName.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GenerateBidsFolderName] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GenerateBidsFolderName

^

Symbol 

GenerateBidsName[parts] generates a Bids file name from the Bids labels association which can be generated by PartitionBidsName.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GenerateBidsName] = {ArgumentsPattern → {`_`}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GenerateBidsName

^

Symbol 

GetConfig[folder] imports a Muscle Bids config file from the given folder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GetConfig

^

Symbol 

GetJSONPosition[{{json..}, {{key, value}..}}] gets the position from a list of JSON association lists where keys have the given value.
GetJSONPosition[{{json..}, {{key, value}..}, sortKey] same but finally sorts the positions for the value of the sortKey.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GetJSONPosition

^

Symbol 

GetProtocolNames[config, subs] gets the list of protocol names in the specified subject folder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`GetProtocolNames

^

Symbol 

ImportJSON[file] imports a JSON file as rawJSON.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`ImportJSON

^

Symbol i

MergeConfig[assoc, replace] merges the replace association with the assoc association.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MergeConfig

^

Symbol i

MergeJSON[[json...]] merges a list of JSON association lists where duplicate keys with the same values are removed and duplicate keys with different values are merged.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MergeJSON

^

Symbol i

MuscleBidsAnalysis[dir] performs analysis on the Muscle-Bids named nii based on the config file in the bids sourceFolder dir. If a segmentation is present it is used to calculate the mean per segmentation.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MuscleBidsAnalysis] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}

Options {ProcessSubjects → All, VersionCheck → False, BidsOutputImages → All}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsAnalysis

^

Symbol i

MuscleBidsConvert[dir] converts all raw nii data in the to Muscle-Bids named nii based on the config file in the bids sourceFolder dir.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MuscleBidsConvert] = {ArgumentsPattern → {_, __, OptionsPattern[]}}

Options {DeleteAfterConversion → True, ProcessSubjects → All}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsConvert

^

Symbol i

MuscleBidsMerge[dir] merges multiple stack data for all Muscle-Bids named nii based on the config file in the bids sourceFolder dir.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MuscleBidsMerge] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}`

Options {ProcessSubjects → All, VersionCheck → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsMerge

^

Symbol i

MuscleBidsProcess[dir] processes all Muscle-Bids named nii based on the config file in the bids sourceFolder dir.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MuscleBidsProcess] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}`

Options {ProcessSubjects → All, VersionCheck → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsProcess

^

Symbol i

MuscleBidsSegment[dir] segments the data of Muscle-Bids named nii based on the config file in the bids sourceFolder dir. The segmentations are generated by the function SegmentData.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MuscleBidsSegment] = {ArgumentsPattern → {_, __, __, OptionsPattern[]}}`

Options {ProcessSubjects → All, VersionCheck → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsSegment

^

Symbol i

MuscleBidsTractography[dir] performs tractography on the Muscle–Bids named nii based on the config file in the bids sourceFolder dir. If a segmentation is present it is used as a mask for per muscle segmentation.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[MuscleBidsTractography] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {ProcessSubjects → All, VersionCheck → False, BidsTractographyMethod → Full}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`MuscleBidsTractography

^

Symbol i

PartitionBidsFolderName[fol] partitions the Bids folder and file name. It returns the bids root folder and the label parts using PartitionBidsName.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[PartitionBidsFolderName] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`PartitionBidsFolderName

^

Symbol i

PartitionBidsName[name] converts a Bids name to the Bids labels as an association, i.e. {"sub", "ses", "stk", "rep", "type", "suf"}.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[PartitionBidsName] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`PartitionBidsName

^

Symbol i

SelectBidsFolders[fol, tag] selects all folders in the fol with the name tag.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[SelectBidsFolders] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`SelectBidsFolders

^

Symbol 

SelectBidsSessions[fol] selects all sessions in the bids subject folder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SelectBidsSessions] = {ArgumentsPattern → {_{..}}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`SelectBidsSessions

^

Symbol 

SelectBidsSubjects[fol] selects all subjects in the bids folder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SelectBidsSubjects] = {ArgumentsPattern → {_{..}}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`SelectBidsSubjects

^

Symbol 

SelectSubjects[dir] selects the subjects in the given data directory which has a config file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SelectSubjects] = {ArgumentsPattern → {_{..}}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`SelectSubjects

^

Symbol 

ViewConfig[config] shows a config file for Muscle Bids processing.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ViewConfig] = {ArgumentsPattern → {_{..}}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`ViewConfig

^

Symbol i

ViewProtocolNames[folder] shows the protocol names in the radData folder specified by the config file with the series number and protocol name from the json files in the raw folders for each subject and session.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ViewProtocolNames] = {ArgumentsPattern → {_, OptionsPattern[]}}

Options ProcessSubjects → All

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`ViewProtocolNames

^

Symbol i

QMRITools`MuscleBidsTools`\$debugBids

Documentation [Local »](#)

Attributes {ReadProtected}

Full Name QMRITools`MuscleBidsTools`\$debugBids

^

Options

Symbol i

BidsIncludeSession is an option for BidsDcmToNii. If True session folders will be used in output even if they are not specified.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`BidsIncludeSession

^

Symbol i

BidsOutputImages is an option for MuscleBidsAnalysis. If set True the output images are saved in the output folder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`BidsOutputImages

^

Symbol 

BidsTractographyMethod is an option for MuscleBidsTractography and can be "Full", "Tractography" or "Segmentation". With "Tractography" the tractography is performed without segmentation. With "Segmentation" only the segmentation is performed without tractography. With Full both are performed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`BidsTractographyMethod

^

Symbol 

DeleteAfterConversion is an option for MuscleBidsConvert. If set True all files that have been converted will be deleted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`DeleteAfterConversion

^

Symbol 

ProcessSubjects is an option for Bids functions. Can be a list of bids subject names else it is All.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`ProcessSubjects

^

Symbol 

VersionCheck is an option for all Bids functions. If set True data processed with an old version is reprocessed.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`MuscleBidsTools`VersionCheck

^

NeuralNetworkTools

Functions

Symbol i

ActivationLayer[type] is a wrapper around ElementwiseLayer needed in MakeNode.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`ActivationLayer

^

Symbol i

AddLossLayer[net] adds loss layers to a NetGraph. The DiceLossLayer, JaccardLossLayer, TverskyLossLayer, MSDLossLayer, TopK, and CELossLayer are added.

Documentation [Local »](#)

Default Definitions SyntaxInformation[AddLossLayer] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`AddLossLayer

^

Symbol i

AnalyzeNetworkFeatures[net, data] gives overview of the information density of the network features by analyzing them with SVD.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`AnalyzeNetworkFeatures

^

Symbol i

CELossFunction[] is a generalization for pixel based loss layers like CELossLayer, FocalLossLayer and TopKLossLayer.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CELossFunction] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`CELossFunction

^

Symbol



CELossLayer[] represents a net layer that computes the Focal loss by comparing input class probability vectors with the target class vector. Identical to `CrossEntropyLossLayer["Probability"]`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CELossLayer] = {ArgumentsPattern → {}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`CELossLayer`



Symbol



ChangeNetDimensions[netIn] changes input channels, output classes, the input patch dimension of the input network netIn.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ChangeNetDimensions] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{Dimensions → None, Channels → None, Classes → None}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`ChangeNetDimensions`



Symbol



ClassDecoder[probability] decodes a probability vector of 1 and 0 into Integers of 0 to the value of the last dimension of probability minus one. **ClassDecoder[probability, nClass]** decodes a probability vector of 1 and 0 into Integers of 0 to nClass - 1.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ClassDecoder] = {ArgumentsPattern → {_, ...}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`ClassDecoder`



Symbol



ClassEncoder[label] encodes Integer label data of 0 to max value of label into a nClass + 1 vector of 1 and 0 as the last dimension. **ClassEncoder[label, nClass]** encodes Integer label data of 0 to nClass into a nClass + 1 vector of 1 and 0 as the last dimension.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ClassEncoder] = {ArgumentsPattern → {_, ...}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`ClassEncoder`



Symbol i

DiceLossLayer[] represents a net layer that computes the Dice loss by comparing input class probability vectors with the target class vector.

DiceLossLayer[n] does the same but n defines the power of the denominator, with n=2 the squared dice score, is calculated.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DiceLossLayer] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NeuralNetworkTools`DiceLossLayer`

^

Symbol i

FocalLossLayer[] represents a net layer that computes the Focal loss by comparing input class probability vectors with the target class vector.

FocalLossLayer[g] does the same but uses g as the tunable focusing parameter gamma which needs to be larger than one.

FocalLossLayer[g, a] does the same but uses a as the balancing factor alpha.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FocalLossLayer] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NeuralNetworkTools`FocalLossLayer`

^

Symbol i

GetNetNodes[net] returns a list of all the nodes in the network net.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetNetNodes] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NeuralNetworkTools`GetNetNodes`

^

Symbol i

JaccardLossLayer[] represents a net layer that computes the Jaccard loss by comparing input class probability vectors with the target class vector.

JaccardLossLayer[n] does the same but n defines the power of the denominator, with n=2 the squared Jaccard score is calculated.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[JaccardLossLayer] = {ArgumentsPattern → {..., ...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NeuralNetworkTools`JaccardLossLayer`

^

Symbol



MakeClassifyImage[data] makes a image of the input data. The data is automatically cropped to remove the background and normalized.

If the input data is 3D a list of images is returned.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeClassifyImage] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `ImageSize → {128, 128}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`MakeClassifyImage`



Symbol



MakeClassifyNetwork[classes] makes a classify network with three convolution layers and 3 fully connected layers.

The input classes should be a list of strings. The input image dimensions should not be smaller than 64x64.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeClassifyNetwork] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `ImageSize → {128, 128}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`MakeClassifyNetwork`



Symbol



MakeNode[scale, conn, blockConfig] makes a node for a UNET.

The input scale defines the input and output scaling, is either an integer or a vector of length dim.

The input conn defines the connections, is a list of two integer values defining the number of input and output ports.

The blockConfig is defined as `{{blockType, settings}, {features,..}, {act, dim}}`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeNode] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options `{DropoutRate → 0, RescaleMethod → Conv, CatenateMethod → Cat}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NeuralNetworkTools`MakeNode`



Symbol i

MakeUnet[nClasses, dimIn] Generates a UNET with one channel as input and nClasses as output.

MakeUnet[nChannels, nClasses, dimIn] Generates a UNET with nChannels as input and nClasses as output.

he number of parameter of the first convolution layer can be set with dep. The data dimensions can be 2D or 3D and each of the dimensions should be 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 224, 240 or 256. However dimensions can be different based on the network depth and the block type. The implemented block types are "Conv", "UNet", "ResNet", "DenseNet", "Inception", or "U2Net".

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeUnet] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options » NetworkArchitecture → UNet ... (12 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`MakeUnet

^

Symbol i

NetDimensions[net] extracts the input channels, output classes, the input patch dimension, and the number of input filters.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NetDimensions

^

Symbol i

NetSummary[net] gives a short summary of the convolution kernels and array elements in the network.

NetSummary[net, what] does the same but what can be "Full" which also includes net and node images or "Mem" which only reports the memory.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NetSummary

^

Symbol i

OverlapLossFunction[] is a generalization for overlap loss layers like TverskyLossLayer, JaccardLossLayer and DiceLossLayer.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[OverlapLossFunction] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`OverlapLossFunction

^

Symbol 

TopKLossLayer[net] represents a net layer that computes the topK 10% loss.

TopKLossLayer[net, k] does the same but k defines the topK between 0 and 1.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TopKLossLayer] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`TopKLossLayer

^

Symbol 

TverskyLossLayer[] represents a net layer that computes the Tversky loss by comparing input class probability vectors with the target class vector.

TverskyLossLayer[b] does the same but b defines the Tversky beta factor. With beta = 0.5 its is the Dice coefficient. Here alpha + beta = 1.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TverskyLossLayer] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`TverskyLossLayer

^

Options

Missing[UnknownSymbol, Channels]

Missing[UnknownSymbol, Classes]

Symbol 

Dimensions[*expr*] gives a list of the dimensions of *expr*.

Dimensions[*expr*, *n*] gives a list of the dimensions of *expr* down to level *n*.

Documentation [Local »](#) | [Web »](#)

Options AllowedHeads → Automatic

Attributes {Protected}

Full Name System`Dimensions

^

Symbol i

ActivationType is an option for MakeUnet. It specifies which activation layer is used in the network. It can be "LeakyRELU" or any type allowed by a "name" definition in ElementwiseLayer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`ActivationType

^

Symbol i

BlockType is an option for MakeUnet. It specifies the type of block used in the network. It can be "Conv", "UNet", "ResNet", "DenseNet", "Inception", or "U2Net".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`BlockType

^

Symbol i

CatenateMethod is an option for MakeUnet. It specifies how the network catenates the skip and upscale channel. It can be "Tot", or "Cat".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`CatenateMethod

^

Symbol i

DownsampleSchedule is an option for MakeUnet. It defines how the data is downsampled for each of the deeper layers of the Unet. By default is is a factor two for each layer. A custom schedule for a 5 layer 3D Unet could be {{2,2,2},{1,2,2},{2,2,2},{1,2,2}, 1}. The deepest layer is always downsampled by 1 and therefore not needed to be specified.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`DownsampleSchedule

^

Symbol i

DropoutRate is an option for MakeUnet. It specifies how much dropout is used after each block. It is a value between 0 and 1, default is .2.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`DropoutRate

^

Symbol i

FeatureSchedule is an option for MakeUnet. It defines how the number of features is up-sampled for each of the deeper layers of the Unet. By default it increases the number of features by a factor 2 each layer, i.e. {1, 2, 4, 8, 16}.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`NeuralNetworkTools`FeatureSchedule

^

Symbol i

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`ImageSize

^

Symbol i

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol i

NetworkArchitecture is an option for MakeUnet. It defines the architecture of the network. It can be "UNet", "UNet+", or "UNet++". For "UNet+" or "UNet++" it can also be {arch, i} where i specifies how many of the top layers are connected to the mapping layer.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`NeuralNetworkTools`NetworkArchitecture

^

Symbol i

NetworkDepth is an option for MakeUnet. It specifies how deep the UNET will be.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`NeuralNetworkTools`NetworkDepth

^

Symbol i

NormalizationType is an option for MakeUnet. It specifies which normalization layer is used in the network. It can be "Batch" or "Instance" for either BatchNormalization or NormalizationLayer which is instance normalization.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NormalizationType

^

Symbol i

RescaleMethod is an option for MakeUnet. It specifies how the network rescales. It can be "Conv" or "Pool".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`RescaleMethod

^

Symbol i

SettingSchedule is an option for MakeUnet. It defines the settings for the Unet blocks. If one setting is given it applied to all layers. If a list of settings is given the settings can be different per layer. The following settings are the default settings.

"Unet": convolution block repetitions, 2, "ResNet" -> convolution block repetitions, 2, "DenseNet" -> {dense depth, block repetitions}, {4,2}, "Inception" -> {inception width, block repetitions}, {4,2}, "U2Net" -> {Unet depth, downscale}, {5, True}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`SettingSchedule

^

NiftiTools

Functions

Symbol i

CompressNiiFiles[] prompts for a folder. It then compresses all nii files to .nii.gz files in the selected folder.

CompressNiiFiles[folder] compresses all nii files to .nii.gz files in folder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CompressNiiFiles] = {ArgumentsPattern -> {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`CompressNiiFiles

^

Symbol 

CorrectNiiOrientation[data,hdr] corrects the data orientation based on the nii header.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CorrectNiiOrientation] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`CorrectNiiOrientation

^

Symbol 

DcmToNii[] converts a dicom folder to nii, you will be prompted for the location of the folders.

DcmToNii[{"input","output"}] converts the "input" dicom folder to nii files which are place in the "output" folder.

For this function to work the dcm2niix.exe file should be present in the QMRITools application folder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DcmToNii] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ▶ CompressNii → True ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`DcmToNii

^

Symbol 

ExportBmat[bmat] exports the diffusion bmatrix to a bmat file in *.txt format.

ExportBmat[bmat, "file"] exports the diffusion bmatrix to "file" in *.txt format.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ExportBmat] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExportBmat

^

Symbol i

ExportBval[bvals] exports the diffusion bvalues to a *.bval format.

ExportBval[bvals, "file"] exports the diffusion bvalues to "file" *.bval format.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportBval] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExportBval

^

Symbol i

ExportBvalvec[{bvals, grad}] exports the diffusion values and gradients to a *.bval and *.bvec format.

ExportBvalvec[{bvals, grad}, "file"] exports the diffusion values and gradients to "file" to a *.bval and *.bvec format.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportBvalvec] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {FlipBvec → False, PositiveZ → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExportBvalvec

^

Symbol i

ExportBvec[grad] exports the diffusion gradients to a *.bvec format.

ExportBvec[grad, "file"] exports the diffusion gradients to "file" *.bvec format.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportBvec] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {FlipBvec → False, PositiveZ → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExportBvec

^

Symbol i

ExportNii[data, vox] exports the nii file and will prompt for a file name.

ExportNii[data, vox, "file"] exports the nii file to the location "file".

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportNii] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▸ NiiDataType → Automatic ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExportNii

^

Symbol i

ExtractNiiFiles[] prompts for a folder. It then extracts all nii.gz files to .nii files in the selected folder.

ExtractNiiFiles[folder] extracts all nii.gz files to .nii files in folder.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExtractNiiFiles] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ExtractNiiFiles

^

Symbol i

GetNiiOrientation[hdr] get the sform and qform orientations from a nii header.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetNiiOrientation] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`GetNiiOrientation

^

Symbol i

ImportBmat[] will prompt to select the *.txt file containing the bmatrix.

ImportBmat[*.txt] imports the given *.txt file containing the bmatrix.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportBmat] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ImportBmat

^

Symbol i

ImportBval[] will prompt to select the *.bval file.

ImportBval[*.bval] imports the given *.bval file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportBval] = {ArgumentsPattern → {...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NiftiTools`ImportBval`

^

Symbol i

ImportBvalvec[] will prompt to select the *.bval and *.bvec files.

ImportBvalvec[file] if file is either a *.bval or *.bvec it will automatically import the *.bval and *.bvec files.

ImportBvalvec[*.bvec,*.bval] imports the given *.bval and *.bvec files.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportBvalvec] = {ArgumentsPattern → {..., ...}, OptionsPattern[]}`

Options {FlipBvec → False, PositiveZ → False}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NiftiTools`ImportBvalvec`

^

Symbol i

ImportBvec[] will prompt to select the *.bvec file.

ImportBvec[*.bvec] imports the given *.bvec file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportBvec] = {ArgumentsPattern → {..., ...}, OptionsPattern[]}`

Options {FlipBvec → False, PositiveZ → False}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NiftiTools`ImportBvec`

^

Symbol i

ImportExploreDTItens["file"] imports the *.nii export for the tensor from explore DTI.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`NiftiTools`ImportExploreDTItens`

^

Symbol i

ImportNii[] prompts to select the nii file to import.

ImportNii["file"] imports the nii file.

The default output is {data, vox}, however using NiiMethod various outputs can be given.

The Nii import is also supported using the native Import function from Mathematica.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportNii] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{NiiMethod → default, NiiScaling → False}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NiftiTools`ImportNii`

^

Symbol i

ImportNiiDiff[] will prompt for the *.nii, *.bvec and *.bval file to import.

ImportNiiDiff[*nii] will import the *.nii file and automatically also imports the *.bvec and *.bval if they have the same name.

ImportNiiDiff[*nii,*.bvec,*.bval] will import the given files.

The output will be {data,grad,bvec,vox}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportNiiDiff] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options `{RotateGradients → False, FlipBvec → True}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NiftiTools`ImportNiiDiff`

^

Symbol i

ImportNiiDix["file"] imports the dixon nii file which should contain all possible outputs given by the scanner and corrects them accordingly.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`NiftiTools`ImportNiiDix`

^

Symbol i

ImportNiiT1["file"] imports the t1 file which should contain the echos and the T1map calculated by the scanner and corrects them accordingly.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ImportNiiT1

^

Symbol i

ImportNiiT2["file"] imports the t2 file which should contain the echos and the T2map calculated by the scanner and corrects them accordingly.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`ImportNiiT2

^

Symbol i

MakeNiiOrientationQ[rot] makes the q vector from rotation matrix rot.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`MakeNiiOrientationQ

^

Symbol i

MakeNiiOrientationS[off, vox] maxes the srow values for nii header assuming not rot and Q.

MakeNiiOrientationS[off, vox, rot] maxes the srow values for nii header using rotation rot.

MakeNiiOrientationS[off, vox, rot, Q] maxes the srow values for nii header using rotation rot and skew Q.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`MakeNiiOrientationS

^

Options

Symbol i

CompressNii is an option for DcmToNii and ExportNii. If set True .nii.gz files will be created.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`CompressNii

^

Symbol 

DeleteOutputFolder is an option of DcmToNii. If the output folder already exists it will be deleted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`DeleteOutputFolder

^

Symbol 

FlipBvec is an option for ImportBvalvec.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`FlipBvec

^

Symbol 

MergeEchos is an option for DcmToNii. If set True the echos are merged into one file.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`MergeEchos

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol i

NiiDataType is an option of ExportNii. The number type of Nii file can be "Integer", "Real", "Complex", or "Automatic".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiDataType

^

Symbol i

NiiLegacy is an option for ExportNii, if set True default orientations are set instead of unknown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiLegacy

^

Symbol i

NiiMethod is an option for ImportNii. Values can be "data", "dataTR", "header", "scaling", "headerMat", "rotation", "all".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiMethod

^

Symbol i

NiiOffset is an option of ExportNii. Is {xoff, yoff, zoff}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiOffset

^

Symbol i

NiiScaling is an option for ImportNii. It scales the nii values with scale slope and offset for quantitative data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiScaling

^

Symbol 

NiiSliceCode is an option for Export nii. With this you can set the slice code of the nii file.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`NiiSliceCode

^

Symbol 

PositiveZ is an options for GradientPlot. If True all Gradients are displayed with a positive z direction.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PositiveZ

^

Symbol 

RotateGradients is an option for ImportNiiDiff.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`RotateGradients

^

Symbol 

UseSubfolders is an option for DcmToNii. If set True the nii conversion is done for each folder in the selected input folder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`UseSubfolders

^

Symbol 

UseVersion is an option for DcmToNii. For windows it allows to switch between different versions of dcm2niix.exe.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NiftiTools`UseVersion

^

PlottingTools

Functions

Symbol i

ColorFAPlot[tenor] create a color coded FA map from the tensor for l1, l2 and l3.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ColorFAPlot] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`ColorFAPlot`

^

Symbol i

GenerateRotationFrames[3Dgraphics, nFrames] generates nFrames of the 3D graphics rotating around the z-axis.

Documentation [Local »](#)

Options `ImageSize → 500`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`GenerateRotationFrames`

^

Symbol i

GetSliceData[data, offsets] gets the slices from the data defined by offsets which are obtained by `GetSlicePosisions`.

GetSliceData[data, offsets, vox] gets the slices from the data defined by offsets which are obtained by `GetSlicePosisions` in mm.

The offsets can also be provided manually which is `{{AX,...},{COR,...},{SAG,...}}`.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetSliceData] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`GetSliceData`

^

Symbol i

GetSlicePositions[data] finds the position of slices with the maximal signal in voxel index.

GetSlicePositions[data, vox] find the position of slices with the maximal signal in mm.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetSlicePositions] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {`MakeCheckPlot → False`, `DropSlices → {1, 1, 1}`, `PeakNumber → {1, 1, 2}`}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`GetSlicePositions`

^

Symbol i

GradientPlot[bvec, bval] plots the given bvec with position of the gradients scaled according to the bval.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GradientPlot] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options ▸ `PlotSpace → bspace ... (4 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`PlottingTools`GradientPlot`

^

Symbol i

LegendImage[colf, w, lab] generates a legend image for the color function colf with a width of w and labels lab.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[LegendImage] = {ArgumentsPattern → {_, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`PlottingTools`LegendImage`

^

Symbol i

Link3DGraphic[3Dgraphics] creates a 3D graphic with a linked view options between all 3D graphics that have been linked.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`PlottingTools`Link3DGraphic`

^

Symbol i

ListSpherePlot[points] plots 3D points as spheres.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ListSpherePlot] = {ArgumentsPattern → {_, OptionsPattern[]}`

Options `{SphereSize → 2, SphereColor → Automatic}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`PlottingTools`ListSpherePlot`

^

Symbol i

LoessPlot[data] plots the data with a loess fit.

Documentation [Local](#) »

Options ▶ Bandwidth → Scaled[0.5] ... (10 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`LoessPlot

^

Symbol i

MakeSlicelImages[imgData] generates images from the imgData which is obtained form GetSliceData.

MakeSlicelImages[imgData, vox] generates images from the imgData which is obtained form GetSliceData, vox is used for the correct aspect ratio of the images.

MakeSlicelImages[imgData, {labData, labels}] generates images from the imgData which is obtained form GetSliceData with an overlay of the segmentations in labData, which can also be obtained using GetSliceData on the segmentations. labels should be the label numbers used in the original segmentation (to allow correct scaling between slices).

MakeSlicelImages[imgData, {labData, labels},vox] generates images from the imgData which is obtained form GetSliceData with an overlay of the segmentations in labData, which can also be obtained using GetSliceData on the segmentations, vox is used for the correct aspect ratio of the images.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[MakeSlicelImages] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▶ PlotRange → Automatic ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`MakeSlicelImages

^

Symbol i

PlotContour[data, vox] creates a contour of the data.

The data can be 3D or 4D, when its 4D each contour is given a random color.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[PlotContour] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options ▶ ContourColor → ■ ... (8 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotContour

^

Symbol i

PlotCorrection[w] plots deformation vectors w {w1,w2..} generated by Registration2D and Registration3D for multiple datasets or registration steps.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotCorrection] = {ArgumentsPattern → {..}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotCorrection

^

Symbol i

PlotData[data] plots the data.

PlotData[data, vox] plots the data and for 3D and 4D data assumes the voxelsize vox (z,x,y).

PlotData[data1, data2] plots data1 and data2.

PlotData[data1, data2, vox] plots data1 and data2 and for 3D and 4D data assumes the voxelsize vox (z,x,y).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotData] = {ArgumentsPattern → {.., .., .., OptionsPattern[]}}`

Options {PlotRange → Auto, ColorFunction → BlackToWhite}

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotData

^

Symbol i

PlotData3D[data,vox] is a 3D dataviewer, data is the 3D dataset and voxsize the size of the voxels in mm (z,x,y).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotData3D] = {ArgumentsPattern → {.., ..}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotData3D

^

Symbol i

PlotDefGrid[data, phasemap, shiftpar] plots the dataset on the background with on top the non deformed and the deformed grid, or arrows or lines.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotDefGrid

^

Symbol i

PlotDuty[[grad, bval, ord], mode] plot the gradient dutycycle.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotDuty] = {ArgumentsPattern → {{_, _, _}, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotDuty

^

Symbol i

PlotIVIM[vals, data, bvals] plots the results of the IVIM fits from IVIMCalc or BayesianIVIMFit2 or Baye.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotIVIM] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▶ Method → ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotIVIM

^

Symbol i

PlotMoments[[G(t),...], te, t] plots the moments generated by CalculateMoments.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotMoments] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotMoments

^

Symbol i

PlotSegmentations[seg, vox] crates a contour of each of the segmentations in seg. Uses PlotContour.

PlotSegmentations[seg, bone, vox] same but also generates an extra contour with the bones in gray.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSegmentations] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▶ ImageSize → 400 ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotSegmentations

^

Symbol i

PlotSequence[seq,var] where seq is the output from GradSeq.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSequence] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`PlotSequence`

^

Symbol i

ShowLink[] shows the linked view options between all 3D graphics that have been linked.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`ShowLink`

^

Symbol i

QMRITools`PlottingTools`\$plotOptions

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`$plotOptions`

^

Options

Symbol i

Axes is an option for graphics functions that specifies whether axes should be drawn.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name `System`Axes`

^

Symbol i

Bandwidth is an option for LoessPlot. It specifies the bandwidth of the loess fit. Its a number between 0 and 1 and specifies which % of the data to include for the local regression.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`Bandwidth`

^

Symbol



ClippingStyle is an option for plotting functions that specifies the style of what should be drawn when curves or surfaces would extend beyond the plot range.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ClippingStyle



Symbol



ColorFunction is an option for graphics functions that specifies a function to apply to determine colors of elements.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ColorFunction



Symbol



ConfidenceLevel is an option for LinearModelFit and other fitting functions that specifies the level to use in various confidence and prediction intervals and bands.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`ConfidenceLevel



Symbol



ContourColor is an option for PlotContour. It specifies the contour color and can be a single color or an array with the same dimensions as the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourColor



Symbol



ContourColorRange is an option for PlotContour. If the ContourColor is an array this values gives the plot range.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourColorRange



Symbol i

ContourOpacity is an option for PlotContour and PlotSegmentations. It specifies the opacity of the contour.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourOpacity

^

Symbol i

ContourResolution is an option for PlotContour. It defines the mesh resolution used, can be a single number or a list of 3 numbers.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourResolution

^

Symbol i

ContourScaling is an option for PlotCountour. The value can be "World" or "Voxel", if the value is "Wold" the segmentation is in mm else in voxel coordinates.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourScaling

^

Symbol i

ContourSmoothRadius is and option for PlotContour. It defines the smoothing radius with an integer, None or 0 does not smooth.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourSmoothRadius

^

Symbol i

ContourSpecularity is an option for PlotContour. It specifies the specularity of the contour.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ContourSpecularity

^

Symbol 

DropSlices is an option for GetSlicePositions and specifies how many slices from the beginning and and should be ignored.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`PlottingTools`DropSlices

^

Symbol 

FitKernel is an option for LoessPlot. It specifies the kernel used for the loess fit. Can be "Triangular" or "Gaussian".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`PlottingTools`FitKernel

^

Symbol 

FitOrder is an option for HomogenizeData. It specifies the order of harmonics to be used for the homogenization.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`MaskingTools`FitOrder

^

Symbol 

Frame is an option for Graphics, Grid, and other constructs that specifies whether to include a frame.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`Frame

^

Symbol 

FrameStyle is an option for Graphics, Grid, and other constructs that specifies the style in which to draw frames.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`FrameStyle

^

Symbol 

ImageLegend is an option for MakeSlicelImages, if set true a barlegend is added to the image.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ImageLegend

^

Symbol 

ImageOrientation is an option for MakeSlicelImages. Can be Automatic, "Vertical" or "Horizontal".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`ImageOrientation

^

Symbol 

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol 

LabelStyle is an option for formatting and related constructs that specifies the style to use in displaying their label-like elements.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`LabelStyle

^

Symbol 

MakeCheckPlot is an option for GetSlicePositions and if set true gives a plot of the slices locations.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`MakeCheckPlot

^

Symbol 

MaskOpacity is an option for MakeSlicelImages. It specifies the opacity of the mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`MaskOpacity

^

Symbol 

MaxPlotPoints is an option for plotting functions like ArrayPlot and ListPlot3D that specifies the maximum number of points that will explicitly be included in the output.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`MaxPlotPoints

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

NormalizeIVIM is an option for IVIMplot. If True the signal at b=0 is 1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`NormalizeIVIM

^

Symbol 

PeakNumber is an option of GetSlicePositions and specifies how many slices per direction need to be found.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PeakNumber

^

Symbol i

PerformanceGoal is an option for various algorithmic and presentational functions that specifies what aspect of performance to try to optimize with Automatic settings for options.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PerformanceGoal

^

Symbol i

PlotColor is an option for GradientPlot can be any color or gradient color name.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotColor

^

Symbol i

PlotHighlighting is an option to Plot, ListPlot and related visualization functions that specifies how points and curves should be highlighted.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotHighlighting

^

Symbol i

PlotRange is an option for graphics functions that specifies what range of coordinates to include in a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotRange

^

Symbol i

PlotSpace is an option for GradientPlot can be "bspace" or "qspace".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PlotSpace

^

Symbol i

PlotStyle is an option for plotting and related functions that specifies styles in which objects are to be drawn.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PlotStyle

^

Symbol i

PositiveZ is an options for GradientPlot. If True all Gradients are displayed with a positive z direction.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PositiveZ

^

Symbol i

PredictionInterval is an option for LoessPlot. It specifies the prediction interval of the loess fit. If True it uses the prediction confidence bands if False it uses the standard error of the local mean.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`PredictionInterval

^

Symbol i

RandomizeColor is an option for PlotSegmentations. If True the colors are randomly assigned to each segmentation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`RandomizeColor

^

Symbol i

SphereColor ListSpherePlor. Default value is Automatic, If a color is given this color will be used for all spheres.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`SphereColor

^

Symbol i

SphereSize is an option for GradientPlot and ListSpherePlot. Sets the size of the spheres that represent the gradients.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`PlottingTools`SphereSize

^

ProcessingTools

Functions

Symbol i

B1MapCalc[data, TR, alpha] calculates the B1 map from a dual TR {tr1, tr2} acquisition (AFI) using magnitude data with reference angle alpha. data has dimensions {z, {tr1, tr2}, x, y}.

B1MapCalc[dataTr1, dataTr2, TR, alpha] where dataTr1 and dataTr2 can have any dimensions.

The Output can be "Map", "MagPhase", or "Complex"

B1MapCalc[] is based on DOI: 10.1002/mrm.21120.

Documentation [Local »](#)

Default Definitions SyntaxInformation[B1MapCalc] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options ▶ B1Output → Map ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1MapCalc

^

Symbol i

B1Shimming[b10, b190, mask] finds the optimal shim values to shim to 100% b1. Assumes B1Scaling "Relative".

B1Shimming[b10, b190, mask, target] finds the optimal shim values to shim to target, which can be a number or a map.

Documentation [Local »](#)

Default Definitions SyntaxInformation[B1Shimming] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}

Options ▶ B1ShimMethod → All ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1Shimming

^

Symbol



CombineB1[b10,b190,{f1,f2,ang}] combines the complex b1 maps with relative amplitudes f1 and f2 using phase angle ang.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CombineB1] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `B1Scaling → Relative`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`CombineB1`



Symbol



CorrectJoinSetMotion[[{dat1, dat2, ...}, vox, over] motion corrects multiple sets with overlap. Over is the number of slices overlap between stes. A Translation registration is performed.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CorrectJoinSetMotion] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `{JoinSetSplit → True, PadOverlap → 2, MonitorCalc → True}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`CorrectJoinSetMotion`



Symbol



DataTransformation[data, vox, w] transforms a 3D dataset according to the affine transformation vector w.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DataTransformation] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `{InterpolationOrder → 1, PadOutputDimensions → False}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`DataTransformation`



Symbol



DatTot[[{data1, data2, ..}, name, vox] calculates the parameter table containing the volume, mean, std and 95 CI for each of the diffusion parameters.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DatTot] = {ArgumentsPattern → {_, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`DatTot`



Symbol i

DatTotXLS[[data1, data2, ..], name, vox] is the same as DatTot, but gives the parameters as strings for easy export to excel.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DatTotXLS] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`DatTotXLS

^

Symbol i

ErrorPlot[data, xdata] plots a error plot of the data where the first dim of the data is the xrange which matches the xdata list.

ErrorPlot[data, xdata, range] similar with a given y range.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ErrorPlot] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options ▶ ColorValue → {█, █} ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`ErrorPlot

^

Symbol i

FindOutliers[data] finds the outliers of a list of data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FindOutliers] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options ▶ OutlierMethod → IQR ... (6 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`FindOutliers

^

Symbol i

FitData[data, range] converts the data into 100 bins within the +/- range around the mean. Function is used in ParameterFit.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FitData] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`FitData

^

Symbol 

GetMaskData[data, mask] returns the data selected by the mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetMaskData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {GetMaskOutput → All, GetMaskOnly → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`GetMaskData

^

Symbol 

GetMaskMeans[dat, mask] calculates the mean, std, 5,50 and 95% CI form the given data for each of the given masks.

Mask can be generated by SegmentTracts.

GetMaskMeans[dat, mask, name] where name is a string that is added to the header.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetMaskMeans] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options MeanMethod → SkewNormalDist

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`GetMaskMeans

^

Symbol 

GetTractMeans[dat, tracts, vox] calculates the mean, std, 5,50 and 95% CI form the given data for each of the given tracts.

Tracts can be by SplitSegmentations. name is a string that is added to the header.

GetTractMeans[dat, tracts, vox, name] where name is a string that is added to the header.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetTractMeans] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options {MeanMethod → SkewNormalDist, InterpolationOrder → 0}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`GetTractMeans

^

Symbol i

Hist[data, range] plots a probability density histogram of the data from xmin to xmax with a fitted (skew)normal distribution. Uses ParameterFit.

Hist[data, range, label] plots a probability density histogram of the data from xmin to xmax with a fitted (skew)normal distribution and label as x-axis label.

Hist[{data1.., data2,..}, {range1, range2,..}] plots a probability density histogram of the data from xmin to xmax with a fitted (skew)normal distribution. Uses ParameterFit.

Hist[{data1, data2,..}, {range1, range2,..}, {label1, label2,..}] plots a probability density histogram of the data from xmin to xmax with a fitted (skew)normal distribution and label as x-axis label.

Documentation [Local](#) »

Default Definitions SyntaxInformation[Hist] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options > ColorValue → {{█, }, █, █, █, █} ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`Hist

^

Symbol i

Hist2[pars, range] plots a probability density histogram of the data over range with two fitted (skew)normal distribution. Uses ParameterFit2.

Hist2[pars, range, label] plots a probability density histogram of the data over range with two fitted (skew)normal distribution. Uses ParameterFit2.

Documentation [Local](#) »

Default Definitions SyntaxInformation[Hist2] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options Scaling → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`Hist2

^

Symbol i

InvertDataset[data] inverts the data along the x y and z axes. In other words it is rotated around the origin such that (x,y,z)=(-x,-y,-z) and (0,0,0)=(0,0,0).

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`InvertDataset

^

Symbol i

JoinSets[{dat1, dat2, ...}, over] joins dat1, dat2, ... with over slices overlap.

JoinSets[{dat1, dat2, ...},{over1, over2, ...}] joins dat1 and dat2 with over1 slices overlap, Joins dat2 and dat3 with over2 slices overlap and so on.

JoinSets[{dat1, dat2, ...},{over, drop1, drop2}, ...] joins dat1, dat2 with over slices overlap and drops drop1 slices for dat1 and drop2 from drop 2.

DOI: 10.1148/radiol.14140702.

Documentation [Local »](#)

Default Definitions SyntaxInformation[JoinSets] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ► ReverseSets → False ... (8 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`JoinSets

^

Symbol i

MeanRange[Range] calculates the median (50%) and standard deviation (14% and 86%) range and reports it as a string.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MeanRange

^

Symbol i

MeanSignal[data] calculates the mean signal per volume of 4D data.

MeanSignal[data, pos] calculates the mean signal per volume of 4D data for the given list of positions.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MeanSignal] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options UseMask → True

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MeanSignal

^

Symbol i

MeanStd[data] calculates the mean and standard deviation and reports it as a string.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MeanStd

^

Symbol i

MeanStdRange[Range] calculates the mean and standard deviation and the min max range and reports it as a string.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MeanStdRange

^

Symbol i

MedCouple[data] calculates the medcouple of a list of data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MedCouple

^

Symbol i

MinMaxRange[Range] calculates the min and max range and reports it as a string.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MinMaxRange

^

Symbol i

NumberTableForm[data] makes a right aligned table of the numbers with 3 decimal precision.

NumberTableForm[data, n] makes a right aligned table of the numbers with n decimal precision.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[NumberTableForm] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options [▶](#) `TableMethod → NumberForm ... (6 total)`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`NumberTableForm

^

Symbol i

ParameterFit[data] fits a (skew)Normal probability density function to the data.

ParameterFit[{data1, data2,...}] fits a (skew)Normal probability density function to each of the datasets. Is used in Hist.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ParameterFit] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{FitFunction → SkewNormal, FitOutput → Parameters, Method → Automatic}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`ParameterFit`

^

Symbol i

ParameterFit2[data] fits two skewNormal probability density functions to the data. Assuming two compartments, one for fat and one for muscle.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ParameterFit2] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `FitOutput → BestFitParameters`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`ParameterFit2`

^

Symbol i

RotateData[data] rotates the data 180 degree, e.g. inversion of the z direction.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotateData] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`RotateData`

^

Symbol i

RotateTensor[tens] rotates the tensor 180 degree, e.g. inversion of the z direction with also inverting the tensor orientation.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RotateTensor] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`ProcessingTools`RotateTensor`

^

Symbol i

SmartMask[input] crates a smart mask of input, which is either the tensor or the tensor parameters calculated using ParameterCalc.

SmartMask[input, mask] crates a smart mask of input and used the mask as a prior selection of the input.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SmartMask] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options ▶ Strictness → 0.5 ... (4 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SmartMask

^

Symbol i

SNRCalc[data, sigma] calculates the ANR of the data using a noise sigma map for example generated by PCADeNoise.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SNRCalc] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SNRCalc

^

Symbol i

SNRMapCalc[data1, noisemap] calculates the signal to noise ratio of the data using $MN[data]/(1/\sqrt{\pi/2} \text{ sigma})$, where sigma is the local mean of the noise map assuming it is a Rician distribution.

SNRMapCalc[{data1,data2}] calculates the signal to noise ratio from two identical images using $MN[data1,data2] / (.5 \text{ SQRT}[2] \text{ STDV}[data2-data1])$.

SNRMapCalc[{data1, .. dataN}] calculates the signal to noise ratio of the data using MN/sigma where the mean signal MN is the average voxel value over all dynamics N and the sigma is the standard deviation over all dynamics N.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SNRMapCalc] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {OutputSNR → SNR, SmoothSNR → 2}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SNRMapCalc

^

Symbol 

SplitSets[data, Nsets, Nover] splits the data in Nsets with Nover slices overlap.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SplitSets] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {ReverseSets → False, ReverseData → True, PadOverlap → 0}

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ProcessingTools`SplitSets`

^

Options

Symbol 

AxesLabel is an option for graphics functions that specifies labels for axes.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name `System`AxesLabel`

^

Symbol 

B1EqualPower is an option for B1shimming. If true equal power for both channels is used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ProcessingTools`B1EqualPower`

^

Symbol 

B1FilterData is an option for B1MapCalc. If True HammingFilterData is applied to the data before B1 calculation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ProcessingTools`B1FilterData`

^

Symbol 

B1Masking is an option for B1MapCalc. If True then values where S2 is larger than S1 are masked.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ProcessingTools`B1Masking`

^

Symbol 

B1MaxPower is an option for B1Shimming. Specifies how much power can be used per channel.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1MaxPower

^

Symbol 

B1Output is an option for B1MapCalc. Values can be "Map", "MagPhase", or "Complex".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1Output

^

Symbol 

B1Scaling is an option for B1Shimming and CombineB1. Values can be "Relative" or "Absolute". "Absolute" assumes b1 maps are given in uT, "Relative" assumes that maps are in %.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1Scaling

^

Symbol 

B1ShimMethod is an option for B1Shimming. Values can be "All", "Phase" or "Magnitude".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`B1ShimMethod

^

Symbol 

ColorValue is an option for Hist and ErrorPlot. Default {Black, Red}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`ColorValue

^

Symbol i

FitFunction is an option for `ParameterFit`. Options are "Normal" or "SkewNormal". Indicates which function will be fitted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`FitFunction

^

Symbol i

FitOutput is an option for `ParameterFit` and `ParameterFit2`. Option can be "Parameters", "Function" or "BestFitParameters".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`FitOutput

^

Symbol i

GetMaskOnly is an option for `GetMaskData`. If set True all values in the mask are given, if set False only non zero values in the mask are give.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`GetMaskOnly

^

Symbol i

GetMaskOutput is an option for `GetMaskData`. Default is "Slices" which gives the mask data per slices. Else the entire mask data is given as output.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`GetMaskOutput

^

Symbol i

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol 

InterpolationOrder is an option for Interpolation, as well as ListLinePlot, ListPlot3D, ListContourPlot, and related functions, that specifies what order of interpolation to use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`InterpolationOrder

^

Symbol 

JoinSetSplit is an option ofr CorrectJoinSetMotion. If True RegisterDataTransformSplit is used else RegisterDataTransform is used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`JoinSetSplit

^

Symbol 

MaskCompartment is an option for SmartMask. Can be "Muscle" or "Fat".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MaskCompartment

^

Symbol 

MeanMethod is an option for GetMaskMeans. The option can be "NormalDist", "SkewNormalDist", or "Mean".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MeanMethod

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol 

MotionCorrectSets is an option for JoinSets. True motion corrects the individual stacks before joining using CorrectJoinSetMotion.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MotionCorrectSets

^

Symbol 

NormalizeOverlap is an option for JoinSets. True removes strong signal dropoff at the end of a stacks.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`NormalizeOverlap

^

Symbol 

NormalizeSets is an option for JoinSets. True normalizes the individual stacks before joining.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`NormalizeSets

^

Symbol 

OutlierCheckNormality is an option for FindOutliers. If set to True the data is checked for normality using the DistributionFitTest function.
If the data is not normal the data is log transformed before outlier detection.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierCheckNormality

^

Symbol 

OutlierIncludeZero is an option for FindOutliers. If set to True all values that are zero are ignored and considered outliers.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierIncludeZero

^

Symbol 

OutlierIterations is an option for FindOutliers. Specifies how many iterations are used to find the outliers.
Each iteration the outliers are reevaluated on the data with the previously found outliers already rejected.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierIterations

^

Symbol 

OutlierMethod is an option for FindOutliers. values can be "IQR", "SIQR" or "aIQR". "IQR" is used for normally distributed data, "SIQR" or "aIQR" are better for skewed distributions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierMethod

^

Symbol 

OutlierOutput is an option for FindOutliers. If value is "Mask" it gives a list of 1 for data and 0 for outliers. Else the output is {data, outliers}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierOutput

^

Symbol 

OutlierRange is an option for FindOutliers. Specifies how many times the IQR is considered an outlier.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutlierRange

^

Symbol 

OutputSNR is an option for SNRMapCalc.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`OutputSNR

^

Symbol 

PadOutputDimensions is an options for DataTransformation. If False the data is the same dimensions as the input else the data is padded.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`PadOutputDimensions

^

Symbol 

PadOverlap is an option of CorrectJoinSetMotion and JoinSets. it allows for extra motion in the z direction.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`PadOverlap

^

Symbol 

PlotLabel is an option for graphics functions that specifies an overall label for a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PlotLabel

^

Symbol 

ReferenceB1 is an option for B1MapCalc. Default value is None. Can be given a numeric value in uT.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`ReferenceB1

^

Symbol i

ReverseData is an option for JoinSets. Reverses each individual dataset given as input for the JoinSets function. True by default.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`ReverseData

^

Symbol i

ReverseSets is an option for JoinSets. Reverses the order of the datasets, False by default.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`ReverseSets

^

Symbol i

Scaling is an option for Hist2. Scales the individual fits of the fat and muscle compartment.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`Scaling

^

Symbol i

SmartMaskOutput is an option for Smartmask. Can be set to "mask" to output only the mask or "full" to also output the probability mask.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SmartMaskOutput

^

Symbol i

SmartMethod is an option for SmartMask. This specifies how the mask is generated. Can be "Continuous" or "Catagorical".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SmartMethod

^

Symbol 

SmoothSNR is an option for SNRMapCalc.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`SmoothSNR

^

Symbol 

Strictness is an option for SmartMask value between 0 and 1. higher values removes more data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`Strictness

^

Symbol 

TableAlignments is an option for TableForm and MatrixForm which specifies how entries in each dimension should be aligned.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`TableAlignments

^

Symbol 

TableDepth is an option for TableForm and MatrixForm that specifies the maximum number of levels to be printed in tabular or matrix format.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`TableDepth

^

Symbol 

TableDirections is an option for TableForm and MatrixForm which specifies whether successive dimensions should be arranged as rows or columns.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`TableDirections

^

Symbol i

TableHeadings is an option for TableForm and MatrixForm that gives the labels to be printed for entries in each dimension of a table or matrix.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`TableHeadings

^

Symbol i

TableMethod is an option for NumberTableForm. It specifies which number form to uses. Values can be NumberForm, ScientificForm or EngineeringForm.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`TableMethod

^

Symbol i

TableSpacing is an option for TableForm and MatrixForm that specifies how many spaces should be left between each successive row or column.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`TableSpacing

^

Symbol i

UseMask is a function for MeanSignal and DriftCorrect.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`UseMask

^

ReconstructionTools

Functions

Symbol i

AverageKspace[kspace, types, met] calculates the average of the kspace over the type "N_aver". The method that can be used are "Mean", "Total", "Hamming".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`AverageKspace

^

Symbol



CoilCombine[sig] combines the coil signals sig. Where sig is {nCoils, ...}.

CoilCombine[sig, cov] combines the coil signals sig. Where sig is {nCoils, ...} and cov the complex noise correlation matrix.

CoilCombine[sig, cov, sens] combines the coil signals sig. Where sig is {nCoils, ...} and cov the complex noise correlation matrix and sense the coils sensitivity.

Possible coil combination methods are "Sum", "RootSumSquares", "RoemerEqualNoise", "RoemerEqualSignal", "WSVD".

RootSumSquares needs the signal. Can be performed with and without the noise covariance matrix

RoemerEqualNoise needs the signal and the noise covariance matrix. Can be performed with

and without the sense data, without sense data the sensitivity is estimated using the signal and the RSS reconstruction of the signal.

RoemerEqualSignal needs the signal and the noise covariance matrix and the sense data.

WSVD needs the signal and the noise covariance matrix.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CoilCombine] = {ArgumentsPattern → {_, ___, ___, OptionsPattern[]}}

Options {Method → RoemerEqualNoise, SenseSmoothing → True, NoisePrewhitening → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`CoilCombine



Symbol



CoilWeightedRecon[kspace, noise, head] performs reconstruction of raw MS2D MRI data. The input kspace, noise and head are obtained using ReadListData.

The coil combination Methods can be "Roemer" or "RSS".

Documentation [Local »](#)

Default Definitions SyntaxInformation[CoilWeightedRecon] = {ArgumentsPattern → {_, ___, ___, OptionsPattern[]}}

Options ▶ EchoShiftData → 0 ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`CoilWeightedRecon



Symbol i

CoilWeightedReconCSI[kspace, noise, head] performs reconstruction of raw 3DCSI data. The input kspace, noise and head are obtained using ReadListData. The coil combination Methods can be "Roemer" or "WSVD".

Documentation [Local »](#)

Default Definitions SyntaxInformation[CoilWeightedReconCSI] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options ▶ HammingFilter → False ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`CoilWeightedReconCSI

^

Symbol i

DeconvolveCSIdata[spectra] deconvolves the CSI spectra after HammingFilterCSI to revert the blurring of the hammingfiltering. DeconvolveCSIdata[spectra, ham] deconvolves the CSI spectra with the acquired weighting ham to revert the blurring of the kspace weighting.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DeconvolveCSIdata] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {WienerRegularization → 0.007, DeconvolutionMethod → Wiener}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`DeconvolveCSIdata

^

Symbol i

FourierKspace2D[kspace,head] performs a 2D reconstruction of 2D kspace data. Where kspace and head are generated by ReadListData.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FourierKspace2D] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`FourierKspace2D

^

Symbol i

FourierKspace3D[kspace,head] performs a 3D reconstruction of 3D kspace data. Where kspace and head are generated by ReadListData.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FourierKspace3D] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`FourierKspace3D

^

Symbol i

FourierKspaceCSI[kspace,head] performs a 3D reconstruction of 3D CSI kspace data. Where kspace and head are generated by ReadListData.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FourierKspaceCSI] = {ArgumentsPattern → {_, _, ...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`FourierKspaceCSI`

^

Symbol i

FourierRescaleData[data] rescales the data to double the dimensions using zero padding in fourier space.

FourierRescaleData[data, facotr] rescales the data to factor times the dimensions using zero padding in fourier space.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FourierRescaleData] = {ArgumentsPattern → {_, ...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`FourierRescaleData`

^

Symbol i

FourierShift[data] shift the data to the right by half the data dimensions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FourierShift] = {ArgumentsPattern → {_, ...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`FourierShift`

^

Symbol i

FourierShifted[kspace] shifts the kspace half the kspace dimensions and then performs a FourierTransform on the kspace.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FourierShifted] = {ArgumentsPattern → {_, ...}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`FourierShifted`

^

Symbol 

HammingFilterCSI[kspace] applies a Hammingfilter to the k-space data. The data can be 1D, 2D or 3D, the spectral dimensions is the last dimensions (x,y,z, spectra).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[HammingFilterCSI] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`HammingFilterCSI`

^

Symbol 

HammingFilterData[kspace] applies a Hammingfilter to the data. The data is in image space and can be 1D, 2D or 3D.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[HammingFilterData] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`HammingFilterData`

^

Symbol 

InverseFourierShift[data] shift the data to the left by half the data dimensions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[InverseFourierShift] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`InverseFourierShift`

^

Symbol 

InverseFourierShifted[data] performs a InverseFourierTransform on the data and then shifts the kspace half the kspace dimensions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[InverseFourierShifted] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`InverseFourierShifted`

^

Symbol i

MakeHammingFilter[xdim] makes a 1D HammingKernel for filtering k-space.

MakeHammingFilter[{xdim}] makes a 1D HammingKernel for filtering k-space.

MakeHammingFilter[{xdim, ydim}] makes a 2D HammingKernel for filtering k-space in 2D CSI data of size {xdim, ydim}.

MakeHammingFilter[{xdim, ydim, zdim}] makes a 3D HammingKernel for filtering k-space in 3D CSI data of size {xdim, ydim, zdim}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeHammingFilter] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`MakeHammingFilter

^

Symbol i

MakeSense[coils, cov] makes a sense map for coils. Each coil signal is divided by the RSS reconstuction of the coils.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeSense] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {SenseSmoothing → Hamming, SenseWeight → 1}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`MakeSense

^

Symbol i

MeanType[kspace, types, type] calcualtes the Mean of the kspace data on type, where type is part of types. The kspace and types are generated by ReadListData.

MeanType[{kspace, types}, type] calcualtes the Mean of the kspace data on type, where type is part of types.

MeanType[kspace, types, {type,...}] calcualtes the Mean of the kspace data on each of the list type, where type is part of types.

Output is {kspace, types}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MeanType] = {ArgumentsPattern → {_, _ , _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`MeanType

^

Symbol



NoiseCorrelation[noise] calculates the noise correlation matrix, noise is {nrCoils, noise Samples}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[NoiseCorrelation] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NoiseCorrelation



Symbol



NoiseCovariance[noise] calculates the noise covariance matrix, noise is {nrCoils, noise Samples}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[NoiseCovariance] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NoiseCovariance



Symbol



NoisePrewhitening[sig, cov] prewhitens the signal sig using the noise covariance matrix cov. The signal and covariance matrix can be any dimension.

Documentation [Local »](#)

Default Definitions SyntaxInformation[NoisePrewhitening] = {ArgumentsPattern → {_, _ OptionsPattern[]}}

Options Method → PCA

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NoisePrewhitening



Symbol



NormalizeSpectra[spec] normalizes spectra to be scaled to the max value of the absolute signal = 1000. Can be any dimension.

Documentation [Local »](#)

Default Definitions SyntaxInformation[NormalizeSpectra] = {ArgumentsPattern → {}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NormalizeSpectra



Symbol i

OrderKspace[kspace, types, order] reorders the kspace data to order, where order is a list and each value is a part of types. The kspace and types are generated by ReadListData.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[OrderKspace] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`OrderKspace`

^

Symbol i

ReadListData[file] reads a list/data raw data file from the philips MR platform. The input file can either be .list or .data file.

Output is {{rawData, noise}, {head, types}}.

ReadListData[file, print] does the same but if print is set False no reporting is done.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ReadListData] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`ReadListData`

^

Symbol i

SagittalTranspose[data] makes a transpose of the data of the second level and reverses the slices.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SagittalTranspose] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`SagittalTranspose`

^

Symbol i

ShiftedFourier[kpace] performs a FourierTransform on the kspace and then shifts the data half the data dimensions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ShiftedFourier] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ReconstructionTools`ShiftedFourier`

^

Symbol 

ShiftedInverseFourier[data] shifts the data half the data dimensions and then performs a InverseFourierTransform on the data.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ShiftedInverseFourier] = {ArgumentsPattern → {_, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`ShiftedInverseFourier

^

Symbol 

TotalType[kspace, types, type] calculates the Total of the kspace data on type, where type is part of types. The kspace and types are generated by ReadListData.

TotalType[{kspace, types}, type] calculates the Total of the kspace data on type, where type is part of types.

TotalType[kspace, types, {type,...}] calculates the Total of the kspace data on each of the list type, where type is part of types.

Output is {kspace, types}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TotalType] = {ArgumentsPattern → {_, _, ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`TotalType

^

Symbol 

ZeropadData[data] upscales the data resolution by a factor 2 by zeropadding.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`ZeropadData

^

Options

Symbol 

AcquisitionMethod is an option for CoilWeightedReconCSI. Values can be "Fid" or "Echo".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`AcquisitionMethod

^

Symbol i

CoilSamples is an option for **CoilWeightedReconCSI** and specifies how many fud samples are used to calculate the coil sensitivity for Roemer reconstruction.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`CoilSamples

^

Symbol i

DeconvolutionMethod is an option for **DeconvolveCSIData**. It specifies which deconvolution method to used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`DeconvolutionMethod

^

Symbol i

DenoiseCSI is an option for **CoilWeightedReconCSI**. Specifies if denoiseing needs to be applied. Can be "Coils", "Spectra" or False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`DenoiseCSI

^

Symbol i

EchoShiftData is an option for **CoilWeightedRecon**.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`EchoShiftData

^

Symbol i

HammingFilter is an option for **CoilWeightedReconCSI**. If True it applies a spatial hamming filter to the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`HammingFilter

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

NoisePrewhiten is an option for CoilWeightedReconCSI. Specifies if noise pre whitening is done for Roemer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NoisePrewhiten

^

Symbol 

NormalizeOutputSpectra is an option for CoilWeightedReconCSI.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`NormalizeOutputSpectra

^

Symbol 

OutputSense is an option for CoilWeightedRecon. If set true the function will also output the used Sense map.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`OutputSense

^

Symbol 

ReconFilter is an option for CoilWeighted recon. If true the reconstruction gets a hamming filter.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`ReconFilter

^

Symbol i

RescaleRecon is an option for CoilWeightedRecon. If set true the data will be scaled to the range 0–1000.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`RescaleRecon

^

Symbol i

SenseSmoothing is an option for MakeSense. If set True the data and sense maps are smoothed using hamming filters.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`SenseSmoothing

^

Symbol i

SenseWeight is an option for MakeSense. Is a integer between 0 and Infinity and define the amount of SOS weigthing in the sensemap.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`SenseWeight

^

Symbol i

WienerRegularization is an option for DeconvolveCSIdata. It defines te amount of regularization used in the wiener deconvoltuion.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ReconstructionTools`WienerRegularization

^

RelaxometryTools

Functions

Symbol i

CalibrateEPGT2Fit[*datan*, *times*, *angle*] calculates the Fat t2 relaxation that will be used in the EPGT2fit.

Outputs the fat t2 value.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CalibrateEPGT2Fit] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▸ `EPGR RelaxPars` → {{0, 100}, {20, 300}, {1400., 365.}}... (4 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`RelaxometryTools`CalibrateEPGT2Fit`

^

Symbol i

CreateT2Dictionary[{*T1m*, *T1f*}, {*nEcho*, *detlaTE*}, *angle*] Creates a EPG signal dictionary used for EPGT2fit.

Every dictionary that is defined is cached.

The output is in units as defined by the *detlaTE*, e.g. if *detlaTE* is in ms the output is in ms.

The TR and TE should be in the same units as *Dela*.

Output is {dictionary, vals}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CreateT2Dictionary] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▸ `DictB1Range` → {0.5, 1.4, 0.01}... (7 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`RelaxometryTools`CreateT2Dictionary`

^

Symbol



DictionaryMinSearch[dictionary, y] performs dictionary minimization of data y. dictionary is generated with CreateT2Dictionary.

Output is {{t2, b1}, fwfraction, residualError}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[DictionaryMinSearch] = {ArgumentsPattern → {_, _ ...}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictionaryMinSearch



Symbol



EPGSignal[{nEcho, echoSpace}, {t1, t2}, {ex_angle, ref_angle}, b1] generates a EPG t2 curve with stimulated echos.

t1, t2 and echoSpace are in ms, angel is in degree, b1 is between 0 and 1.

Output is the EPG Signal vector.

EPGSignal[] is based on DOI: 10.1002/jmri.24619.

Documentation [Local »](#)

Default Definitions SyntaxInformation[EPGSignal] = {ArgumentsPattern → {_, _ , _ , _ , OptionsPattern[]}}

Options EPGOutput → Magnitude

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGSignal



Symbol i

EPGT2Fit[data, {nEcho, detlaTE}, {exitation, refoucs}] fits the t2 based on Marty B et.al. Simultaneous muscle water t2 and fat fraction mapping using transverse relaxometry with stimulated echo compensation.

Exitation and refocus are the RF pulse angles e.g. 90,180. They can also be a range of angeles over the slice profile as defined by GetSliceProfile.

The output is in units as defined by the detlaTE, e.g. if detlaTE is in ms the output is in ms.

The exitation and refocus are defined in Degrees.

Output is {{{T2map,B1Map},{wat, fat, fatMap}, residual},callibration} or {{T2map,B1Map},{wat, fat, fatMap}, residual}.

EPGT2Fit[] is based on DOI: 10.1002/nbm.3459.

Documentation [Local](#) »

Default Definitions SyntaxInformation[EPGT2Fit] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options ▶ EPGRelaxPars → {1400., 365.} ... (18 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGT2Fit

^

Symbol i

NonLinearEPGFit[{vals, T2cons}, y] performs dictionary minimization of data y. vals = {{T1muscle, T1fat, T2fat}, {nEcho, echoSpace, angle}}.

Output is {{t2, b1}, fwfraction, residualError}.

Documentation [Local](#) »

Default Definitions SyntaxInformation[NonLinearEPGFit] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`NonLinearEPGFit

^

Symbol 

ShiftPulseProfile[angs, shift] shifts the reference pulse profile by shift and makes the. ans = {exitation, refocus} as generated by GetPulseProfile. Shift is the shift in sample points.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`ShiftPulseProfile

^

Symbol 

T1Fit[data, TR] fits the t1 value to the data using a nonlinear method. The output is in units as defined by the TR, e.g. if TR is in ms the TR is in ms. Output is {t1, apar, bpar}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[T1Fit] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`T1Fit

^

Symbol 

T1rhoFit[data, EchoTimes] fits the T1rho value to the data using linear or nonlinear methods.

The output is in units as defined by the EchoTimes, e.g. if EchoTimes is in ms the output is in ms.

Output is {S(0), T1rhomap}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[T1rhoFit] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options Method → Linear

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`T1rhoFit

^

Symbol i

T2Fit[data, EchoTimes] fits the t2 value to the data using linear or nonlinear methods. The output is in units as defined by the EchoTimes, e.g. if EchoTimes is in ms the output is in ms. Output is {S(0), t2}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[T2Fit] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options Method → Linear

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`T2Fit

^

Symbol i

TriExponentialT2Fit[data, EchoTimes] fits the t2 based on Azzabou N et.al.

Validation of a generic approach to muscle water t2 determination at 3T in fat-infiltrated skeletal muscle. J. Magn. Reson. 2015.

The fat t2 parameters are automatically estimated from the high signal voxels from the last echo.

The output is in units as defined by the EchoTimes, e.g. if EchoTimes is in ms the output is in ms.

The output fraction is between 0 and 1.

Output is {{S(0), fatFraction, muscleFraction, T2map}, calibration} or {S(0), fatFraction, muscleFraction, T2map}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TriExponentialT2Fit] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options OutputCalibration → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`TriExponentialT2Fit

^

Options

Symbol i

DictB1Range is an option for CreateT2Dictionary and EPGT2Fit. It specifies the range and step of the b1 values in the dictionary {min, max, step}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictB1Range

^

Symbol i

DictT2fRange is an option for CreateT2Dictionary and EPGT2Fit. is specifies the range and step of the t2 fat values in the dictionary {min, max, step} in ms.

If a single value is given this fixed value is used a long as EPGCalibrate is False.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictT2fRange

^

Symbol i

DictT2fValue is an option for EPGFit.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictT2fValue

^

Symbol i

DictT2IncludeWater is an options for EPGT2Fit.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictT2IncludeWater

^

Symbol i

DictT2Range is an option for CreateT2Dictionary and EPGT2Fit. is specifies the range and step of the t2 values in the dictionary {min, max, step} in ms.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`DictT2Range

^

Symbol i

EPGCalibrate is an option for EPGT2Fit. If set to True it does autmatic callibration of the t2 fat relaxation time.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGCalibrate

^

Symbol 

EPGFatShift is an options for EPGT2Fit. Specifies the amount of shift of the fat refocusing pulse relative to the fat exitation pulse.
Can be obtained form GetPulseProfile.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`RelaxometryTools`EPGFatShift

^

Symbol 

EPGFitFat is an option for EPGT2Fit.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`RelaxometryTools`EPGFitFat

^

Symbol 

EPGFitPoints is a option for CalibrateEPGT2Fit and EPGT2Fit. Number of points is 200 by default.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`RelaxometryTools`EPGFitPoints

^

Symbol 

EPGMethod is an optionf for EPGT2Fit. Values can be "NLLS", "dictionary" or "dictionaryM".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`RelaxometryTools`EPGMethod

^

Symbol 

EPGMethodCal is an option for CalibrateEPGT2Fit and EPGT2Fit. The calibration can be done using "1comp", "2comp", "2compF".

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`RelaxometryTools`EPGMethodCal

^

Symbol i

EPGOutput is an option for EPGBSignal. It specifies how to format the output, as magnitude or complex.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGOutput

^

Symbol i

EPGRefocussingPhase is an option for EPGT2Fit and CreateT2Dictionary. It specifies the phase of the refocussing pulse.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGRefocussingPhase

^

Symbol i

EPGR RelaxPars is and option for EPGT2Fit. Needs to be {T1muscl, T1Fat, T2Fat} in ms, default is {1400,365,137} in ms.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGR RelaxPars

^

Symbol i

EPGSmoothB1 is an options for EPGT2Fit. If set to True the b1 map of the fit will be smoothed after which the minimization if perfomed again but with a fixed b1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`EPGSmoothB1

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol i

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol i

OutputCalibration is an option for EPGT2Fit and TriExponentialT2Fit. If true it outputs the calibration values.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`OutputCalibration

^

Symbol i

WaterFatShift is an options for EPGT2Fit. It specifies the amount of water fat shift in voxels.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`WaterFatShift

^

Symbol i

WaterFatShiftDirection is an options for EPGT2Fit. It specifies the water fat shift direction: "left", "right", "up" and "down".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`RelaxometryTools`WaterFatShiftDirection

^

ScientificColorData

Functions

Symbol i

AddScientificColors[dir] adds the scientific colour data (<https://zenodo.org/records/8409685>) from the specified folder.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ScientificColorData`AddScientificColors

^

Symbol
ExtractColorData[] Extracts the scientific colordata archive (https://zenodo.org/records/8409685).
Documentation Local »
Attributes {Protected, ReadProtected}
Full Name QMRITools`ScientificColorData`ExtractColorData

Options

SegmentationTools

Functions

Symbol
ApplySegmentationNetwork[data, net] segments data using net. Data can be an array or a nii file path. Net can be a network name, file, or NetGraph. ApplySegmentationNetwork[data, net, node] returns the network output at the specified intermediate node rather than the final segmentation. ApplySegmentationNetwork[{datFol, outFol}, net] processes all nii files in datFol and saves results to outFol. ApplySegmentationNetwork[{{datFol, outFol}, {inTag, outTag}}, net] uses custom file tags to find input and name output files.
Documentation Local »
Default Definitions SyntaxInformation[ApplySegmentationNetwork] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}
Options ▶ TargetDevice → CPU ... (4 total)
Attributes {Protected, ReadProtected}
Full Name QMRITools`SegmentationTools`ApplySegmentationNetwork

Symbol
AugmentImageData[image, {rotate, flip}] augments the input image by rotating between -180 and 180 degrees and flipping. The inputs rotate and flip can be set to True or False. AugmentImageData[{image, ..}, {rotate, flip}] same but for a list of images.
Documentation Local »
Attributes {Protected, ReadProtected}
Full Name QMRITools`SegmentationTools`AugmentImageData

Symbol i

AugmentTrainingData[{data, seg}, vox] augments data and segmentation consistently.

AugmentTrainingData[{data, seg}, vox, aug] controls augmentation. aug can be True/False for all, or a list {flip, rotate, scale, noise, blur}. Default is all True.

AugmentTrainingData[data, vox] augments data only, returning the data without segmentation.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[AugmentTrainingData] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `Augment2D → False`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`AugmentTrainingData`

^

Symbol i

CheckSegmentation[seg] checks the segmentation for errors and returns a vector of two numbers, the first indicates if the segmentation has more than one region, the second indicates if it has holes.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CheckSegmentation] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`CheckSegmentation`

^

Symbol i

ClassifyData[data, method] classifies the input data using the Body classification network.

Method "Body" returns {side, positions}. Any method containing "Side" returns the detected side only.

Any other method returns the raw network output.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ClassifyData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `{TargetDevice → CPU, Monitor → False}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`ClassifyData`

^

Symbol 

DataToPatches[data, patchSize] creates non-overlapping patches covering data with minimal overlap.

DataToPatches[data, patchSize, n] creates patches with n additional overlap steps per dimension.

DataToPatches[data, patchSize, pts] extracts patches at the given pre-computed ranges pts.

DataToPatches[data, pts] extracts patches at ranges pts without resizing.

Output is {patches, ranges} where ranges can be passed to PatchesToData.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DataToPatches] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {PatchNumber → 0, PatchPadding → 0}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`DataToPatches

^

Symbol 

DiceSimilarity[ref, pred] gives the Dice Similarity between 1 and 0 of segmentations ref and pred for class equals 1.

DiceSimilarity[x, y, class] gives the Dice Similarity of segmentations ref and pred for class.

DiceSimilarity[x, y, {class, ...}] gives the Dice Similarity of segmentations ref and pred for the list of gives classes.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DiceSimilarity] = {ArgumentsPattern → {_, _, ...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`DiceSimilarity

^

Symbol 

GetNeuralNet[name] loads a pre-trained neural net by asset name. The net is cached within a session.

GetNeuralNet[file] loads a net from a file path directly.

GetNeuralNet[net] passes through an existing NetGraph or NetChain unchanged.

GetNeuralNet["Clear"] clears the session cache.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetNeuralNet] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`GetNeuralNet

^

Symbol



GetTrainData[data, batch size, patch] creates a training batch of size batch size with patch size patch.

The input data can be out of memory in the form of a list of "*.wxf" files that contain the data, segmentation and voxel size or a list of "*.nii" files in the form {"data.nii", "segmentation.nii"}..}. The input data can be in memory in a list in the form {{data, segmentation, vox}..}

GetTrainData[data, batch size, patch, nClass] If nClass is set to an value n > 0 the segmentations are decoded in n classes.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetTrainData] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}

Options {PatchesPerSet → 1, AugmentData → True, PadData → False}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`GetTrainData



Symbol



ImportITKLabels[] loads the default MusclesLegLabels asset.

ImportITKLabels[file] imports an ITKSnap label file by path or asset name.

ImportITKLabels[file, "Labels"] returns label integer → name replacement rules.

ImportITKLabels[file, "Names"] returns name → label integer replacement rules.

ImportITKLabels[file, "List"] returns {names, labels} as separate lists.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ImportITKLabels] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`ImportITKLabels



Symbol



JaccardSimilarity[ref, pred] gives the Jaccard Similarity between 1 and 0 of segmentations ref and pred for class equals 1.

JaccardSimilarity[x, y, class] gives the Jaccard Similarity of segmentations ref and pred for class.

JaccardSimilarity[x, y, {class, ..}] gives the Jaccard Similarity of segmentations ref and pred for the list of gives classes.

Documentation [Local »](#)

Default Definitions SyntaxInformation[JaccardSimilarity] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`JaccardSimilarity



Symbol



MakeChannelClassGrid[data, label] makes a 3 x 3 grid of cross-sectional images of the channels data overlaid with a cross-sectional image of the classes label of a training dataset generated

MakeChannelClassGrid[data, label, n] makes a n x n.

MakeChannelClassGrid[data, label, {n, m}] makes a n x m.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeChannelClassGrid] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SegmentationTools`MakeChannelClassGrid`



Symbol



MakeChannelClassImage[data, label] makes a cross-sectional image of the channels data overlaid with a cross-sectional image of the classes label of a training dataset generated

MakeChannelClassImage[data, label, {off,max}] same but with explicit definition of background value b and number of classes n.

MakeChannelClassImage[data, label, vox] same but with the aspect ratio determined by vox.

MakeChannelClassImage[data, label, {off,max}, vox] same with explicit definition and aspect ratio definition.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeChannelClassImage] = {ArgumentsPattern → {_, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SegmentationTools`MakeChannelClassImage`



Symbol



MakeChannellImage[data] makes a cross-sectional image of the channels data of a training dataset generated by GetTrainData.

MakeChannellImage[data, vox] same but with the aspect ratio determined by vox.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeChannellImage] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SegmentationTools`MakeChannellImage`



Symbol i

MakeClassImage[label] makes a cross-sectional image of the classes label of a training dataset generated by `GetTrainData`

MakeClassImage[label, {b, n}] same but with explicit definition of background value `b` and number of classes `n`.

MakeClassImage[data, vox] same but with the aspect ratio determined by `vox`.

MakeClassImage[label, {b, n}, vox] same with explicit definition and aspect ratio definition.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeClassImage] = {ArgumentsPattern → {_, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`MakeClassImage`

^

Symbol i

MakeDistanceMap[mask] makes a distance map of the given mask in voxels. The distance map is negative inside the mask and positive outside the mask.

MakeDistanceMap[mask, vox] makes a distance map of the given mask in the same unit as `vox`. The distance map is negative inside the mask and positive outside the mask.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeDistanceMap] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `DistanceRange → Automatic`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`MakeDistanceMap`

^

Symbol i

MuscleLabelToName[{lab, ..}] converts list of `lab`, which need to be integers to names using the file `GetAssetLocation["MusclesLegLabels"]`.

MuscleLabelToName[{lab, ..}, file] does the same but uses a user defined ITKSnap label definition file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MuscleLabelToName] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SegmentationTools`MuscleLabelToName`

^

Symbol



MuscleNameToLabel[{name, ..}] converts list of muscle names to integer labels using the file GetAssetLocation["MusclesLegLabels"]

MuscleNameToLabel[{name, ..}, file] does the same but uses a user defined ITKSnap label definition file.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MuscleNameToLabel] = {ArgumentsPattern → {_, ..}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`MuscleNameToLabel



Symbol



PatchesToData[patches, ranges] reconstructs data from patches at the given ranges.

PatchesToData[patches, ranges, dim] reconstructs into an array of dimensions dim. Overlapping patches are averaged.

PatchesToData[patches, ranges, dim, labels] reconstructs segmentation patches. For each

label only the largest connected component is kept and overlapping labels are resolved. Returns a merged integer segmentation.

Documentation [Local »](#)

Default Definitions SyntaxInformation[PatchesToData] = {ArgumentsPattern → {_, .., .., ..}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PatchesToData



Symbol



PrepareTrainingData[inFolder, outFolder] prepares the data in de inFolder for training a neural network for segmentation and outputs in outFolder.

PrepareTrainingData[{labFolder, datFolder}, outFolder] does the same but the labels are stored in labFolder and data is stored in datFolder.

Documentation [Local »](#)

Default Definitions SyntaxInformation[PrepareTrainingData] = {ArgumentsPattern → {_, .., OptionsPattern[]}}

Options > LabelTag → label ... (9 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PrepareTrainingData



Symbol i

SegmentData[data] segments the data using the default "Legs" method.

SegmentData[data, what] segments using the specified anatomical region. What can be "Legs", "LegsHip", "UpperLegs", "LowerLegs", "Shoulder", "Hip", or "Body".

SegmentData[data, {what, netFile}] uses a custom network file instead of the built-in net.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SegmentData] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options ▸ TargetDevice → CPU ... (5 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SegmentData

^

Symbol i

SegmentDataGUI[] is a function that creates a graphical user interface (GUI) for segmenting data.

It prompts the user to enter the paths for the input and output files, and allows them to select the segmentation type.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SegmentDataGUI

^

Symbol i

ShowTrainLog[folder] shows an interactive plot of the training log stored in folder as JSON files.

ShowTrainLog[folder, min] filters rounds with fewer than min progress entries. Default is 5.

The interface allows selecting metrics, toggling log scale, Gaussian filtering, and live folder reloading.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`ShowTrainLog

^

Symbol



SplitDataForSegmentation[data] splits data for "Legs" segmentation, detecting left/right side and upper/lower leg position automatically using a classification network.

SplitDataForSegmentation[data, what] splits for the specified region. What can be "Legs", "LegsHip", "UpperLegs", "LowerLegs", "Shoulder", "Hip", or "Body".

SplitDataForSegmentation[data, seg] splits both data and segmentation identically for "Legs".

SplitDataForSegmentation[data, seg, what] does the same for the specified region.

Output is {{patches, ranges, dim}, locations}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SplitDataForSegmentation] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {Monitor → False, TargetDevice → CPU}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SplitDataForSegmentation



Symbol



SurfaceDistance[ref, pred] gives the surface distance of segmentations ref and pred for class 1, assuming isotropic unit voxels. Use the vox argument for correct physical distances.

SurfaceDistance[ref, pred, class] gives the surface distance for the given integer class.

SurfaceDistance[ref, pred, {class, ..}] gives the surface distance for each class in the list.

SurfaceDistance[ref, pred, class, vox] gives the surface distance in millimeters using voxel size vox.

SurfaceDistance[ref, pred, {class, ..}, vox] does the same for a list of classes.

The Method option controls the distance metric: "Mean", "Median", "RMS", "HD", "HD95" (default), or "Std". A list of methods can be given to return multiple metrics at once.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SurfaceDistance] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options Method → HD95

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SurfaceDistance



Symbol i

TrainSegmentationNetwork[{inFol, outFol}] trains a segmentation network from scratch. Training data should be wxf files in inFol. Progress is saved each round to outFol.

TrainSegmentationNetwork[{inFol, outFol}, netCont] continues training. netCont can be "Start" to restart, a NetGraph, a wlnet file, or a previous outFol to continue from the last saved network.

Loss functions can be All or a subset of {"Dice", "Jaccard", "Tversky", "MSD", "CE", "Focal", "TopK"}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TrainSegmentationNetwork] = {ArgumentsPattern → {{_, _}, _}, OptionsPattern[]}}

Options ▶ LoadTrainingData → True ... (26 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`TrainSegmentationNetwork

^

Symbol i

QMRITools`SegmentationTools`\$debugUnet

Documentation [Local »](#)

Attributes {ReadProtected}

Full Name QMRITools`SegmentationTools`\$debugUnet

^

Options

Missing [UnknownSymbol, Augment2D]

Symbol i

ActivationType is an option for MakeUnet. It specifies which activation layer is used in the network. It can be "LeakyRELU" or any type allowed by a "name" definition in ElementwiseLayer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`ActivationType

^

Symbol i

AugmentData is an option for GetTrainData and TrainSegmentationNetwork. If set True the training data is augmented.

It can also be set to "2D" or "3D" to control if augmentation is done through plane or only in-plane.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`AugmentData

^

Symbol 

BatchSize is an option for NetTrain and related functions that specifies the size of a batch of examples to process together.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`BatchSize

^

Symbol 

BlockType is an option for MakeUnet. It specifies the type of block used in the network. It can be "Conv", "UNet", "ResNet", "DenseNet", "Inception", or "U2Net".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`BlockType

^

Symbol 

CatenateMethod is an option for MakeUnet. It specifies how the network catenates the skip and upscale channel. It can be "Tot", or "Cat".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`CatenateMethod

^

Symbol 

CleanUpSegmentations is an option for PrepareTrainingData. If set to True the segmentations are cleaned up by removing holes reducing to one volume and smoothing.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`CleanUpSegmentations

^

Symbol 

DataPadding is an option for ApplySegmentationNetwork. Defines how much to pad the data patches in all directions.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`DataPadding

^

Symbol 

DataTag is an option for PrepareTrainingData. It defines the tag used in the filenames of the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`DataTag

^

Symbol 

DistanceRange is an option for MakeDistanceMap. It defines the range of the distance map outside the segmentation in voxels.

Values can be Automatic, All, or a integer value. If All the distance map is calculated for the whole image. If 0 the distance map is only calculated inside the segmentation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`DistanceRange

^

Symbol 

DownsampleSchedule is an option for MakeUnet. It defines how the data is downsampled for each of the deeper layers of the Unet.

By default is a factor two for each layer. A custom schedule for a 5 layer 3D Unet could be `{{2,2,2},{1,2,2},{2,2,2},{1,2,2}, 1}`.

The deepest layer is always downsampled by 1 and therefore not needed to be specified.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`DownsampleSchedule

^

Symbol 

DropoutRate is an option for MakeUnet. It specifies how much dropout is used after each block. It is a value between 0 and 1, default is .2.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`DropoutRate

^

Symbol i

FeatureSchedule is an option for MakeUnet. It defines how the number of features is up-sampled for each of the deeper layers of the Unet. By default it increases the number of features by a factor 2 each layer, i.e. {1, 2, 4, 8, 16}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`FeatureSchedule

^

Symbol i

InputLabels is an option for PrepareTrainingData. Can be set to a list of integers corresponding to the labels to be used from the given segmentation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`InputLabels

^

Symbol i

L2Regularization is an option for TrainSegmentationNetwork. It defines the L2 regularization factor.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`L2Regularization

^

Symbol i

LabelTag is an option for PrepareTrainingData. It defines the tag used in the filenames of the label data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`LabelTag

^

Symbol i

LearningRate is an option for NetTrain that specifies the rate at which to adjust neural net weights in order to minimize the training loss.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`LearningRate

^

Symbol i

LoadTrainingData is an option for TrainSegmentationNetwork. If set to True the training data is loaded from the disk.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`LoadTrainingData

^

Symbol i

LossFunction is an option for NetTrain that specifies how to compare actual and requested outputs from a neural net.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`LossFunction

^

Symbol i

MaxPatchSize is an option for SegmentData and ApplySegmentationNetwork. Defines the maximum memory a patch can use in Gb. Higher is better.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`MaxPatchSize

^

Symbol i

MaxTrainingRounds is an option for NetTrain and related functions that specifies the maximum number of rounds of training to do.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`MaxTrainingRounds

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol i

Monitor*[expr, mon]* generates a temporary monitor cell in which the continually updated current value of *mon* is displayed during the course of evaluation of *expr*.

Documentation [Local »](#) | [Web »](#)

Attributes {HoldAll, Protected, ReadProtected}

Full Name System`Monitor

^

Symbol i

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol i

MonitorInterval is an option for TrainSegmentationNetwork. It defines how often the training is monitored.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`MonitorInterval

^

Symbol i

MultiChannel is an option for TrainSegmentationNetwork, If set to True it will train on multi channel input data. If set to False it will select a random channel.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`MultiChannel

^

Symbol i

NetworkArchitecture is an option for MakeUnet. It defines the architecture of the network. It can be "UNet", "UNet+", or "UNet++".

For "UNet+" or "UNet++" it can also be {arch, i} where i specifies how many of the top layers are connected to the mapping layer.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NetworkArchitecture

^

Symbol 

NetworkDepth is an option for MakeUnet. It specifies how deep the UNET will be.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NetworkDepth

^

Symbol 

NormalizationType is an option for MakeUnet. It specifies which normalization layer is used in the network. It can be "Batch" or "Instance" for either BatchNormalization or NormalizationLayer which is instance normalization.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`NormalizationType

^

Symbol 

OutputLabels is an option for PrepareTrainingData. Can be set to a list of integers. The used label number will be replaced by these numbers.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`OutputLabels

^

Symbol

OutputTag is an option for PrepareTrainingData. It defines the tag that will be added to the output training data.

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`OutputTag

^

Symbol 

PadData is an option for GetTrainData and TrainSegmentationNetwork.. If set to an integers the that number of slices on the top and bottom of the data can be made 0. This is done to learn cut of datasets.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PadData

^

Symbol 

PatchesPerSet is an option for GetTrainData. Defines how many random patches per dataset are created within the batch.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PatchesPerSet

^

Symbol 

PatchNumber is an option for DataToPatches. Can be an integer value ≥ 0 . The larger the number the more overlap the patches have.
The minimal number of patches in each direction is calculated, and then for each dimension the given number is added.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PatchNumber

^

Symbol 

PatchPadding is an option for DataToPatches. Can be an integer value ≥ 0 . It pads the chosen patch size with the given number.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PatchPadding

^

Symbol 

PatchSize is an option for TrainSegmentationNetwork. Defines the patch size used in the network training.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`PatchSize

^

Symbol 

RescaleMethod is an option for MakeUnet. It specifies how the network rescales. It can be "Conv" or "Pool".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`RescaleMethod

^

Symbol 

RoundLength is an option for TrainSegmentationNetwork. Defines how many batches will be seen during each training round.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`RoundLength

^

Symbol 

SegmentationDimension is an option for SegmentData. The default value is "3D" but can also be set to "2D" to used 2D slice by slice segmentation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SegmentationDimension

^

Symbol 

SegmentationResolution is an option for SegmentData. If specified it allows to rescale the data before its processed by the segmentation network. It can be {voxData, voxSegment} or {dimension} which are the same inputs as for the RescaleData function which is used internally.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SegmentationResolution

^

Symbol

SegmentationsPerSlice is an option for PrepareTrainingData. It defines how many segmentations should be present in a slice for it to be used in the training data.

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`SegmentationsPerSlice

^

Symbol i

SettingSchedule is an option for MakeUnet. It defines the settings for the Unet blocks. If one setting is given it applied to all layers.

If a list of settings is given the settings can be different per layer. The following settings are the default settings.

"Unet": convolution block repetitions, 2, "ResNet" -> convolution block repetitions, 2, "DenseNet" -> {dense depth, block repetitions}, {4,2},

"Inception" -> {inception width, block repetitions}, {4,2}, "U2Net"-> {Unet depth, downscale}, {5, True}.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`NeuralNetworkTools`SettingSchedule

^

Symbol i

TargetDevice is an option for certain functions that specifies on which device the computation should be attempted.

Documentation [Local](#) » | [Web](#) »

Attributes {Protected, ReadProtected}

Full Name System`TargetDevice

^

Symbol i

TestRun is an option for PrepareTrainingData. If set to True the data is not saved only analyzed.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`TestRun

^

Symbol i

TrainVoxelSize is an option for PrepareTrainingData. It defines the voxel size of the training data. When set to Automatic the voxel size is that of the data.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`SegmentationTools`TrainVoxelSize

^

ShapeTools

Functions

Symbol i

ApplyShapeModel[model, std] applies the std to the shape model to create the points. The model is made by MakeShapeModel, the std is obtained by FitShapeModel. The std can be a number, vector or matrix.

ApplyShapeModel[model, std, n] does the same but std is a number and n the integer indicating which vector to be used.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ApplyShapeModel] = {ArgumentsPattern → {_, _ ...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`ApplyShapeModel

^

Symbol i

EvaluateModel[points, mesh] return a dynamic plot showing an evaluation of the shape model.

The points and mesh are obtained from TemplateToVolume.

EvaluateModel[{mean, mat}, points, cells] does the same but for a explicit shape model given by mean and mat.

the shape model can be obtained from MakeShapeModel.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`EvaluateModel

^

Symbol i

FitShapeModel[{meanPoints, model}, points] fits the shape model to the points, the shape model is defined by meanPoints and model which are obtained form MakeShapeModel.

FitShapeModel[{meanPoints, model}, points, n] fits the model using the first n shape vectors from the model.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FitShapeModel] = {ArgumentsPattern → {_, _ ...}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`FitShapeModel

^

Symbol i

MakeMuscleTemplate[masks, vox] makes a muscle template from a series of masks. The output is the mean template muscle mask as a binary volume.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeMuscleTemplate] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MakeMuscleTemplate

^

Symbol i

MakeRegionMesh[data] makes a MeshRegion from a binary mask assuming an 1x1x1mm isotropic voxel size and 1000 mesh coordinates.

MakeRegionMesh[data, n] makes a MeshRegion with exactly n mesh coordinates.

MakeRegionMesh[data, vox] makes a MeshRegion taking in account the given voxel size vox with 1000 mesh coordinates.

MakeRegionMesh[data, vox, n] makes a MeshRegion taking in account the given voxel size vox with exactly n mesh coordinates.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeRegionMesh] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options MeshOutput → Mesh

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MakeRegionMesh

^

Symbol i

MakeShapeModel[points] makes the statistical shape model for the points given. The points per muscle must have the same number of coordinates.

The points can be obtained from TemplateToVolume. The output is {meanPoints, model} which are needed in FitShapeModel.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeShapeModel] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MakeShapeModel

^

Symbol



MakeTemplatePlot[meshes, points] visualizes the template alignment. The input meshes and points are made by TemplateToVolume with the TemplateOutput set to "Meshes".

Documentation [Local »](#)

Default Definitions SyntaxInformation[MakeTemplatePlot] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MakeTemplatePlot



Symbol



MaskFromDistanceMap[dist] converts a mask distance map created with MaskToDistanceMap back to a binary mask.

MaskFromDistanceMap[dist, method] converts a mask distance map created with MaskToDistanceMap back to a binary mask. The method can be "both", "out" or "in" which calculates the mask distance inside and outside, outside, or inside the mask.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MaskFromDistanceMap] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MaskFromDistanceMap



Symbol



MaskToDistanceMap[mask] converts a binary mask into a mask distance map.

MaskToDistanceMap[mask, method] converts a binary mask into a distance mask. The method can be "both", "out" or "in" which calculates the mask distance inside and outside, outside, or inside the mask.

Documentation [Local »](#)

Default Definitions SyntaxInformation[MaskToDistanceMap] = {ArgumentsPattern → {_, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MaskToDistanceMap



Symbol



`MeshGridPlot[meshes, points]` gives a grid of shape meshes defined by meshes and points. These can be obtained from with the `TemplateOutput` set to "Meshes".

Documentation [Local »](#)Options `MeshesPerRow` → 5

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ShapeTools`MeshGridPlot`

Symbol



`PlotMesh[mesh]` Plots a `MeshRegion` as a `GraphicsComplex`.

`PlotMesh[points, cells]` plots a `MeshRegion` which has been split into the coordinates and cells using `SplitRegionMesh`.

Documentation [Local »](#)Default Definitions `SyntaxInformation[PlotMesh] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`Options [»](#) `MeshOpacity` → 1 ... (5 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ShapeTools`PlotMesh`

Symbol



`PlotShapeVariation[points, mesh]` plots the template mesh which shows where the model has variation for each principal component. The needed points and mesh are obtained from `TemplateToVolume`.

`PlotShapeVariation[points, mesh, n]` does the same but only shows the first `n` principal components.

Documentation [Local »](#)Options `MeshesPerRow` → 5

Attributes {Protected, ReadProtected}

Full Name `QMRITools`ShapeTools`PlotShapeVariation`

Symbol i

ScaleToVolume[mask, vox] rescales a series of mask all to the same volume, which will be the median volume of all the masks.

ScaleToVolume[data, vox, vol] does the same but then rescales to the target volume vol.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ScaleToVolume] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`ScaleToVolume

^

Symbol i

SplitRegionMesh[mesh] splits the mesh in the MeshCoordinates and the mesh cells.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SplitRegionMesh] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`SplitRegionMesh

^

Symbol i

TemplateToVolume[{masks, voxM}, {template, voxT}] registers a mask or a series of masks to the given muscle template.

The default output are the mesh points per mask volume and the cell needed to generate the mesh as {points, mesh}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TemplateToVolume] = {ArgumentsPattern → {{_, _}, {_, _}, OptionsPattern[]}`

Options ▶ MeshPoints → {500, 2000} ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`TemplateToVolume

^

Options

Symbol i

MeshColor is an option for PlotMesh. It specifies the color of the mesh. The color can be a single color or a list of colors with the same length as the mesh coordinates.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshColor

^

Symbol 

MeshEdgeColor is an option for PlotMesh. It specifies the color for the border of the mesh vertices.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshEdgeColor

^

Symbol 

MeshesPerRow is and option for MeshGridPlot and PlotShapeVariation. It specifies how many volumes are shown per row.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshesPerRow

^

Symbol 

MeshOpacity is an option for PlotMesh. It specifies how much opacity the mesh has.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshOpacity

^

Symbol 

MeshOutput is an option for MakeRegionMesh. Values can be "Mesh", "Points", "Cells", "PointsCells", "All".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshOutput

^

Symbol 

MeshPointColor is an option for PlotMesh. It specifies the color of the mesh coordinates. The points need to be one and the points are shown as spheres.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshPointColor

^

Symbol i

MeshPoints is an option for TemplateToVolume. It specifies how many mesh coordinates will be used in the template.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshPoints

^

Symbol i

MeshPointSize is an option for PlotMesh. It specifies how large the points will be, its defined in mm.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`MeshPointSize

^

Symbol i

Monitor[*expr*, *mon*] generates a temporary monitor cell in which the continually updated current value of *mon* is displayed during the course of evaluation of *expr*.

Documentation [Local](#) » | [Web](#) »

Attributes {HoldAll, Protected, ReadProtected}

Full Name System`Monitor

^

Symbol i

SplineSpacing is an option for TemplateToVolume. It defines how big the b-splines are during the registration process. Its value is multiplied with the voxel size which then becomes the spline spacing.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`SplineSpacing

^

Symbol i

TemplateDilation is an option for TemplateToVolume. It specifies how much the template is dilated during registration. The value is specified in mm and it helps to properly align the points.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`TemplateDilation

^

Symbol i

TemplateOutput is an option for TemplateToVolume. The value can be "Meshes" "PointsCells" or "Points". The "Meshes" "PointsCells" outputs are needed for template and model visualization. The "Points" is the only thing needed for model fitting.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`TemplateOutput

^

Symbol i

TemplatePadding is an option for TemplateToVolume. It specifies how much the template will be padded to prevent cut off of larger muscles.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`TemplatePadding

^

Symbol i

VolumeRescale is an option for TemplateToVolume. If this option is set to True the volume of each muscle will be normalized to the mean volume of the template volumes or the template volume.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ShapeTools`VolumeRescale

^

SimulationTools

Functions

Symbol i

AddNoise[data, noise] adds rician noise to the data with a given sigma or SNR value.

Documentation [Local »](#)

Default Definitions SyntaxInformation[AddNoise] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options {NoiseSize → Sigma, NoiseType → Absolute}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`AddNoise

^

Symbol i

BlochSeries[vectorIn, deltai, freqRange, B1] performs a Bloch simulation of an RF pulse.

Documentation [Local](#) »

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`BlochSeries

^

Symbol i

CalculateGfactor[factors, sensitivity, wMat] calculates a gfactor for given sensitivity maps and noise correlation w. given the sense factors which is a list of three integers.

Documentation [Local](#) »

Default Definitions SyntaxInformation[CalculateGfactor] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}

Options GRegularization → 0.

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`CalculateGfactor

^

Symbol i

CreateDiffData[sig, eig, bvec, gradients, dim] creates a DTI datasets of dimensions dim with sig as unweighted signal s0 and bvec and gradients. eig can be {l1, l2, l3}, {{l1, l2, l3}, {e1, e2, e3}}, {{l1, l2, l3}, "Random"}, {{l1, l2, l3}, "RandomZ"} or {{l1, l2, l3}, "OrtRandom"}.

Uses Tensor internally.

CreateDiffData[] is based on DOI: 10.1002/nbm.2959.

Documentation [Local](#) »

Default Definitions SyntaxInformation[CreateDiffData] = {ArgumentsPattern → {_, _, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`CreateDiffData

^

Symbol i

ErnstAngle[] shows Ernst angle plot for $t_1 = 1400\text{ms}$ and $tr = 15\text{ ms}$.

ErnstAngle[t1] shows Ernst angle plot for t_1 and $tr = 15\text{ ms}$.

ErnstAngle[t1, tr] shows Ernst angle plot for t_1 and tr .

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ErnstAngle] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`ErnstAngle`

^

Symbol i

GESignal[ang, {tr, t1}] calculates the gradient echo signal for flipangles ang using tr and t1.

GESignal[ang_?ListQ, {{tr1_, tr2_}, t1_}] calculates the dual tr gradient echo signal for flipangles ang using tr1, tr2 and t1.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`GESignal`

^

Symbol i

GetPulseProfile[excitation, refocus] gives the pulsl angle profiles for the exitation and refocusing pulses.

a pulse is defined as {"name", flipangle, {G_strnth, Dur, BW}}.

GetPulseProfile[{"name", flipangle, {G_strnth, Dur, BW}}] gives detaile slice profile information of one pulse.

output is {ex_angle_profiel, ref_angel_profile, {plots}}.

output for single pulse is {{distance, Mt, Mz, Mx, My, ang, phase}, plots}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetPulseProfile] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options **>** `MagnetizationVector → {0, 0, 1} ... (4 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`GetPulseProfile`

^

Symbol i

GfactorSimulation[sensitivity, cov, {dir,sense}] calculates the gfactormaps for given sensitivity maps and noise corraltion cov in one direction.

The sensefactors are a list of integers in a given direction: "LR", "FH", or "AP".

GfactorSimulation[sensitivity, cov, {dir1,sense1}, {dir2,sense2}] calculates the gfactormaps for given sensitivity maps and noise corraltion w in two directions.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GfactorSimulation] = {ArgumentsPattern → {_, _, _, _, _}, OptionsPattern[]}`

Options `{GRegularization → 0., GOutput → Grid}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`GfactorSimulation`

^

Symbol i

PlotSimulation[par, xval, true, label, color] plots the pars (output form Parameters). Using label as PlotLabel and xval as x axis Thics. tr are the true parameter values. color are the color used for the plot.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSimulation] = {ArgumentsPattern → {_, _, _, _, _}, OptionsPattern[]}`

Options `PlotRange → {{0, 3}, {0, 3}, {0, 3}, {0, 3}, {0, 1}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`PlotSimulation`

^

Symbol i

PlotSimulationAngle[par, xdata, label, col] plots pars (output from Anlge Parameters).

Documentation [Local »](#)

Options `PlotRange → {0, 90}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`PlotSimulationAngle`

^

Symbol i

PlotSimulationAngleHist[par, label, xdata] plots pars (output from Anlge Parameters).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSimulationAngleHist] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SimulationTools`PlotSimulationAngleHist`

^

Symbol i

PlotSimulationHist[pars, label, xdata, tr] plots the pars (output form Parameters).
Using label as plotlabel and xdata as x axis label. tr are the true parameter values.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSimulationHist] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`PlotSimulationHist

^

Symbol i

PlotSimulationVec[tens, xdata, label] plots the eigenvectors from simulated tensors.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSimulationVec] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options SortVecs → True

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`PlotSimulationVec

^

Symbol i

Pulses[name] gives the pulse shape of some predefined Philips pulse shapes.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`Pulses

^

Symbol i

Signal[par, tr, te] calculates the MRI signal at a given tr and te. Par is defined as {pd, t1, t2}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Signal] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`Signal

^

Symbol i

SimAngleParameters[tens,vec] calculates the diffusion eigenvectors for tens compared to the true values vec.

The output can be used in PlotSimulationAngleHist and PlotSimulationAngle.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimAngleParameters] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SimAngleParameters

^

Symbol i

SimParameters[tens] calculates the diffusion parameters for tens. The output can be used in PlotSimulationHist and PlotSimulation.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimParameters] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options Reject → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SimParameters

^

Symbol i

SimulateDualTR[] simulates the signal of a Dual tr t1 map.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimulateDualTR] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SimulateDualTR

^

Symbol i

SimulateSliceEPG[exitation, refocus, {{t1, t2}, {nEcho, echoSp}, b1}] gives a simulated slice profile and EPG signal plot.

exitation and refocus are generated by GetPulseProfile.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SimulateSliceEPG] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options ReportFits → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SimulateSliceEPG

^

Symbol i

Tensor[{l1, l2, l3}] creates a diffusion tensor with vectors {{0,0,1},{0,1,0},{1,0,0}} and eigenvalues {l1, l2, l3}.

Tensor[{l1, l2, l3}, {e1, e2, e3}] creates a diffusion tensor with vectors {e1, e2, e3} and eigenvalues {l1, l2, l3}.

Tensor[{l1, l2, l3}, "Random"] creates a diffusion tensor with random orthogonal eigenvectors {e1, e2, e3} and eigenvalues {l1, l2, l3}.

Tensor[{l1, l2, l3}, "RandomZ"] creates a diffusion tensor with random orthogonal eigenvectors {{1,0,0}, e2, e3} with random eigenvectors and eigenvalues {l1, l2, l3}.

Tensor[{l1, l2, l3}, "OrtRandom"] creates a diffusion tensor with random orthogonal eigenvectors {{1,0,0},{0,1,0},{0,0,1}} and eigenvalues {l1, l2, l3}.

Tensor[] is based on DOI: 10.1002/nbm.2959.

Documentation [Local »](#)

Default Definitions SyntaxInformation[Tensor] = {ArgumentsPattern → {_, _}, OptionsPattern[]}

Options TensOutput → Vector

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`Tensor

^

Options

Symbol i

FatFieldStrength is an option for GetPulseProfile. If the value >0 it will calculate the shift of the fat refocusing pulse compared to the fat excitation pulse.

The shift is in SliceRangeSamples steps.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`FatFieldStrength

^

Symbol i

GOutput is an option for GfactorSimulation. can be "Grid" or "List".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`GOutput

^

Symbol i

GRegularization is an option for CalculateGfactor and GfactorSimulation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`GRegularization

^

Symbol i

MagnetizationVector is an option for GetPulseProfile. It defines the start magnetization vector for the bloch simulation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`MagnetizationVector

^

Symbol i

NoiseSize is an option for AddNoise. Values can be "Sigma", then the noise sigma is given or "SNR", then the SNR is given.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`NoiseSize

^

Symbol i

NoiseType is an option for AddNoise. Values can be "Absolute" or "Complex", and will add either Rician absolute noise or complex noise to the data.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`NoiseType

^

Symbol i

PlotRange is an option for graphics functions that specifies what range of coordinates to include in a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotRange

^

Symbol i

Reject is an option for EigenvalCalc. It True then voxels with negative eigenvalues are rejected and set to 0.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`Reject

^

Symbol i

ReportFits is an option for SimulateSliceEPG. If True it also reports the fit values.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`ReportFits

^

Symbol i

SliceRange is an option for GetPulseProfile. It specifies over which range the slice profile is generated (in mm). the total profile is 2xSliceRange.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SliceRange

^

Symbol i

SliceRangeSamples is an option for GetPulseProfile. defines how many samples are used to generate half a puls profile.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SliceRangeSamples

^

Symbol i

SortVecs is an option for PlotSimulationVec.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`SortVecs

^

Symbol i

TensOutput is an option for Tensor. Values can be "Vector" or "Matrix".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SimulationTools`TensOutput

^

SpectroTools

Functions

Symbol i

ApodizeEcho[echo] performs apodization on the echo. The apodization function is set with the option ApodizationFunction.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ApodizeEcho] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `ApodizationFunction → Hanning`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`ApodizeEcho`

^

Symbol i

ApodizeFid[fid] performs apodization on the fid. The apodization function is set with the option ApodizationFunction.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ApodizeFid] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `ApodizationFunction → Hanning`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`ApodizeFid`

^

Symbol i

ApodizePadEcho[echo] performs apodization on the echo and pads the echo with zeros to increase its length.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ApodizePadEcho] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{ApodizationFunction → Hanning, PaddingFactor → 2}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`ApodizePadEcho`

^

Symbol i

ApodizePadFid[fid] performs apodization on the fid and pads the fid with zeros to increase its length.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ApodizePadFid] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{ApodizationFunction → Hanning, PaddingFactor → 2}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`ApodizePadFid`

^

Symbol i

ApodizePadSpectra[spec] and doubles the number of spectral points while maintaining the dwell time.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[ApodizePadSpectra] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options {ApodizationFunction → Hanning, PaddingFactor → 2, ReadoutType → Fid}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ApodizePadSpectra

^

Symbol i

ApodizeSpectra[spec] performs apodization of the spectra. The apodization function is set with the option ApodizationFunction.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[ApodizeSpectra] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options {ApodizationFunction → Hanning, ReadoutType → Fid}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ApodizeSpectra

^

Symbol i

ChangeDwellTimeFid[fid, dt, dtnew] changes the sampling time of an fid from dwelltime dt to dwelltime dtnew.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[ChangeDwellTimeFid] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ChangeDwellTimeFid

^

Symbol i

CompareFidFitPlot[time, fidPlot, fitPlot] function not done.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[CompareFidFitPlot] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`CompareFidFitPlot

^

Symbol i

CompareSpectraFitPlot[ppmPl, specPlot, fitPlot] function not done.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CompareSpectraFitPlot] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`CompareSpectraFitPlot

^

Symbol i

CorrectTEFid[fid, dw, te] corrects the fid for 1st order phase by extrapolating the missing FID samples in the TE using Henkel matrix SVD analysis.

CorrectTEFid[fid, dw, te, gyro, ppmRan] corrects the fid for 1st order phase by extrapolating the missing FID samples in the TE using Henkel matrix SVD analysis. Only the part of the spectra in the ppmRan is used for optimization.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CorrectTEFid] = {ArgumentsPattern → {_, _, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`CorrectTEFid

^

Symbol i

CorrectTESpec[spectra, dw, te] corrects the spectra for 1st order phase by extrapolating the missing FID samples in the TE using Henkel matrix SVD analysis.

CorrectTESpec[spectra, dw, te, gyro, ppmRan] corrects the spectra for 1st order phase by extrapolating the missing FID samples in the TE using Henkel matrix SVD analysis. Only the part of the spectra in the ppmRan is used for optimization.

Documentation [Local »](#)

Default Definitions SyntaxInformation[CorrectTESpec] = {ArgumentsPattern → {_, _, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`CorrectTESpec

^

Symbol i

CSIIInterface[] opens the CSI interface. Function not done.

CSIIInterface[te, bw] opens the CSI interface with known te and bw.

CSIIInterface[file] opens the CSI interface with the data from file loaded.

CSIIInterface[file, {tei, bw}] opens the CSI interface with the data from file loaded with known te and bw.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[CSIIInterface] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `{SpectraFieldStrength → 7, SpectraNucleus → 31P}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`CSIIInterface`

^

Symbol i

ExportSparSdat[file, specs, {bw, te}, {gyro, nuc}] exports specs to file. Function not done.

Documentation [Local](#) »

Default Definitions `SyntaxInformation[ExportSparSdat] = {ArgumentsPattern → {_, _ {_, _}, {_, _}, _}, OptionsPattern[]}`

Options `{SparName → QMRITools, SparOrientation → {0, 0}, SparID → }`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`ExportSparSdat`

^

Symbol i

FindSpectraPpmShift[spectra, {dw, gyro}, peaks] finds the ppm value that aligns the spectra with the given peak positions peaks wich is a list of ppm values.

FindSpectraPpmShift[spectra, {dw, gyro}, {peaks, amps}] finds the ppm value that aligns the spectra with the given peak positions peaks wich is a list of ppm values and amps are ther relative amplitudes.

FindSpectraPpmShift[spectra, {dw, gyro}, specTar] finds the ppm value that aligns the spectra with the given target spectra specTar.

Documentation [Local](#) »

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`FindSpectraPpmShift`

^

Symbol i

FitSpectra[specBasis, spec, {st,end}, dt, {lwvals,lwamsp}] Fits the basis spectra from GetSpectraBasisFunctions to the spec over the ppm range {st, end} and dt the dwelltime.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FitSpectra] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}`

Options ▶ `SpectraNucleus → 1H ... (10 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`FitSpectra`

^

Symbol i

FitSpectraResultTable[parFit, parsF, names, ref, out] function not done.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FitSpectraResultTable] = {ArgumentsPattern → {_, _, _, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`FitSpectraResultTable`

^

Symbol i

GetGyro[nuc, field] geth the gyromagnetic ratio with field the field strength in Tesla and nuc the nucleus available in GyromagneticRatio.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetGyro] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`GetGyro`

^

Symbol i

GetPpmRange[spec, {dt, field, nuc}] get the ppm values of the spec where dt is the well time in ms, field the field strength in Tesla and nuc the nucleus available in GyromagneticRatio.

GetPpmRange[spec, dt, field, nuc] get the ppm values of the spec.

GetPpmRange[spec, dt, gyro] get the ppm values of the spec.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetPpmRange] = {ArgumentsPattern → {_, _, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`GetPpmRange`

^

Symbol



`GetSpectraBasisFunctions[{met1, ..., metn}]` generates a list of spectra basis functions with names met1 to metn. The names are strings and are the metabolites available in `GetSpinSystem`.

`GetSpectraBasisFunctions[{{props1}, ..., {propsn}}]` generates a list of spectra basis functions with properties prop1 to propn. The properties are those specified in `MakeSpinSystem`.

`GetSpectraBasisFunctions[inp, split]` generates a list of spectra basisfunctions. Each metabolite name present in the list split will be split in individual spectra per peak.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetSpectraBasisFunctions] = {ArgumentsPattern → {_, _, OptionsPattern[]}}`

Options [►](#) `BasisSequence → {PulseAcquire, 0} ... (6 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`GetSpectraBasisFunctions`



Symbol



`GetTimePpmRange[spec, {dt, field, nuc}]` get the timing of the fid and the ppm values

of the spec where dt is the well time in ms, field the field strength in Tesla and nuc the nucleus available in `GyromagneticRatio`.

`GetTimePpmRange[spec, dt, field, nuc]` get the timing of the fid and the ppm values of the spec.

`GetTimePpmRange[spec, dt, gyro]` get the timing of the fid and the ppm values of the spec.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetTimePpmRange] = {ArgumentsPattern → {_, _, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`GetTimePpmRange`



Symbol



`GetTimeRange[fid, dt]` get the timing of the fid where dt is the well time in ms.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetTimeRange] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`GetTimeRange`



Symbol i

ImportSparSdat[fspar, fsdat] imports fspar and fsdata file. Function not done.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportSparSdat] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SpectroTools`ImportSparSdat`

^

Symbol i

MakeSpectraResultPlot[ppmF, specF, {fit, basisFit}, names, sc, met] function not done.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MakeSpectraResultPlot] = {ArgumentsPattern → {_, _, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SpectroTools`MakeSpectraResultPlot`

^

Symbol i

PadEcho[echo] pads the echo with zeros to increase its length.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PadEcho] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `PaddingFactor → 2`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SpectroTools`PadEcho`

^

Symbol i

PadFid[fid] pads the fid with zeros to increase its length.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PadFid] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `PaddingFactor → 2`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`SpectroTools`PadFid`

^

Symbol 

PadSpectra[spec] doubles the number of spectral points while maintainig the dwell time.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PadSpectra] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{PaddingFactor → 2, ReadoutType → Fid}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`PadSpectra`

^

Symbol 

PhaseCorrectSpectra[spec] performs 0th order phase correction of the spectra by minimizing the difference between the real and absolute spectra velaue.

PhaseCorrectSpectra[spec, dw] performs 0th order phase correction of the spectra using Henkel matrix SVD fitting.

PhaseCorrectSpectra[spec, dw, te] := performs 0th and 1st order phase correction of the spectra using Henkel matrix SVD fitting. The first order phase is corrected by padding the fid with the missing values in the time befroe the TE.

PhaseCorrectSpectra[spec, dw, te, gyro, ppmRan] performs 0th and 1st order phase correction of the spectra using Henkel matrix SVD fitting. Only the part of the spectra in the ppmRan is used for optimization.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PhaseCorrectSpectra] = {ArgumentsPattern → {_, _, _, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`PhaseCorrectSpectra`

^

Symbol 

PhaseShiftFid[spectra, phi0] aplies the 0th order phase phi0 to the FID.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PhaseShiftFid] = {ArgumentsPattern → {_, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`SpectroTools`PhaseShiftFid`

^

Symbol i

PhaseShiftSpectra[spectra, phi0] applies the 0th order phase phi0 to the spectra.

PhaseShiftSpectra[spectra, ppm, gyro, phi1] applies the 1st order phase phi1 to the spectra. The ppm can be obtained using GetPpmRange and gyro with GetGyro.

PhaseShiftSpectra[spec, ppm, gyro, {phi0, phi1}] applies the 0th and 1st order phases {phi0, phi1} to the spectra. The ppm can be obtained using GetPpmRange and gyro with GetGyro.

The 0th order phase phi0 is in radians and the 1st order phase phi1 is in ms.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PhaseShiftSpectra] = {ArgumentsPattern -> {_, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`PhaseShiftSpectra

^

Symbol i

PlotCSIData[spectra, {dwell, gyro}] plots the CSI spectra which has dimensions {z,y,x,nsamp}. The ppm axes is determined by dwell and gyro. Gyro can be obtained with GetGyro.

PlotCSIData[spectra, {dwell, field, nuc}] plots the CSI spectra which has dimensions {z,y,x,nsamp}. The ppm axes is determined by dwell and field and nuc.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotCSIData] = {ArgumentsPattern -> {_, _, _, _, OptionsPattern[]}}`

Options PlotRange -> Full

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`PlotCSIData

^

Symbol i

PlotFid[fid, dwell] plots the fid assuming dwell as the sampling time.

PlotFid[time, fid] plot the fid where time is the timing of the fid which can be obtained with GetTimeRange.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotFid] = {ArgumentsPattern -> {_, _, OptionsPattern[]}}`

Options > PlotRange -> Full ... (8 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`PlotFid

^

Symbol i

PlotSpectra[spectra, {dwell, gyro}] plots the spectra, the ppm axes is determined by dwell and gyro. Gyro can be obtained with GetGyro.

PlotSpectra[spectra {dwell, field, nuc}] plots the spectra, the ppm axes is determined by dwell field and nuc.

PlotSpectra[ppm, spectra] plots the spectra where ppm is the ppm range of the spectra which can be obtained with GetPpmRange.

Documentation [Local »](#)

Default Definitions SyntaxInformation[PlotSpectra] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ➤ PlotRange → Full ... (12 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`PlotSpectra

^

Symbol i

ReadjMRUI[file] read a jMRUI spectrum file.

Output is the {time, spec, {begintime, samplingInterval}}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ReadjMRUI] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ReadjMRUI

^

Symbol i

ShiftSpectra[spectra, {dw, gyro}, shift] shifts the spectra by shift. The shift is in ppm.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ShiftSpectra] = {ArgumentsPattern → {_, _, OptionsPattern[]}}

Options ReadoutType → Fid

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ShiftSpectra

^

Symbol i

SpectraFitResult[spec, {fit, basisFit}, te, {dw, gyro}, {pars, names, metRef, log}, plots, OptionsPattern[]] function not done.

Documentation [Local »](#)

Default Definitions SyntaxInformation[SpectraFitResult] = {ArgumentsPattern → {_, _, _, _, _, OptionsPattern[]}}

Options PlotRange → Full

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraFitResult

^

Symbol i

SpectraSimulator[] opens a interactive spectra simulator.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraSimulator

^

Symbol i

TimeShiftEcho[fid, time, gam] applies a line broadening with linewidth gam and a Voigt line shape to the fid. The time can be obtained using GetTimeRange.

TTimeShiftEcho[fid, time, {gam, f}] applies a line broadening with linewidth gam and a custom line shape f to the fid (f=0, "Gaussian", f=1 "Lorentzian").

TTimeShiftEcho[fid, time, gyro, {gam, eps}] applies a line broadening with linewidth gam to the fid and a phase eps that results in eps ppm shift of the spectra. The gyro can be obtained with GetGyro.

TTimeShiftEcho[fid, time, gyro, {gam, eps, f}] applies a line broadening with linewidth gam using a custom line shape f to the fid and a phase eps that results in eps ppm shift of the spectra.

The linewidth gam is given in ms and the spectra shift eps is given in ppm.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TimeShiftEcho] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`TimeShiftEcho

^

Symbol



TimeShiftFid[fid, time, gam] applies a line broadening with linewidth gam and a Voigt line shape to the fid. The time can be obtained using GetTimeRange.

TimeShiftFid[fid, time, {gam, f}] applies a line broadening with linewidth gam and a custom line shape f to the fid (f=0, "Gaussian", f=1 "Lorentzian").

TimeShiftFid[fid, time, gyro, {gam, eps}] applies a line broadening with linewidth gam to the fid and a phase eps that results in eps ppm shift of the spectra. The gyro can be obtained with GetGyro.

TimeShiftFid[fid, time, gyro, {gam, eps, f}] applies a line broadening with linewidth gam using a custom line shape f to the fid and a phase eps that results in eps ppm shift of the spectra.

The linewidth gam is given in ms and the spectra shift eps is given in ppm.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TimeShiftFid] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`TimeShiftFid



Symbol



TimeShiftFidV[fid, time, gam] applies a line broadening with linewidth gam and a Voigt line shape to the fid. The time can be obtained using GetTimeRange.

TimeShiftFidV[fid, time, {gamL, gamG}] applies a line broadening with linewidth gamG "Gaussian" and gamL "Lorentzian".

TimeShiftFidV[fid, time, gyro, {gam, eps}] applies a line broadening with linewidth gam to the fid and a phase eps that results in eps ppm shift of the spectra. The gyro can be obtained with GetGyro.

TimeShiftFidV[fid, time, gyro, {{gamL, gamG}, eps}] applies a line broadening with

linewidth linewidth gamG "Gaussian" and gamL "Lorentzian" to the fid and a phase eps that results in eps ppm shift of the spectra.

The linewidth gam is given in ms and the spectra shift eps is given in ppm.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TimeShiftFidV] = {ArgumentsPattern → {_, _, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`TimeShiftFidV



Options

Symbol



ApodizationFunction is an options for ApodizeFid, ApodizeSpectra, ApodizePadFid, and ApodizePadSpectra. Values can be "Hanning", "Hamming", "Gaussian", "Lorentzian", and "Voigt".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ApodizationFunction



Symbol 

AspectRatio is an option for Graphics and related functions that specifies the ratio of height to width for a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`AspectRatio

^

Symbol 

BasisSequence is an option for GetSpectraBasisFunctions and specifies which sequence to use.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`BasisSequence

^

Symbol 

CenterFrequency is an option for GetSpinSystem and defines the center frequency in ppm.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`JcouplingTools`CenterFrequency

^

Symbol 

Filling is an option for ListPlot, Plot, Plot3D, and related functions that specifies what filling to add under points, curves, and surfaces.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Filling

^

Symbol 

FineTuneFit is an option for FitSpectra and when True it performs a second fitting run where for each peak is an individual linewidth, line shape and shift are fitted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`FineTuneFit

^

Symbol i

FitLineShape is an option for FitSpectra and when True allows to fit the lineshap. If False a voigt line shape is used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`FitLineShape

^

Symbol i

GridLines is an option for two-dimensional graphics functions that specifies grid lines.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`GridLines

^

Symbol i

GridLineSpacing is an option of TransmuralPlot. It defines the spacing of the grid lines.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`CardiacTools`GridLineSpacing

^

Symbol i

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol i

InitializeFit is an option for FitSpectra and is used to set initila values for the global fit {gami,epsi,{phi0i,phi1i},line shape}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`InitializeFit

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`Method

^

Symbol i

PaddingFactor is an option for PadFid, PadSpectra, ApodizePadFid, ApodizePadSpectra and FitSpectra. It Specifies with which factro to lengthen the fid.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`SpectroTools`PaddingFactor

^

Symbol i

PlotColor is an option for GradientPlot can be any color or gradient color name.

Documentation [Local »](#)
Attributes {Protected, ReadProtected}
Full Name QMRITools`PlottingTools`PlotColor

^

Symbol i

PlotLabel is an option for graphics functions that specifies an overall label for a plot.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected}
Full Name System`PlotLabel

^

Symbol i

PlotLabels is an option for visualization functions that specifies what labels to use for each data source.

Documentation [Local »](#) | [Web »](#)
Attributes {Protected, ReadProtected}
Full Name System`PlotLabels

^

Symbol 

PlotRange is an option for graphics functions that specifies what range of coordinates to include in a plot.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected, ReadProtected}

Full Name System`PlotRange

^

Symbol 

ReadoutType is an option for FitSpectra and padding and apodization functions. Value can be "Fid" or "Echo".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`ReadoutType

^

Symbol 

SparID is an option for ExportSparSdat.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SparID

^

Symbol 

SparName is an option for ExportSparSdat.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SparName

^

Symbol 

SparOrientation is an option for ExportSparSdat.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SparOrientation

^

Symbol 

SpectraBandwidth is an option for GetSpectraBasisFunctions and sets the bandwidth of the spectra.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraBandwidth

^

Symbol 

SpectraFieldStrength is an option for GetSpectraBasisFunctions and FitSpectra and sets the field strenght at which the simulations and fitting is performd.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraFieldStrength

^

Symbol 

SpectraNucleus is an option for GetSpectraBasisFunctions and FitSpectra and specifies which nucleus to Simulate or fit, see GyromagneticRatio.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraNucleus

^

Symbol 

SpectraOutputPlots is an option for FitSpectra. If True the automatica calibration plot for the initial fit are generated.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraOutputPlots

^

Symbol 

SpectraPpmShift is an option for GetSpectraBasisFunctions and FitSpectra and defines how much the center frequency is shifted, default is water at 4.65 ppm.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraPpmShift

^

Symbol i

SpectraSamples is an option for GetSpectraBasisFunctions and sets the number of samples in the spectra.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraSamples

^

Symbol i

SpectraSpacing is an option for PlotSpectra and defines the amount of spacing between spectra when multiple spectra are plotted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SpectraSpacing

^

Symbol i

SplineSpacingFactor is an option for FitSpectra and defines the distance between the bsplien points relative the the mean linewidthd of the peaks.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`SpectroTools`SplineSpacingFactor

^

TaggingTools

Functions

Symbol i

AnalyzeTagging[gridC] work in progress...

Documentation [Local »](#)

Options {HistoryWeighting → 0.7, MonitorTagging → True}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TaggingTools`AnalyzeTagging

^

Symbol i

CalculateDisplacementParameters[{motx, moty}, mask] work in progress...

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TaggingTools`CalculateDisplacementParameters

^

Options

Symbol i

HistoryWeighting is an options for AnnalyzeTagging.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TaggingTools`HistoryWeighting

^

Symbol i

MonitorTagging is an options for AnnalyzeTagging.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TaggingTools`MonitorTagging

^

TensorTools

Functions

Symbol i

ADCCalc[eigenvalues] calculates the ADC from the given eigenvalues.

Documentation [Local »](#)

Default Definitions SyntaxInformation[ADCCalc] = {ArgumentsPattern → {_}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`ADCCalc

^

Symbol i

AngleCalc[data, vector] calculates the angel between the vector and the data. Data should be an array of dimensions {xxx,3}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[AngleCalc] = {ArgumentsPattern → {_ , OptionsPattern[]}}

Options Distribution → 0–180

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`AngleCalc

^

Symbol



AngleMap[data] calculates the zennith and azimuth angles of a 3D dataset (z,x,y,3) containing vectors relative to the slice direction.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[AngleMap] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRTTools`TensorTools`AngleMap`



Symbol



ConcatenateDiffusionData[{data1, ..., dataN}, {grad1, ..., gradN}, {bval, ..., bvalN}, {vox, ..., voxN}] concatenates the diffusion data sets.

ConcatenateDiffusionData[{data1, ..., dataN}, {grad1, ..., gradN}, {bval, ..., bvalN}, {vox, ..., voxN}] concatenates the diffusion data sets.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ConcatenateDiffusionData] = {ArgumentsPattern → {_, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRTTools`TensorTools`ConcatenateDiffusionData`



Symbol



Correct[data, phase, shiftpar] corrects the dataset data using the phase map and the shiftpar and interpolation order 1.

Correct[data, phase, shiftpar, int] corrects the dataset data using the phase map and the shiftpar and interpolation order int.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Correct] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRTTools`TensorTools`Correct`



Symbol



CreateRPBMDictionary[tm] creates a dictionary of RPBM functions for the given mixing times tm.

Options are the used parameters in the dictionary and the simulated RD values, {pars, sim}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[CreateRPBMDictionary] = {ArgumentsPattern → {_, _, _}}`

Options {RPBMRangeTau → {100, 2000}, RPBMRangeZeta → {0.1, 3}, RPBMDictionarySize → {100, 1000}}

Attributes {Protected, ReadProtected}

Full Name `QMRTTools`TensorTools`CreateRPBMDictionary`



Symbol i

Deriv[disp, vox] calculates the derivative of the displacement along the three main axes. disp is the displacement field, vox is the voxel size.

Deriv[disp, vox, mask] calculates the derivative of the displacement along the three main axes. Sharp edges between the background and disp are solved by the mask. mask is a mask defining the edge of the displacement field.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Deriv] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`Deriv

^

Symbol i

DriftCorrect[data, bval] drift corrects the data using the signals of the lowest bvalue that has 6 or more unique volumes.

For the function to work optimal it is best to have these volumes evenly spread throughout the data and for the first and last volume to have this low bvalue.

DriftCorrect[] is based on DOI: 10.1002/mrm.26124.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[DriftCorrect] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options {NormalizeSignal → True, UseMask → True}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`DriftCorrect

^

Symbol i

ECalc[eigenvalues] calculates the E from the given eigenvalues.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ECalc] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`ECalc

^

Symbol 

EigensysCalc[tensor] calculates the eigensystem for the given tensor.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EigensysCalc] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{RejectMap → False, Reject → True, PerformanceGoal → Quality}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`EigensysCalc`

^

Symbol 

EigenvalCalc[tensor] calculates the eigenvalues for the given tensor.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EigenvalCalc] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{RejectMap → False, Reject → True, PerformanceGoal → Quality}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`EigenvalCalc`

^

Symbol 

EigenvecCalc[tensor] calculates the eigenvectors for the given tensor.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EigenvecCalc] = {ArgumentsPattern → {_, OptionsPattern[]}}`

Options `{RejectMap → False, Reject → True, PerformanceGoal → Quality}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`EigenvecCalc`

^

Symbol 

ExpTensor[tensor] transforms the tensor from LogEuclidean space.

ExpTensor[] is based on DOI: 10.1109/42.963816.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExpTensor] = {ArgumentsPattern → {_}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`ExpTensor`

^

Symbol 

FACalc[eigenvalues] calculates the FA from the given eigenvalues.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FACalc] = {ArgumentsPattern -> {  }}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`FACalc`

^

Symbol 

FitRPBMDictionary[sig, {pars, sim}, snr] fits the RPBM function to the simulated data sig using the parameters pars and the simulated data sim.

The needed dictionary {pars, sim} is generated using CreateRPBMDictionary. The snr is the estimated snr values of the data.

FitRPBMDictionary[sig, {pars, sim}, snr, d0] does the same but fixes the initial diffusion to the given d0.

Documentation [Local »](#)

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`FitRPBMDictionary`

^

Symbol 

FitRPBMFunction[tms, dat] fits the RPBM function to the data dat using the mixing times tms using FindMinimum.

The output is a rule of the fitted parameters {d0->val, tau-> tau, zeta ->zeta}.

FitRPBMFunction[tms, dat, init] does the same but uses init as initial guess values.

FitRPBMFunction[tms, dat, init, fix] does the same but uses init as initial guess values and fixes the parameters in fix.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FitRPBMFunction] = {ArgumentsPattern -> {  ,   ,   ,   }}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TensorTools`FitRPBMFunction`

^

Symbol i

FlipGradientOrientation[grad, perm] permutes the internal orientation of the gradients, perm can be any permutation of {"x","y","z"}.

FlipGradientOrientation[grad, flip] flips the internal orientation of the gradients, flip can be {1,1,1}, {-1,1,1}, {1,-1,1} or {1,1,-1}.

FlipGradientOrientation[grad, flip, perm] flips and permutes the internal orientation of the gradients.

FlipGradientOrientation[grad, perm, flip] flips and permutes the internal orientation of the gradients.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`FlipGradientOrientation

^

Symbol i

FlipTensorOrientation[tens, perm] permutes the internal orientation of the tensor, perm can be any permutation of {"x","y","z"}.

FlipTensorOrientation[tens, flip] flips the internal orientation of the tensor, flip can be {1,1,1}, {-1,1,1}, {1,-1,1} or {1,1,-1}.

FlipTensorOrientation[tens, flip, perm] flips and permutes the internal orientation of the tensor.

FlipTensorOrientation[tens, perm, flip] flips and permutes the internal orientation of the tensor.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FlipTensorOrientation] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`FlipTensorOrientation

^

Symbol i

GetRPBMValues[{d0, tau, zeta}] derives parameters from RPB function. d0, tau and zeta are the parameters of the RPB function.

Output is a list containing {d0, tau, zeta, dinf, td, tr, l, sv, a, kappa}.

Documentation [Local »](#)

Default Definitions SyntaxInformation[GetRPBMValues] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`GetRPBMValues

^

Symbol 

LogTensor[tensor] transforms the tensor to LogEuclidean space.

LogTensor[] is based on DOI: 10.1109/42.963816.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[LogTensor] = {ArgumentsPattern → {  }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`LogTensor

^

Symbol 

ParameterCalc[tensor] calculates the eigenvalues and MD and FA from the given tensor. The parameters are l1, l2, l3, MD and FA. l1, l2, l3, MD are in (10^{-3} mm²/s).

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ParameterCalc] = {ArgumentsPattern → {  , OptionsPattern[]}}`

Options {Reject → False, PerformanceGoal → Quality}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`ParameterCalc

^

Symbol 

Removelsolimages[data, grad, bval] Removes the ISO images from the philips scanner from the data. ISO images have g={0,0,0} and b>0.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[Removelsolimages] = {ArgumentsPattern → {  ,   ,   }}`

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`Removelsolimages

^

Symbol i

ResidualCalc[dti,{tensor,s0},gradients,bvector] calculates the tensor residuals for the given dataset.

ResidualCalc[dti,{tensor,s0},outlier,gradients,bvector] calculates the tensor residuals for the given dataset taking in account the outliers.

ResidualCalc[dti,{tensor,s0},bmat] calculates the tensor residuals for the given dataset.

ResidualCalc[dti,{tensor,s0},outlier,bmat] calculates the tensor residuals for the given dataset taking in account the outliers.

ResidualCalc[dti,tensor,gradients,bvector] calculates the tensor residuals for the given dataset. Tensor must contain Log[s0].

ResidualCalc[dti,tensor,outlier,gradients,bvector] calculates the tensor residuals for the given dataset taking in account the outliers. Tensor must contain Log[s0].

ResidualCalc[dti,tensor,bmat] calculates the tensor residuals for the given dataset. Tensor must contain Log[s0].

ResidualCalc[dti,tensor,outlier,bmat] calculates the tensor residuals for the given dataset taking in account the outliers. Tensor must contain Log[s0].

Documentation [Local »](#)

Default Definitions SyntaxInformation[ResidualCalc] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options MeanRes → All

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`ResidualCalc

^

Symbol i

RPBMFunction[tm, {d0, tau, zeta}] generates the RPBM function for the given mixing times tm, with d0, tau and zeta.

RPBMFunction[tm, {tau, zeta}] generates the RPBM function for the given mixing times tm, assuming a d0 of 1.

RPBMFunction[tm, {{d0, tau, zeta}, ..}] generates the RPBM function for the given mixing times tm, for a list of d0, tau and zeta.

Documentation [Local »](#)

Default Definitions SyntaxInformation[RPBMFunction] = {ArgumentsPattern → {_, ..}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RPBMFunction

^

Symbol i

SigmaCalc[dti,grad,bvec] calculates the noise sigma based on the tensor residual, using a blur factor of 10.

SigmaCalc[dti,tens,grad,bvec] calculates the noise sigma based on the tensor residual, using a blur factor of 10.

SigmaCalc[dti,grad,bvec,blur] calculates the noise sigma based on the tensor residual, If blur is 1 there is no blurring.

SigmaCalc[dti,tens,grad,bvec,blur] calculates the noise sigma based on the tensor residual. If blur is 1 there is no blurring.

Documentation [Local](#) »

Default Definitions SyntaxInformation[SigmaCalc] = {ArgumentsPattern → {_, _, _, _, _}, OptionsPattern[]}}

Options FilterShape → Median

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`SigmaCalc

^

Symbol i

SortDiffusionData[data, grad, bval] sorts the diffusion datasets grad and bval for magnitude of bvalue.

Documentation [Local](#) »

Default Definitions SyntaxInformation[SortDiffusionData] = {ArgumentsPattern → {_, _, _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`SortDiffusionData

^

Symbol



TensorCalc[data, gradients, bvalue] calculates the diffusion tensor for the given dataset. Allows for one unweighted image and one b value.

Gradient directions must be in the form {{x1,y1,z1}, ..., {xn,yn,zn}} without the unweighted gradient direction.

bvalue is a single number indicating the b-value used.

TensorCalc[data, gradients, bvec] calculates the diffusion tensor for the given dataset. allows for multiple unweighted images and multiple bvalues.

allows for different tensor fitting methods. gradient directions must be in the form {{x1,y1,z1}, ..., {xn,yn,zn}} with the unweighted direction as {0,0,0}.

bvec the b-vector, with a bvalue defined for each gradient direction. b value for unweighted images is 0.

TensorCalc[data, bmatrix] calculates the diffusion tensor for the given dataset. allows for multiple unweighted images and multiple bvalues.

bmat is the bmatrix which can be generated using Bmatrix.

The bvalue assumed to be is in s/mm^2 and therefore the output is in $\text{mm}^2/2$.

TensorCalc[] is based on DOI: 10.1016/j.neuroimage.2013.05.028 and 10.1002/mrm.25165.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TensorCalc] = {ArgumentsPattern → {_, _, _, _}, OptionsPattern[]}

Options ▶ MonitorCalc → True ... (6 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`TensorCalc



Symbol



TensorCorrect[tensor, phase, shift, vox] corrects the tensor based on B0 field map. Can perform both translation and rotation of tensor.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TensorCorrect] = {ArgumentsPattern → {_, _, _, _, _}, OptionsPattern[]}

Options RotationCorrect → False

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`TensorCorrect



Symbol i

TransformTensor[tensor, disp, vox] corrects the tensor with voxel size vox based on the displacement field disp. The displacement field is the displacement in mm for each voxel location in x, y and z.

TransformTensor[] is based on DOI: 10.1109/42.963816.

Documentation [Local »](#)

Default Definitions SyntaxInformation[TransformTensor] = {ArgumentsPattern → {_, _ , _}}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`TransformTensor

^

Symbol i

WestinMeasures[eigenvalues] calculates the westin measures.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`WestinMeasures

^

Options

Symbol i

Distribution is an option for AngleCalc. values can be "0–180", "0–90" and "–90–90".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`Distribution

^

Symbol i

FilterShape is an option for SigmaCalc. Can be "Gaussian" or "Median".

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`FilterShape

^

Symbol 

FullOutput is an option for TensorCalc when using b-vector. When True also the s0 is given as output.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`FullOutput

^

Symbol 

MeanRes is an option for ResidualCalc. When True the root mean square of the residual is calculated.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`MeanRes

^

Symbol 

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol 

MonitorCalc is an option for many processing functions. When true the process of the calculation is shown.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`MonitorCalc

^

Symbol 

NormalizeSignal is an option for DriftCorrect.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`NormalizeSignal

^

Symbol i

Parallelize[*expr*] evaluates *expr* using automatic parallelization.

Definitions

```
Parallelize[Parallel`Evaluate`Private`expr_, Parallel`Evaluate`Private`opts : OptionsPattern[]] :=
Module[{Parallel`Evaluate`Private`res, Parallel`Evaluate`Private`ahead, Parallel`Evaluate`Private`fopts}, Parallel`Protected`tryRelaunch[];
If[Parallel`Protected`$seQ, Message[Parallelize::nopar];
Return[Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`fopts = Sequence @@ {Method → OptionValue[Method],
DistributedContexts → Parallel`Protected`DistOptCheck[Parallelize, OptionValue[DistributedContexts]], ProgressReporting → OptionValue[ProgressReporting]};
Parallel`Evaluate`Private`ahead = Replace[Head[Unevaluated[Parallel`Evaluate`Private`expr], Hold],
{Hold[Parallel`Evaluate`Private`s_Symbol] => Parallel`Evaluate`Private`s, _ => $Failed}];
With[{Parallel`Evaluate`Private`handler = Lookup[Parallel`Evaluate`$ParallelExtensions, Parallel`Evaluate`Private`ahead, $Failed]},
If[Parallel`Evaluate`Private`handler != $Failed, Return[Parallel`Evaluate`Private`handler[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts]]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`tryCombine[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`tryCombine, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`wrapAround[Parallel`Evaluate`Private`expr, Parallel`Evaluate`Private`fopts];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`wrapAround, Return[Parallel`Evaluate`Private`res]];
Parallel`Evaluate`Private`res = Parallel`Evaluate`Private`silentFail[Parallel`Evaluate`Private`expr];
If[Head[Parallel`Evaluate`Private`res] != Parallel`Evaluate`Private`silentFail, Return[Parallel`Evaluate`Private`res]];
If[Parallel`Evaluate`Private`$seqWarning, Message[Parallelize::nopar1, Parallel`Evaluate`Private`expr]];
Parallel`Evaluate`Private`expr]
```

Documentation [Local »](#) | [Web »](#)

Options {DistributedContexts => \$Context, Method → Automatic, ProgressReporting => \$ProgressReporting}

Attributes {HoldFirst, Protected}

Full Name System`Parallelize

^

Symbol i

PerformanceGoal is an option for various algorithmic and presentational functions that specifies what aspect of performance to try to optimize with Automatic settings for options.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PerformanceGoal

^

Symbol i

Reject is an option for EigenvalCalc. If True then voxels with negative eigenvalues are rejected and set to 0.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`Reject

^

Symbol i

RejectMap is an option for EigenvalCalc. If Reject is True and RejectMap is True both the eigenvalues as well as a map showing je rejected values is returned.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RejectMap

^

Symbol i

RobustFit is an option for TensorCalc. If true outliers will be rejected in the fit, only works with WLLS.

If FullOutput is given the outlier map is given.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RobustFit

^

Symbol i

RobustFitParameters is an option for TensorCalc. gives the threshold for stopping the iterations and the kappa for the outlier margin, {tr,kappa}.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RobustFitParameters

^

Symbol i

RotationCorrect is an option for TensorCorrect. Default is False. Is a tensor is deformed setting to True also the shear is accounted for by local rotation of the tensor.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RotationCorrect

^

Symbol 

RPBMDictionarySize is an option for CreateRPBMDictionary. Default is {100, 1000}.

The number of dictionaries and number of points in the dictionary.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RPBMDictionarySize

^

Symbol 

RPBMRangeTau is an option for CreateRPBMDictionary. Default is {100, 2000}. The range of the tau values used in the RPBM function.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RPBMRangeTau

^

Symbol 

RPBMRangeZeta is an option for CreateRPBMDictionary. Default is {0.1, 3}. The range of the zeta values used in the RPBM function.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TensorTools`RPBMRangeZeta

^

Symbol 

UseMask is a function for MeanSignal and DriftCorrect.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`ProcessingTools`UseMask

^

TractographyTools

Functions

Symbol 

EndpointDensityMap[tracts, vox, dim] makes an endpoint density map based on the tracts endpoints.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[EndpointDensityMap] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`EndpointDensityMap`

^

Symbol 

ExportTCK[file, tracts, vox, dim] exports the tracts to the given *.tck file and automatically generates the header.

ExportTCK[file, tracts, header] exports the tracts to the given *.tck file. The header can be the header string or the header association as given by ImportTCK.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportTCK] = {ArgumentsPattern → {_, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`ExportTCK`

^

Symbol 

ExportTracts[file, tracts, vox, dim, seeds] exports the tracts, vox, dim and seeds to *.trk file.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ExportTracts] = {ArgumentsPattern → {_, _, _, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`ExportTracts`

^

Symbol i

FiberTractography[tensor, vox] performs fiber tractography on the tensor with voxel dimensions vox.

FiberTractography[tensor, vox, {par, {min, max}}] performs fiber tractography on the tensor with voxel dimensions vox with additional stopping criteria par, where tracts are only generated between values of par min and max.

FiberTractography[tensor, vox, {{par, {min, max}}, ..}] performs fiber tractography on the tensor with voxel dimensions vox with multiple additional stopping criteria.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FiberTractography] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options ▶ `FiberLengthRange → {20, 500} ... (11 total)`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`FiberTractography`

^

Symbol i

FilterTractLength[tracts, {min, max}] filters the tracts based on the length of the tracts.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FilterTractLength] = {ArgumentsPattern → {_, _, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`FilterTractLength`

^

Symbol i

FilterTracts[tracts, vox, {select..}] filters the tracts based on the list of select criteria.

Select criteria are defined as {"logic", {"how", criteria}}.

The "logic" parameter can be "and", "or" and "not".

The "how" parameter can be:

- "x", "y", or "z" for slice selection, here criteria is a slice number
- "through" for selecting tract that go through a roi, here criteria is a 3D mask.
- "within" for selecting tract that fit fully within the roi, here criteria is a 3D mask.
- "partwithin" for selecting the part of the tracts that fall within the roi, here criteria is a 3D mask.

Any number of select criteria can be listed.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FilterTracts] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options FiberLengthRange → {20, 500}

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FilterTracts

^

Symbol i

FindTensorPermutation[tensor, vox] performs tractography for all tensor permutations and gives back the one that has the longest tracts.

FindTensorPermutation[tensor, vox, {par, {min, max}}] same but with additional stopping criteria par, where tracts are only generated between values of par min and max.

FindTensorPermutation[tensor, vox, {{par, {min, max}}, ..}] same but with multiple additional stopping criteria.

Output = {permutations, flips, plot}

FindTensorPermutation[] is based on DOI: 10.1016/j.media.2014.05.012.

Documentation [Local »](#)

Default Definitions SyntaxInformation[FindTensorPermutation] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}

Options ▶ FiberLengthRange → {20, 500} ... (7 total)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FindTensorPermutation

^

Symbol i

FitTracts[tract] fits a tract or a list of tracts, each defined as a list of {x, y, z} coordinates with a polynomial function.

FitTracts[tract, vox, dim] does the same but constrains all the tract coordinates to the volume defined by dim.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[FitTracts] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `FittingOrder → 5`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`FitTracts`

^

Symbol i

GetTractValues[tracts, parameter, vox] gets the value of the parameter map at each tract coordinate.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[GetTractValues] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Options `InterpolationOrder → 1`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`GetTractValues`

^

Symbol i

ImportTCK[file] imports a *.tck file. the output is {tracts, vox, header}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportTCK] = {ArgumentsPattern → {_, _}, OptionsPattern[]}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`ImportTCK`

^

Symbol i

ImportTracts[file] imports a *.trk file. It can contain {tracts, vox, dim, seeds}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ImportTracts] = {ArgumentsPattern → {_, _}}`

Attributes `{Protected, ReadProtected}`

Full Name `QMRITools`TractographyTools`ImportTracts`

^

Symbol i

MoveTracts[tracts, off] moves the tract coordinates by off, which is {x, y, z}.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[MoveTracts] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`MoveTracts`

^

Symbol i

PlotSegmentedTracts[tracts, segments, dim, vox] plots the tracts after segmenting each segment.

PlotSegmentedTracts[tracts, segments, bones, dim, vox] plots the tracts after segmenting each segment also rendering a bone volume.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotSegmentedTracts] = {ArgumentsPattern → {_, _, _, _, OptionsPattern[]}}`

Options ▶ `MaxTracts` → 5000 ... (11 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`PlotSegmentedTracts`

^

Symbol i

PlotTracts[tracts, vox] plots the tracts assuming a BoxRatio based on vox.

PlotTracts[tracts, vox, dim] plots the tracts assuming a BoxRatio based on vox with a PlotRange spanning the full dim.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[PlotTracts] = {ArgumentsPattern → {_, _, OptionsPattern[]}}`

Options ▶ `MaxTracts` → 2000 ... (10 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`PlotTracts`

^

Symbol i

ResampleTracts[tracts, n] resample each Tract to exactly n vertices.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[ResampleTracts] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`ResampleTracts`

^

Symbol i

RescaleTracts[tracts, sc] scales the tract coordinates by 1/sc, which is {x, y, z} or single number.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[RescaleTracts] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`RescaleTracts`

^

Symbol i

SeedDensityMap[seeds, vox, dim] makes a seed density map based on the seed locations.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SeedDensityMap] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`SeedDensityMap`

^

Symbol i

SegmentTracts[tracts, segments, vox, dim] segments the tracts based on segments.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[SegmentTracts] = {ArgumentsPattern → {_, _, _, OptionsPattern[]}}`

Options ▶ `FiberLengthRange` → {15, 500} ... (5 total)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`SegmentTracts`

^

Symbol i

TractAngle[tracts] calculates the angle of each tract segment with the z-plane.

TractAngle[tracts, v1] calculates the angle of each tract segment with the plane normal to v1.

TractAngle[tracts, {v1, v2}] calculates the elevation and azimuth angle of each tract segment with the plane normal to v1 and the azimuth angle in that plane relative to v2.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractAngle] = {ArgumentsPattern → {_, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractAngle`

^

Symbol i

TractAngleMap[tracts, vox, dim] makes a tract angle map based on the tracts angles with the z-plane.

TractAngleMap[tracts, v1, vox, dim] makes a tract angle map based on the tracts angles with the plane normal to v1.

TractAngleMap[tracts, {v1, v2}, vox, dim] makes a tract angle map based on the tracts elevation angles with the plane normal to v1 and the azimuth angle in that plane relative to v2.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractAngleMap] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractAngleMap`

^

Symbol i

TractCurvature[tracts] calculates the curvature of each tract segment.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractCurvature] = {ArgumentsPattern → {_}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractCurvature`

^

Symbol i

TractCurvatureMap[tracts, vox, dim] makes a tract curvature map based on the tracts curvature.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractCurvatureMap] = {ArgumentsPattern → {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractCurvatureMap`

^

Symbol i

TractDensityMap[tracts, vox, dim] makes a tract density map based on the tracts vertices.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractDensityMap] = {ArgumentsPattern → {_, _, _}, OptionsPattern[]}`

Options `NormalizeDensity → True`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractDensityMap`

^

Symbol i

TractLength[tracts] calculates the length of each tract.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractLength] = {ArgumentsPattern -> {}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractLength`

^

Symbol i

TractLengthMap[tracts, vox, dim] makes a tract length map based on the tracts lengths.

Documentation [Local »](#)

Default Definitions `SyntaxInformation[TractLengthMap] = {ArgumentsPattern -> {_, _, _}}`

Attributes {Protected, ReadProtected}

Full Name `QMRITools`TractographyTools`TractLengthMap`

^

Options

Symbol i

Boxed is an option for Graphics3D that specifies whether to draw the edges of the bounding box in a three-dimensional picture.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name `System`Boxed`

^

Symbol i

ColorFunction is an option for graphics functions that specifies a function to apply to determine colors of elements.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name `System`ColorFunction`

^

Symbol i

ContourOpacity is an option for PlotContour and PlotSegmentations. It specifies the opacity of the contour.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name `QMRITools`PlottingTools`ContourOpacity`

^

Symbol 

FiberAngle is an option for FiberTractography and specifies the allowed angle change per tract step.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FiberAngle

^

Symbol 

FiberLengthRange is an option for FiberTractography and specifies the allowed tract range.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FiberLengthRange

^

Symbol 

FittingOrder is an option for FitTracts. It specifies the polynomial order of the function to fit the tract.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FittingOrder

^

Symbol 

FitTractSegments is an option for SegmentTracts. If set True the segmented tracts are fitted with FitTracts.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`FitTractSegments

^

Symbol 

ImageSize is an option that specifies the overall size of an image to display for an object.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`ImageSize

^

Symbol i

InterpolationOrder is an option for Interpolation, as well as ListLinePlot, ListPlot3D, ListContourPlot, and related functions, that specifies what order of interpolation to use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`InterpolationOrder

^

Symbol i

MaxSeedPoints is an option for FiberTractography and defines the maximum number of seed points to be used.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`MaxSeedPoints

^

Symbol i

MaxTracts is an option for PlotTracts. It specifies how many tracts are plotted.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`MaxTracts

^

Symbol i

Method is an option for various algorithm-intensive functions that specifies what internal methods they should use.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Method

^

Symbol i

Monitor[*expr*, *mon*] generates a temporary monitor cell in which the continually updated current value of *mon* is displayed during the course of evaluation of *expr*.

Documentation [Local »](#) | [Web »](#)

Attributes {HoldAll, Protected, ReadProtected}

Full Name System`Monitor

^

Symbol i

NormalizeDensity is an option for TractDensityMap. If set True the tract density is normalized, if False then it is the true tract count.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`NormalizeDensity

^

Symbol i

OutputForm[*expr*] prints as a two-dimensional representation of *expr* using only keyboard characters.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`OutputForm

^

Symbol i

Parallelization is an option for Compile that specifies whether it should create a compiled function that could run in parallel.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`Parallelization

^

Symbol i

PerformanceGoal is an option for various algorithmic and presentational functions that specifies what aspect of performance to try to optimize with Automatic settings for options.

Documentation [Local »](#) | [Web »](#)

Attributes {Protected}

Full Name System`PerformanceGoal

^

Symbol i

StepSize is an option for FiberTractography and defines the tractography step size.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`StepSize

^

Symbol i

StopThreshold is an option for FiberTractography and defines the stop threshold which is a value between 0 and 1.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`StopThreshold

^

Symbol i

TensorFlips is an option for FiberTractography and specifies if the tensor orientation is flipped, see FlipTensorOrientation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TensorFlips

^

Symbol i

TensorPermutations is an option for FiberTractography and specifies if the tensor orientation is permuted, see FlipTensorOrientation.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TensorPermutations

^

Symbol i

TractColoring is an option for PlotTracts and sets how the tracts are colored. Values can be "Direction", "Length", "Angle", {par}, or RGBColor[]. For "Length", "Angle", {par} it can be defined in the form {..., {min, max}} where the {min, max} specifies the range of the color function.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TractColoring

^

Symbol i

TractMonitor is an option for FiberTractography. When set True it prints the progress.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TractMonitor

^

Symbol 

TractReduction is an option for PlotTracts. Value can be an Integer > 0, which determines with which factor the tract coordinates are sub sampled.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TractReduction

^

Symbol 

TractScaling is an option for PlotTracts. The value can be "World" or "Voxel", if the value is "World" the tracts are in mm else in voxel coordinates.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TractScaling

^

Symbol 

TractSize is an option for PlotTracts. When tubes are used it specifies the tube width.

Documentation [Local »](#)

Attributes {Protected, ReadProtected}

Full Name QMRITools`TractographyTools`TractSize

^